Computational Physics

Eitan Lees

2025-02-24

Table of contents

Preface

I first learned to program in 2011 in Dr. Richard O. Gray's computational physics course.

It was a revelatory experience that shaped my life. I enjoyed the class so much that I served as the TA the following year and later earned a PhD in Computational Science.

I've kept the lecture notes on my various computers ever since. I want to convert the notes into a Quarto book as a conservation project. In addition to replicating the source material I want to add some examples from Python and Scipy.

> **Note**
>
> This project is a work in progress. I will be releasing lectures out of order as I finish them. I will leave a note here when the project is complete.

Colophon

This is a book created from markdown and executable code.

See Knuth (1984) for additional discussion of literate programming.

To learn more about Quarto books, visit https://quarto.org/docs/books

1 Introduction to C – part I

In this class, we will be employing the modern "C" language which is commonly used for scientific programming[1]. "C" is closely related to C++ which is an Object Oriented language. It turns out that, while occasionally useful, the Object Oriented properties of C++ are not required for most types of scientific programming. In addition, acquiring a working knowledge of C++ can involve quite a steep learning curve. The object of this class is to get you up to speed as rapidly as possible in a powerful and versatile programming language (we have about three weeks to do that!) and then to introduce you to a number of numerical techniques useful and essential for work in physics and engineering. Some of you will already have a working knowledge of C or C++. If you don't, the following three chapters will get you started. Otherwise, consider them a quick refresher.

Read: *The C Programming Language*: Chapter 1.

1.1 Computer Memory and Variables

Basically speaking, a computer program is a series of statements that manipulate data stored in the computer's memory and yields a (hopefully useful) result. One can think of the memory of a computer as a series of boxes, each of which can contain one piece of data (a number, a character, etc.). Each of these boxes, or locations in memory, has its own **address**. A **variable** is a label which points to the address of a particular location in memory. It is very seldom necessary in scientific programming to know the *actual* location or address of a particular piece of data. However, in C programming, the concept of address is very important, although we will always reference a piece of data using a variable name.

There are a number of different types of variable in C. For instance, we may have to deal with integers (whole numbers, such as 1, 5, -32, etc.) in our program. These need to be distinguished from numbers that have decimal points (0.56, 2.934, 4.345e-20) called floating-point numbers. We may also have variables that refer to characters and to strings.

All variables in C must be *declared* or defined, and are generally of the types mentioned above, although we will have opportunity to use integer and floating point variables of different **precision**. For instance, we have two types of integers, single precision integers, referred to simply as **integer**, and double precision integers referred to as **long**. In addition, we have

[1] If individual errors σ_i are not given or specified, one can set $\sigma_i = 1$ in this and all equations in this chapter.

single and double precision floating point, **float** and **double**. Variables are declared at the beginning of the program as follows:

```
double x;       /* double precision floating point */
int i,j;        /* integers */
float y;        /* single precision floating point */
long n;         /* double precision integer */
char s;         /* character */
```

(Note – text in between `/*` and `*/` are comments and not part of the program. It is always a good idea to place plenty of comments in your program, but try to avoid being pedantic).

What is the difference between, for instance, a variable declared as a **float** and one that is declared as a **double**? Both are what we call floating point variables, but a double-precision floating point variable has twice the amount of memory allocated to it than a single-precision floating point variable. Since the computer uses twice as much memory to represent a double than a float, this means that the double is represented in memory with more precision (effectively, this means the number can be represented with more decimal places). One would thus use doubles instead of floats if one were concerned with the precision of the mathematics (for instance, if one needs to subtract two large numbers which are almost equal). Years ago, when computer memory was in short supply and expensive, it was advisable to declare most floating point variables as float, and reserve the double declaration only for variables which required the extra precision. At present, computers usually have so much memory that it is becoming routine to declare one's floating point variables as double unless one is working with exceptionally large amounts of data. The reason for this is that round-off error is occasionally troublesome with single-precision floating point variables.

It can be helpful to use certain unsigned types as well. For instance, on some old machines and compilers, the maximum range for an integer was from -32768 to 32768. An **unsigned int** variable on the same machine would range from 0 to 65536.

It is sometimes useful to declare **arrays** of variables. One-dimensional arrays may be declared as follows:

```
float r[5];
```

The components of this array (usually called a *vector* in C) can be accessed as `r[0]`, `r[1]`, `r[2]`, `r[3]` and `r[4]`. Notice that vectors in C generally begin with index 0, going up to $n-1$ where in the above example, $n = 5$.

Another way to declare a vector and allocate memory for it is as follows. First, declare a *pointer* to a vector:

```
float *r;
```

then, use the function `vector` to allocate memory for the vector:

```
r = vector(0,4);
```

These two steps will have exactly the same effect as the statement `float r[5];`. We will discuss later why using the second method may be preferable to the first. Note that `vector` is not a built-in C-function, such as `sin` or `cos` (see Section 1.3). However, this function is included in a file named `comphys.c` that your instructor has provided you. This means that whenever you want to use a function out of `comphys.c`, you will have to remember to place the following statements at the top of your program –

```
#include "comphys.h"
#include "comphys.c"
```

but more on that later!

Two-dimensional arrays may also be defined in a similar way, thus

```
float A[5][4];
```

will yield a matrix with the following components:

```
A[0][0], A[0][1], A[0][2], A[0][3],
A[1][0], A[1][1], A[1][2], A[1][3],
A[2][0], A[2][1], A[2][2], A[2][3],
A[3][0], A[3][1], A[3][2], A[3][3],
A[4][0], A[4][1], A[4][2], A[4][3]
```

and likewise there is a function in `comphys.c` called `matrix` which you can use to do the same thing, if you want.

A single character variable may be declared as follows:

```
char c;
```

although a string, i.e. a variable containing a number of characters, such as a word or multiple words of text will be declared as follows:

```
char str[80];
```

Here the variable `str` will be able to store text up to 80 characters long.

Occasionally one may have a value that is used commonly in a program that never gets changed, such as the value for π, or the value of the speed of light. In such cases, it is useful to employ a **macro** as follows:

```
#define PI 3.14159265
#define C  2.99792458e+08
```

Such macro definitions occur at the very beginning of the program. When compiling the program (a necessary step in producing an "executable" file), the compiler will replace any occurrence of `PI` in your code with `3.14159265` and `C` with `2.99792458e+08`. For this reason, it is customary to use uppercase letters in macros and then to write the rest of your code with lowercase letters.

An alternate way of doing essentially the same thing is by using the `const` declaration:

```
const double pi = 3.14159265;
const double c = 2.99792458e+08;
```

Read: *The C Programming Language*: pp. 35 – 41

1.2 Input and Output

Most programs need to input data and output results. We will discuss a number of ways to do this in class, but the simplest way to interact with a program is via the screen and the keyboard. Reading input from the keyboard is simple in C. Let us suppose we have declared two variables, a double `x` and an integer `k`, along with another integer variable `ni` with the following statements:

```
double x;
int k,ni;
```

and now want to input some values from the keyboard for these variables. We might use the following lines of code:

```
printf("\nEnter a value for x > ");
ni = scanf("%lf",&x);
printf("\nEnter a value for k > ");
ni = scanf("%d",&k);
```

The function `printf` prints a line to the screen – notice that we began the string "Enter a value for x >" with the line feed/carriage return control character `\n`. The program will wait for you to enter a value via the keyboard. Once you hit `<Enter>`, the function `scanf` will read the value you entered and "return" on success the number of items read (in this case 1). This number is placed in the integer variable `ni`[2]. We will discuss in detail later on how functions can "return" values. Note that the arguments for `scanf` include a formating statement (`%lf`) and the *address* of the variable you are reading in (`&x`). The function `scanf` always requires the address of the variable or variables (designated using `&` in front of the variable name), and not the variable name itself. The format statement `%lf` simply states that the variable is a **l**ong **f**loating point (double) variable. In the second `scanf` example, the variable is an integer, and so the formating statement `%d` is used. The function `scanf` can read in multiple variables; we will discuss how to do this later. Below, find some typical format statements for different types of variables:

Format	Type
`%f`	float
`%lf`	double
`%d`	integer
`%ld`	long (double precision integer)
`%e`	floating point entered in scientific notation - 1.523e-13
`%le`	double entered in scientific notation
`%g`	free format float
`%lg`	free format double
`%c`	character
`%s`	string

The `printf` statement may be used, with formating, to output values of variables to the screen. Let us suppose we wish to output the value of `x`, a variable that has been declared as a double and `k`, an integer. We may use the following statements:

```
printf("The value of x is %f and k is %d\n",x,k);
```

Notice in the `printf` statement, one need not differentiate in the formating between a double and a float. Also, `printf` takes the variable name, not the address, unlike `scanf`.

Now, if the value of `x` is 5.0 and the value of `k` is 4, the above statement will print as:

```
The value of x is 5.000000 and k is 4
```

[2]A set of m functions, $X_i(x)$ is called *linearly independent* if no function $X_j(x)$ in that set may be expressed as a linear combination of the other functions in that set. Thus, the set $1, x, x^2$ is a linearly independent set, as, for instance, x may not be expressed as a sum of 1 and x^2.

It might look nicer if the statement were printed as

```
The value of x is 5.0 and k is 4
```

This can be accomplished with the following `printf` statement:

```
printf("The value of x is %3.1f and k is %d\n",x,k);
```

Notice in the formating statement `%3.1f` the "3" stands for the total number of characters in the output ("5","."," 0"), including the decimal point, and the "1" stands for the number of digits after the decimal point. Lets see how to use this material in an actual computer program.

Example and Exercise 1.1: In the first lab, you learned how to compile a simple program, called "Hello World". The C-code for this program is as follows:

```
#include <stdio.h>

int main()
{
  printf("\nHello World!\n");
  return(0);
}
```

The significance of the different lines (for instance, the one with `#include <stdio.h>`) will be discussed in the next section. Let's now use our newfound knowledge of input/output to modify this simple "Hello World" program in an interesting way. Type the following into emacs, and save it under `hello2.c`:

```
#include <stdio.h>

int main()
{
  char name[30];
  int ni;

  printf("\nEnter your first name > ");
  ni = scanf("%s",name);

  printf("\nHello %s!\n",name);

  return(0);
}
```

compile it with the following command line:

```
gcc -o hello2 hello2.c
```

and then run it with the following command:

```
./hello2
```

Read: *The C Programming Language*: Chapter 7

1.3 Mathematics

The basic arithmetical operations are defined in the manner that you would expect, using, for instance, the operators `+`, `-`, `*`, `/`, `%`. The last operator, "%" is the modulus operator which gives the remainder from integer division. For instance,

```
z = 13 % 7        /* The result is 6 */
```

Exponentiation, however, is different in C than in many other computer languages. To raise a variable `x` to a power `y` and place the result in another variable `z`, the function `pow` is used, thus:

```
z = pow(x,y);
```

in addition, if one wants to add, subtract, multiply or divide and put the result back into the same variable, one can use some special C shortcuts:

```
x += 2.5;     /* same as x = x + 2.5; */
y -= 3;       /* same as y = y - 3;   */
z *= x;       /* same as z = z * x;   */
y /= z;       /* same as y = y/z;     */
```

A number of predefined mathematical functions can be used in C; some of the more common are listed in the table below.

Table 1.1: Commonly Used Math Functions in C

Function	Description
`pow(x,y)`	Raising to a power x^y
`fabs(x)`	Absolute value of x
`sqrt(x)`	Square root
`sin(x)`, `cos(x)`	Sine and Cosine of x
`tan(x)`	Tangent of x
`asin(x)`, etc.	Inverse trig functions
`exp(x)`	e^x
`log(x)`, `log10(x)`	Natural log, $\ln x$ and Common log, $\log_{10} x$
`floor(x)`	Round down to nearest integer
`ceil(x)`	Round up to the nearest integer

It is important to remember that when carrying out arithmetic with integers the computer truncates (i.e. rounds down to the nearest integer) the result. Thus, in floating point, `3.0/2.0 = 1.5` but in integer arithmetic `3/2 = 1`. If you mix types in an arithmetical statement, it is a good idea to *cast* the variables to a consistent type. For instance, let us suppose we have the following declarations:

```
double x=30.5,y;
int a=3;
```

and we want to calculate $y = x/a$. It is best to write this statement as

```
y = x/(double)a;
```

in this example, we have *cast* the type of `a` to be a double, that is to say, in this expression, `a` is represented as 3.0, not 3.

Read: *The C Programming Language*: Chapter 2; Appendix B4

1.4 A Simple Program

The following program may be typed into your programming environment:

```
#include <stdio.h>
#include <math.h>

int main()
```

```
{
  double wave,T;
  double p = 1.19106e+27;
  double p1;
  int ni;

  printf("\nEnter the wavelength in Angstroms > ");
  ni = scanf("%lf",&wave);

  printf("\nEnter the temperature in Kelvin > ");
  ni = scanf("%lf",&T);

  p1 = p/(pow(wave,5.0)*(exp(1.43879e+08/(wave*T)) - 1.0));

  printf("The Planck function at %7.2f A and %7.2f K is %e\n",
          wave,T,p1);
  return(0);
}
```

Save this program as `planck.c`. Before we learn how to compile and run this program, let us look at each line in detail.

```
#include <stdio.h>
#include <math.h>
```

These two lines include *header files* in the program. In the C language, each function (such as `exp`, `pow`) must be declared beforehand (we will discuss later what these declarations involve); for built-in functions, these declarations are contained in header files. In the case of math functions, the declarations are found in the header file `math.h`. In the case of input and output functions, such as `printf` and `scanf`, the declarations are found in `stdio.h`. Other common header files that you may have to use are `stdlib.h` and `string.h`. We will discuss which function declarations are found in those header files in class.

```
int main()
{
```

Every C program is composed of a number of *functions* which perform certain tasks. These functions (also known as routines or subroutines) always have a name and a *return type*. All C programs must have a `main` function. The `main` function by standard convention has a return type of `int`, which means that it returns an integer to the operating system. By convention, if a "0" is returned, then the program is assumed to have terminated normally without error. If

the program returns a "1", this indicates to the operating system that an error has occurred. Some compilers accept a `void` type for `main`. Some other functions (see lecture 3) will return a character, others a double, etc. Some functions have a `void` return type which means that they don't return anything! We will discuss this in more detail later. The bracket immediately below the `main` function declaration indicates the beginning of the function. The function will be ended with a close bracket.

```
double wave,T;
double p = 1.19106e+27;
double p1;
int ni;
```

These statements declare a number of variables used in the program. Note that the `p` variable is *initialized* with a certain value. Note also that in a program, all statements end in a semicolon ";". The exceptions to this rule are the `#include` statements and macro statements.

```
printf("\nEnter the wavelength in Angstroms > ");
ni = scanf("%lf",&wave);
printf("\nEnter the temperature in Kelvin > ");
ni = scanf("%lf",&T);
```

These are input statements similar to those we discussed before.

```
p1 = p/(pow(wave,5.0)*(exp(1.43879e+08/(wave*T)) - 1.0));
```

This is the mathematical statement that actually calculates the Planck function.

```
printf("The Planck function at %7.2f A and %7.2f K is %e\n",
        wave,T,p1);
```

This statement prints to the screen the value of the Planck function for the input values of the wavelength and the temperature.

```
return(0);
```

This is the *return* statement which returns the *return value* from the `main` function. Since `main` is an `int` function, an integer (0) is returned, which indicates normal termination of the program.

```
}
```

Finally, all C functions end with a close-bracket.

This program may be compiled with the following command:

```
gcc -o planck planck.c -lm
```

The flag `-lm` must be included in the `gcc` command whenever a program includes a math function, such as, in this case, `exp` and `pow`. This flag "links" in the math library. Run the program using the command `./planck` and enter some reasonable values such as `wave = 5000` and `T = 6000`.

Exercise 1.2

Write a short program that will prompt the user for an initial velocity v_0 and final time, and then calculate and output to the screen the velocity at the final time for an object subject to a gravitational acceleration of $g = -9.8$ m/s^2. The following is the relevant equation, familiar to you from introductory physics:

$$v = v_0 + gt$$

Hint: you can get `scanf` to read in two variables simultaneously in the following way:

```
ni = scanf("%lf,%lf",&v0,&tf);
```

Note that the two formating codes are separated by a comma. This means that the two variables must be entered at the prompt separated by a comma, for instance `100.0,0.0`. You should alert the user to the need for such a comma or "delimiter" in the prompting message.

Exercise 1.3

Magnetic Dipole
The magnetic field of the "pure" dipole in SI units is given as:

$$\vec{B}(r) = \left(\frac{\mu_0 m}{\pi r^3}\right)\left(2\cos\theta\hat{r} + \sin\theta\hat{\theta}\right)$$

where $\mu_0 = 4\pi \times 10^{-7}$ [NA^{-2}]. Assume that the magnitude of the magnetic dipole moment (m) is 1 [Am^2]. Query the user to provide a distance, r [in meters], and an angle, θ, [in degrees] and output the magnitude of the $\hat{r}$ and $\hat{\theta}$ components [in tesla = NA^{-1}m^{-1}] to the screen. Do you get what you'd expect for θ=0, 90, 180, and 270 degrees?
Here is some code to get you started. Notice that there is code that handles the divide-by-

zero error by asking the user to re-enter a value for r. This uses "loops" and "conditionals" which we will talk about very soon...just complete the code with the math statements required. HINT: `math.h` uses radians in its `sin` and `cos` functions. The user is entering degrees – you need to do a conversion before computing the results.

```
/* Bdip exercise
   YOUR NAME HERE  */
#include <stdio.h>
#include <math.h>

const double pi=3.141592653589793;

int main()
{
  double r=0.0;
  int ni;

/* MAKE THE REST OF YOUR DECLARATIONS HERE */

  printf("\nThis program will provide the r(hat) and theta(hat)
              components of the B field for a pure dipole.\n");

/* PROMPT THE USER FOR r AND theta HERE */

/* THE FOLLOWING CODE HANDLES THE DIVIDE BY ZERO POSSIBILITY */

  while (r == 0.0) {
    printf("\nEnter the distance, r > ");
    ni = scanf("%lf",&r);
    if (r == 0.0) {
      printf("\nr can not be zero, try again...\n\n");
    }
  }

/* PUT THE REST OF YOUR CODE HERE */

  return(0);
}
```

Exercise 1.4

Electrostatic Forces: Coulomb's law is given by the equation:

$$\vec{F}_{ab} = \frac{Q_a Q_b}{4\pi\epsilon_0 r^2}\hat{r}_{ab}$$

where $\hat{r}_{ab}$ is a unit vector that points from charge Q_a to charge Q_b and r is the distance between the two charges. The charges are given in coulombs, and ϵ_0, the *permittivity of free space* has the following value:

$$\epsilon_0 = 8.85418782 \times 10^{-12} \text{ farad/meter}$$

A charge $Q_1 = 4.54$ coulombs is situated at the x, y coordinates $(0, 0)$, and another charge, $Q_2 = 3.883$ coulombs is situated at point (x, y) to be specified by the user. Write a program that will prompt the user for the coordinates of Q_2 and will then compute the x and y components of the coulomb force on Q_2. Here is some code to get you started:

```c
/* Coulomb exercise
   YOUR NAME HERE  */
#include <stdio.h>
#include <math.h>

const double pi=3.141592653589793;

int main()
{
  double x=0.0,y=0.0;
  double Q1=4.54,Q2=3.883;
  int ni = 0;
  /* Make the rest of your declarations here */

  printf("\nThis program will print the x and y components of the Coulomb
         force on charge Q2\n");

/* Here is some code that will help the program to deal with the situation
   where the user inputs coordinates (0,0) for charge Q2, which would
   lead to a divide by zero error. */

  while(x == 0.0 && y == 0.0) {
    printf("\nEnter x,y coordinates for Q2, avoiding 0,0 > ");
    ni = scanf("%lf,%lf",&x,&y);
    if(x == 0.0 && y == 0.0)
      printf("\nBoth x and y cannot be 0.0; please try again");
  }

/* Compute the two components of the force here. Remember that like
   charges repel! */

  return(0);
}
```

1.5 Introduction to Python

Why hello there! It's me Eitan. I am going to be popping in at the end of these lectures to give you my two cents on the subjects covered and how to use Python for related tasks. In 2011 Dr.Gray's computational physics course was done entirely in C. While I am thankful to have started my programming journey in C, these days I first turn to Python for all of my scientific

computing needs.

Python is an *interpreted language* meaning there is no compilation. In C we write code, compile it, and then run the binary. In Python we write code and then send that code to the Python interpreter.

You can play around with the Python interpreter directly by typeing `python` on the command line (assuming you have installed Python correctly).

I am almost never in an interpreter directly, instead I am writting scripts and running them. When you run `python my_script.py` you can think about it as feeding each line to the interpreter one line at a time.

While there is a way to define an entry point like `main` in Python I won't discuss it here. I like to think of my programs has running top down.

As an example let's revisit the Plank equation from the lecture

$$p_1 = \frac{p}{\lambda^5 \left(e^{\frac{1.43879 \times 10^8}{\lambda T}} - 1 \right)}$$

In Python you could write

```
from math import exp

p = 1.19106e27
wave = float(input("Enter the wavelength in Angstroms > "))
T = float(input("Enter the temperature in Kelvin > "))

p1 = p / (wave**5 * (exp(1.43879e08 / (wave * T)) - 1))
print(f"The Planck function at {wave} A and {T} K is {p1:.6e}")
```

When run this script we should get the same result from the lecture

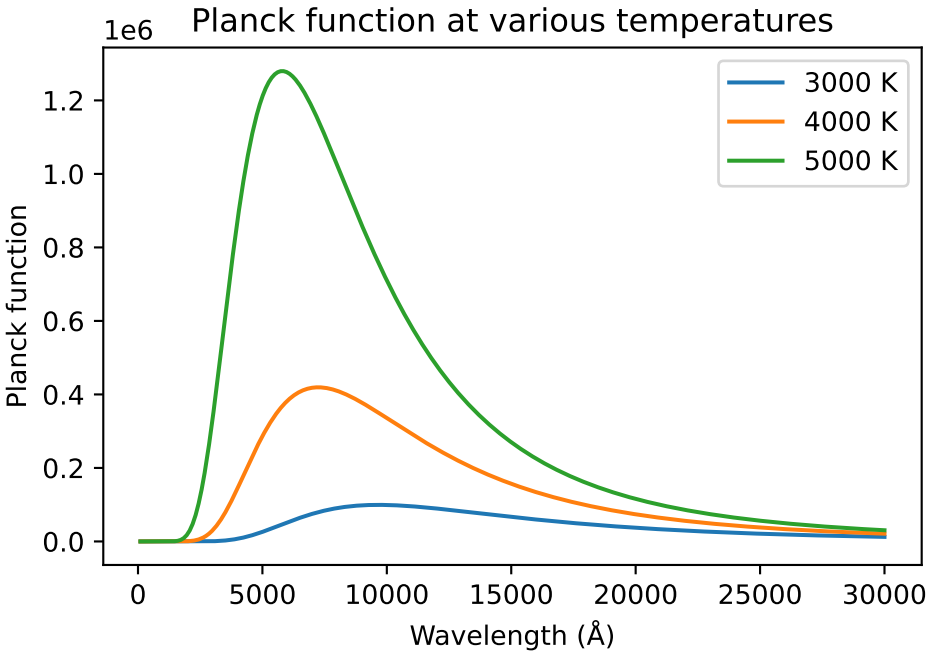
```
$ python planck.py
Enter the wavelength in Angstroms > 5000
Enter the temperature in Kelvin > 6000
The Planck function at 5000.0 A and 6000.0 K is 3.175596e+06
```

Python is *dynamically typed* so we don't need to be so explicit about every variable. The only gotcha here is that the `input` method always returns a string, so we have to cast that to a `float` to make use of it in our equation.

Dr. Gray metnioned the use of `comphys.c` which defines a *vectors* and a *matrix* object to bring our physics language in closer alignment with our programs. In Python the Numpy library does much of the heavy lifting in that respect.

One of the reasons I like Python is with a few lines of code you can generate publication quality results

```
import numpy as np
import matplotlib.pyplot as plt

def planck(wave, T):
    p = 1.19106e+27
    return p / (wave**5 * (np.exp(1.43879e+08/(wave*T)) - 1))

waves = np.linspace(100, 30000, 500)
temperatures = [3000, 4000, 5000]

fig, ax = plt.subplots()

for T in temperatures:
    plancks = planck(waves, T)
    ax.plot(waves, plancks, label=f"{T} K")

ax.set_xlabel("Wavelength (Å)")
ax.set_ylabel("Planck function")
ax.set_title("Planck function at various temperatures")
ax.legend()
plt.show()
```



More on that in due time! For now I recommned playing around. Try to redo the examples from this lecture in Python. There are a million "Intro to Python" resources online of varying quality. I highly recommend Whirlwind Tour Of Python by Jake VanderPlas. It is a good read for all skill levels.

2 Introduction to C – part II

2.1 Loops

The program `planck.c` that you compiled and ran last time can be used to compute the Planck function (blackbody spectrum) for a given temperature by inputing different values for the wavelength while holding the temperature constant. Instead of running the program multiple times, it is better to do this by introducing a loop into your program. Please read over the section in your textbook on Loops and Conditionals. Let us see how to modify `planck.c` to include a loop to make our life easier.

```
#include <stdio.h>
#include <math.h>

int main()
{
  double wave,T;
  double p = 1.19106e+27;
  double p1;
  int ni;

  printf("\nEnter the temperature in Kelvin > ");
  ni = scanf("%lf",&T);

  wave = 500.0;

  while(wave <= 12000.0) {
    p1 = p/(pow(wave,5.0)*(exp(1.43879e+08/(wave*T)) - 1.0));
    printf("The Planck function at %7.2f A and %7.2f K is %e\n",
            wave,T,p1);
    wave += 500.0;
  }
  return(0);
}
```

Here we have introduced a *while loop* to evaluate the Planck function at values of the wavelength between 500 and 12000 Å with an interval of 500 Å. The *while loop* begins with the statement

```
while(wave <= 12000.0) {
```

and ends with the close bracket

```
}
```

five lines down. What this means is that the statements in between these two brackets are executed in order multiple times until the condition `wave <= 12000.0` is no longer true. Notice that the *while loop* contains the statement `wave += 500.0;` which increments `wave` by 500 every time the statement is executed. Eventually `wave` will exceed 12000.0 and then the program will exit the loop.

Another type of loop is called a *do while loop*. We could replace the *while loop* above with the following statements:

```
  do {
    p1 = p/(pow(wave,5.0)*(exp(1.43879e+08/(wave*T)) - 1.0));
    printf("The Planck function at %7.2f A and %7.2f K is %e\n",
            wave,T,p1);
    wave += 500.0;
  } while(wave <= 12000.0)
```

This loop functions very similarly to the *while loop*, except that the condition `wave <= 12000.0` is evaluated at the end of the loop instead of the beginning. It turns out that *do while* loops are almost never used. However, every once in a while it proves to be useful to have the condition evaluated at the end of the loop, for instance when you want to guarantee that the loop will be executed at least once.

Another way of writing this program is to initialize `wave` as a vector and to use a *for loop*, as follows:

```
#include <stdio.h>
#include <math.h>

int main()
{
  double T;
  double p = 1.19106e+27;
  double p1;
  double wave[24] = {  500.0, 1000.0, 1500.0, 2000.0, 2500.0,
                      3000.0, 3500.0, 4000.0, 4500.0, 5000.0,
                      5500.0, 6000.0, 6500.0, 7000.0, 7500.0,
```

```
                        8000.0, 8500.0, 9000.0, 9500.0,10000.0,
                       10500.0,11000.0,11500.0,12000.0};
  int ni,i;

  printf("\nEnter the temperature in Kelvin > ");
  ni = scanf("%lf",&T);

  for(i=0;i<24;i++) {
    p1 = p/(pow(wave[i],5.0)*(exp(1.43879e+08/(wave[i]*T)) - 1.0));
    printf("The Planck function at %7.2f A and %7.2f K is %e\n",
            wave[i],T,p1);
  }
  return(0);
}
```

Notice how we have initialized the values for the vector `wave` in this example. The *for loop* begins with the `for(i=0;i<24;i++)`. This `for` statement says "initialize `i` to 0 and for each passage through the loop, increment `i` by 1 (`i++`). Keep cycling through the loop while `i < 24`. When `i = 24` exit the loop." Notice inside the loop that the different values in the `wave` vector are accessed with `wave[i]` as `i` is different for each passage through the loop.

All three examples should give identical output. In this application, the *for loop* example is probably unnecessarily complicated, but this method of initializing a vector can be quite useful, especially if the values of the vector are not readily computed. For instance, let us suppose we wanted to evaluate the Planck function at the wavelengths: 515.0, 912.0, 3123.0, 6615.0 …. In such a case we would have been forced to use a *for loop* because these values cannot be computed with an increment statement such as `wave += 500`. In addition, *for loops* are used when we want to control exactly the number of times the loop is executed.

Exercise 2.1

Write a program containing a *while loop* which will output the velocity of a ball thrown upwards with an initial velocity v_0 (which should be input to the program) moving under the influence of gravity. The program should output the velocity of the ball for times $t = 0$ to $t = 100$ seconds with an interval of 5 seconds.

Exercise 2.2

Accomplish the task for Exercise 2.1 using a *for loop*.

Exercise 2.3

A particle moves through space with the parametric equations

$$x = 5.0 + 0.12t^2$$

and

$$y = 0.02t^3$$

Write a program that will compute and output the (x, y) coordinates of the particle between $t = 0$ and $t = 5.0$ sec at steps of 0.5 sec. Use a *while loop* to do this.

Exercise 2.4

Accomplish the task for Exercise 2.3 using a *for loop*.

Read: *The C Programming Language*: Chapter 3 (sections 3.5 – 3.7)

2.2 Conditionals

In the previous section we were exposed to the use of conditionals in the *while* and *do while* loops. Generally speaking, a conditional is a statement which can be used to perform different actions depending upon whether a certain condition is evaluated to be either true or false. Conditionals are commonly used to cause the program to execute a certain block of statements when the condition is true, and another block of statements when false. The simplest way to accomplish this is through the *if* statement and its extension, the *if else* statement. Consider the following fragment of code to see how the *if* statement works:

```
if(n == 0) T = 5000;
```

This code states that if `n` is equal to 0, then `T` should be assigned the value 5000. But, what if `n` is not equal to 0? This is taken care of by the *if else* statement:

```
if(n == 0) T = 5000;
else T = 4000;
```

This clearly indicates that if `n` is not 0, then `T` should be assigned the value 4000. An *if else* statement can be extended with *else if* statements such as:

```
if(n == 0) T = 5000;
else if(n < 20) T = 4000;
else T = 3000;
```

It is up to the programmer to guarantee that the set of *if*, *else if* and *else* statements cover all possible contingencies. If an entire block of statements needs to be executed as the result of an *if*, *else* or *else if* statement, this block can be enclosed in parentheses:

```
if(n == 0) {
  T = 5000;
  x = 62.5*T;
}
```

The condition `n == 0` (which compares `n` to 0 – note the double `=` sign)[1] is called a *Boolean expression*. A Boolean expression evaluates to 1 if it is true and to 0 if it is false. So, if `n` is indeed 0, then `n == 0` takes on the value of 1. Else its value is 0. The table below lists the comparison operators that can be used in Boolean expressions.

Table 2.1: Comparison Expressions

Operator	Meaning
`==`	is equal to
`!=`	is not equal to
`>`	is greater than
`>=`	is greater than or equal to
`<`	is less than
`<=`	is less than or equal to

Boolean expressions can be combined using Boolean algebra. For instance, Boolean expressions can be combined using `&&` (and) and `||` (or). If two expressions are combined with `&&`, then both must be true for the entire expression to be true. If the two expressions are combined with `||` then only one of the expressions need be true for the entire expression to be true. More than two expressions can be combined using `&&` and `||` to give quite complex Boolean expressions. In the following examples, let `A = 10`, `B = 1` and `C = 5`. The character "!" is used to negate a statement.

Expression	Result
`A == 10 || B == 5`	True

[1] If individual errors σ_i are not given or specified, one can set $\sigma_i = 1$ in this and all equations in this chapter.

Expression	Result
`!(A == 10 || B == 5)`	False
`A == 10 && B == 5`	False
`A != 3 && B == 1`	True
`A >= C && B >= A`	False
`(A != 5 && B == 1) || C == 3`	True
`B`	True
`!B`	False

Exercise 2.5

Up until now, the variable `ni` in the expression `ni = scanf(...` has simply been thrown away. We can use it, however, to verify that the user has entered the correct number of variables. Write a program which prompts the user for numerical values for two variables, say v_0 and t_f. Use a conditional test on `ni` to check to see if two variables have, indeed, been entered, and if not, then to print an error message.

Exercise 2.6

Write a program that incorporates a certain key number which is unknown to the user. The program should prompt the user to input a guess for this key number, and then print to the screen whether the guess is less than, equal to, or greater than the key. The program should incorporate a loop so that the user can keep guessing until the correct number is found.

Exercise 2.7

Random numbers can be generated between 0.0 and 1.0 using the function `ran1` from the file `comphys.c`. The random number generator needs to be initialized, and this code fragment shows you how to do it:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "comphys.h"
#include "comphys.c"

int main()
{
  long idum = -1;
  time_t now;
  float x,rn;

  /* Use the time function to supply a different seed to the
     random number generator every time the program is run */

  now = time(NULL);

  idum = -1*now;

  /* Initialize random number generator */

  rn = ran1(&idum);

  /* Now you can generate any number of random numbers between 0.0
     and 1.0 with statements like the following: */

  x = ran1(&idum);
```

Write a program using this random number generator that will sort the random numbers x that you generate into 4 bins, one bin with $0 \leq x < 0.25$, the next with $0.25 \leq x < 0.50$, etc. Generate 1000 random numbers, and have the program count the number in each bin.

Exercise 2.8

In many applications, whether they are directly related to physics or not, one needs to sort data based on some criterion. This need arises quite often and, if not approached correctly, could consume a considerable amount of CPU time. There are many sorting applications out there, all of which rely on conditional statements to sort the data. A highly efficient and commonly used sorting routine is "quicksort.c".
The quicksort routine uses recursion to divide a given array into smaller and smaller

partitions, sorting as it goes. Quicksort and its subroutine, `partition.c`, are provided for you within the `comphys.c` library in two forms: `qsint`, which applies quicksort to integers, and `qsfloat`, which applies quicksort to floating point values:

```
void qsint( int[], int, int);
void qsfloat( float[], int, int);
```

The `qsint` routine is shown below, but remember, you don't need to add this code to your programs, all you need to do is place:

```
#include "comphys.h"
#include "comphys.c"
```

at the beginning of your program and then call `qsint` or `qsfloat` as you need them. For example, the following demo sorts integers. To compile it, just run:

```
gcc -o sort_demo sort_demo.c -lm
```

```
#include <stdio.h>
#include "comphys.h"
#include "comphys.c"

int main()
{
  int a[9]={7,12,1,-2,0,15,4,11,9};
  int a0[9];
  int i=0;

  for (i=0;i<=8;i++) a0[i]=a[i];

/* Here is where you call quicksort (for integers): */

  qsint(a,0,8);

/* Notice: you pass the array to be sorted, a, the lower limit index, 0
   and the upper limit index, 8 (in this case).
*/

  printf("\nQuicksort demo:\n");
  printf("\nOriginal    Sorted\n");
  for(i=0;i<9;++i) printf("%4d    %4d\n",a0[i],a[i]);
  return(0);
}
```

Here is the `qsint` code for your reference.

```
/* quicksort integers */
void qsint( int a[], int l, int r)
{
  int partint( int a[], int l, int r)
  int j;

  if( l < r )
  {
    j = partint( a, l, r);
    qsint( a, l, j-1);
    qsint( a, j+1, r);
  }
}

/* partition integers */
int partint( int a[], int l, int r) {
  int pivot, i, j, t;
  pivot = a[l];
  i = l; j = r+1;

  while( 1) {
    do ++i; while( a[i] <= pivot && i <= r );
    do --j; while( a[j] > pivot );
    if( i >= j ) break;
    t = a[i]; a[i] = a[j]; a[j] = t;
  }
  t = a[l]; a[l] = a[j]; a[j] = t;
  return j;
}
```

As an example of the use of quicksort, in earth surface physics, one often needs to determine the grain size distribution of grains in a given population. This is usually done to model the boundary layer behavior between a bed of grains and a fluid that is moving relative to that bed. The grain sizes beneath a bed, known as "granular surface roughness", will determine the drag experienced by the fluid as it passes overhead. Large grains induce turbulence, draining energy from the flow and slowing it down. Very small grains (or a smooth surface) may allow laminar flow, speeding it up.
In the following assignment, you will sort 50 grains whose size [in mm] is provided in the file `grains.dat`. You must sort the grains in two ways:

1. Write to screen the list of 50 grains sorted by size from smallest to largest.

2. Generate the number of grains that fall within each size bin, where the bins correspond to sizes 1.0–1.1mm, 1.1–1.2mm, … , 1.9–2.0mm. For grains that fall on a boundary, place them in the smaller size bin (for example, place a grain with size 1.3mm into the 1.2–1.3mm bin). You will output to the screen the number of grains per size bin.

3. Print the average grain size of the distribution.

4. Print the standard deviation of the size distribution, defined as

$$SD = \sqrt{\frac{1}{N}\sum (gs(i) - avegs)^2}$$

where gs=grainsize, avegs=average grain size, N=number of grains, and i is the grain index ranging from 1 to N.
FYI: Fluid models would then use the average grain size and the standard deviation to model the boundary layer. You don't need to use quicksort to obtain these statistics.
Note: this exercise requires you to read the data in `grains.dat` into your program, but we have not covered how to do that yet. This task can be accomplished in a quick and dirty way by including the following code in your program:

```
float grain[50];
int i,ni;

for(i=0;i<50;i++) ni = scanf("%f",&grain[i]);
.
.
.
```

and then, after compiling your program, running it with the command `./ex24 < grains.dat` where `ex24` is the name of your program. The use of "<" is called a pipe (see next section). We will see a better way to get file data into a program in the next lecture.

GRADUATE STUDENTS ONLY

5. Create a new routine `qsdouble` which applies quicksort to arguments that are of double precision. Include this in your versions of `comphys.h` and `comphys.c` and run part (1.) of this assignment with the grain sizes defined as double.

Exercise 2.9

A certain physical quantity S exhibits the property of *bifurcation* when a critical value of ϵ is reached. Below $\epsilon = 4.5553$, S is described by the equation:

$$S = 7.3329 + 1.1107\epsilon$$

Above $\epsilon = 4.5553$, S displays stocastic behavior, and has a 35.4% chance of following the branch described by

$$S = 12.3925 + (\epsilon - 4.5553) + 0.23(\epsilon - 4.5553)^2$$

and a 64.6% change of following the branch given by

$$S = 12.3925 + 4.9311(\epsilon - 4.5553) - 1.02(\epsilon - 4.5553)^2$$

Use the random number generator to generate 100 random values of ϵ between 0 and 10.0, and for each value of ϵ above the bifurcation point use the random number generator to simulate the stocastic property of S, printing out the value of S for each generated value of ϵ.

Read: *The C Programming Language*: Chapter 3 (sections 3.1 – 3.4)

2.3 An Aside – Command-line input and Piping

Sometimes it is useful to write your program so that all the input information can be introduced on the *command line*. For instance, take the example program in Section 2.4. This program prompts the user for a temperature, and then prints out the value of the Planck function for wavelengths between 500 and 12,000 Å. It would be nice to get the program to accept its input from the command line, because this then makes it possible to redirect the output from the program from the screen to a file. For instance, with the command line (entered at the Linux prompt):

```
./planck 7500.0 > planck.out
```

a suitably modified `planck` program would read in 7500K for the temperature, and then would *pipe* the output into the file `planck.out`. To make this possible, modify the program in Section 2.4 to read as follows:

```
#include <stdio.h>
#include <stdlib.h>  /* Required for the function atof */
#include <math.h>
```

```c
int main(int argc, char *argv[])
{
  double wave,T;
  double p = 1.19106e+27;
  double p1;
  int ni;

  if(argc != 2) {
    printf("\nEnter the temperature in Kelvin > ");
    ni = scanf("%lf",&T);
  } else T = atof(argv[1]);

  wave = 500.0;

  while(wave <= 12000.0) {
    p1 = p/(pow(wave,5.0)*(exp(1.43879e+08/(wave*T)) - 1.0));
    printf("%7.2f %e\n",wave,p1);  /* Note modification here */
    wave += 500.0;
  }
  return(0);
}
```

How does this work? The parameters to the `main` function `int argc, char *argv[]` allow the program to read input from the command line. If the program sees no input on the command line (for instance, if the command line looks simply like `./planck`), then `argc` is set to 1. If there is one input item on the command line (for instance, `./planck 7500`), then `argc` is set to 2, etc. So, the above program reads the number of input items on the command line. If `argc == 2`, then the program reads the temperature `T` from `argv[1]` (`argv[0]` is, by default, always set to the name of the program, in this case `planck`). If, however, `argc != 2`, the program prompts the user for the temperature. Try it! Then use gnuplot to plot the Planck function using the output file `planck.out`, using the gnuplot command:

```
gnuplot> plot "planck.out" using 1:2 with lines
```

Exercise 2.10

Use piping to redirect the output from the program you wrote for Exercise 2.9 into a file `bifurcation.dat`. Then use gnuplot to plot the output. To see the behavior of the function more clearly, increase the number of points generated to 1000.

Read: *The C Programming Language*: Section 5.10

2.4 Programming Aids – Pseudocode and Flow Charts

Many people, when first introduced to programming, have problems with seeing the "big picture", that is how a program should be set out logically, and then how to translate that logical structure into actual computer code. There are two programming aids that can help in this process – pseudocode and flow charts. Lets begin with pseudocode, as it is simpler to understand.

Pseudocode simply lays out the programming steps in more-or-less normal language. Take as an example how to compute $n!$. First, the program has to read the value of n from the keyboard (user prompt), and then set up a loop to calculate:

$$n! = 1 \cdot 2 \cdot 3 \cdot ... \cdot n$$

Here is how we might write the pseudocode:

```
prompt user for value of n
read value for n
set x = 1
set f = 1
enter loop
  if x is greater than n, drop out of loop
  else f = f * x
  increment x by 1
go back to the top of the loop
print f
end
```

It is then pretty easy to translate this into actual C-code:

```c
#include <stdio.h>

int main()
{
  double n,x=1.0,f=1.0;
  int ni;

  printf("\nEnter value for n (an integer) > ");
  ni = scanf("%lf",&n);

  while(x <= n) {
    f *= x;
```

```
    x += 1.0;
  }
  printf("n! = %5.0f\n",f);
  return(0);
}
```

This same program structure can be illustrated with a flowchart:

where the diagonal box is known as a "decision box", which corresponds to the conditional in the `while` statement (which asks, is $x > n$?). The *while* loop itself is represented by the arrow moving to the right from "No", which leads to execution of $f = f * x$ and $x = x + 1$ followed by a return to the top of the decision box, when the question "is $x > n$?" asked again.

The *if*, *else if*, *else* set of conditionals, as represented by the example below:

```
if(n == 0) T = 50
else if(n == 1) T = 100
else if(n == 2) T = 200
else T = 0
```

can be represented in flowchart form by a series of decision boxes:

Flowchart diagrams can be quite useful in reasoning out a complex program, but may be overkill (once programming is learned) for simple programs. We don't have time to go into more detail on flowcharting, but there are resources on the web.

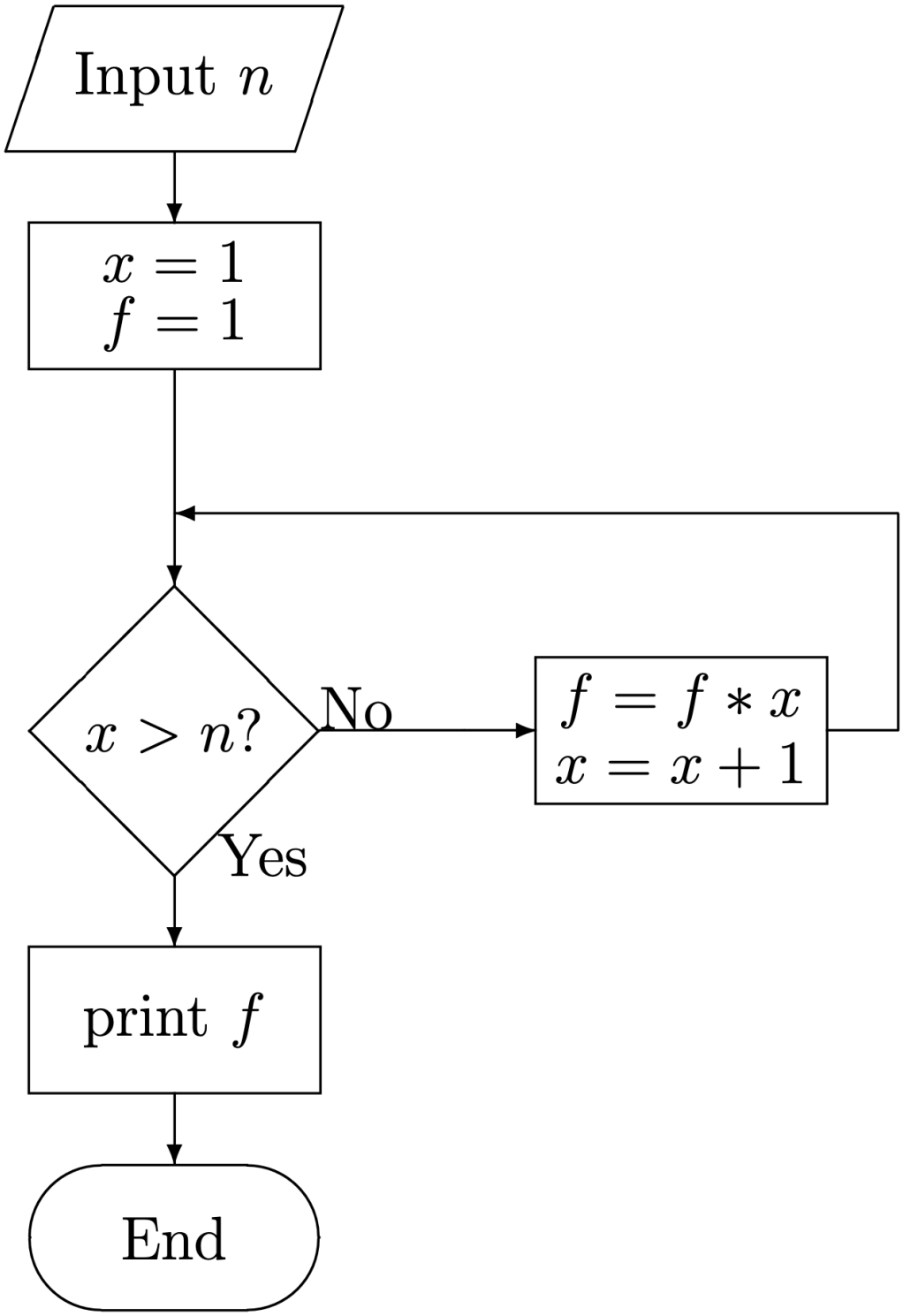


Figure 2.1: Flowchart for computing n!

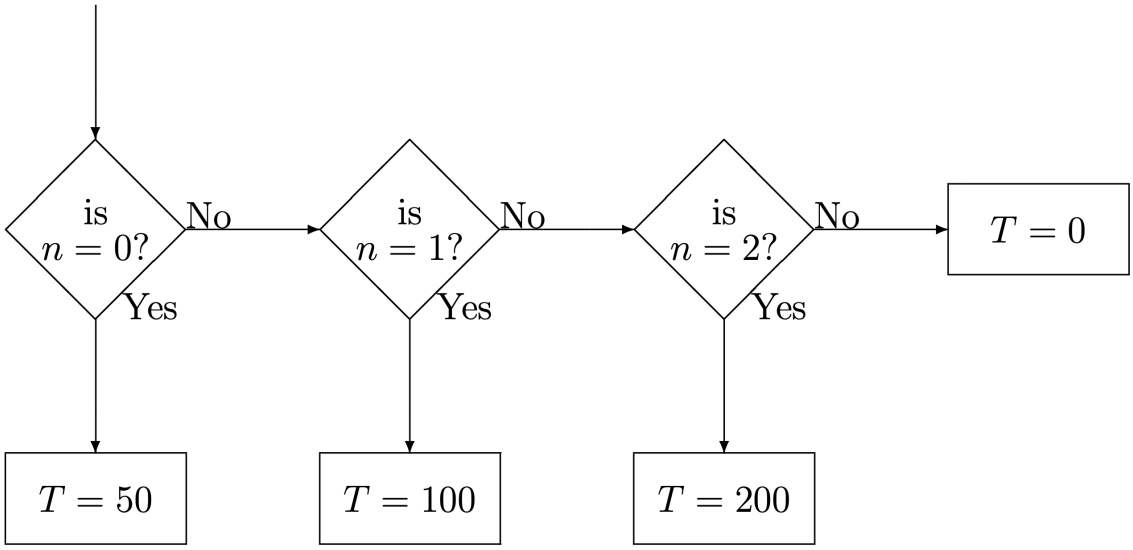


Figure 2.2: Flowchart for if-else chain

3 Introduction to C – part III

3.1 Functions

We have already learned in class that a C-program is made up of a number of functions. Every C-program has at least one function – the `main` function. C-programs can also make use of built-in functions such as math functions and input/output functions, and we have seen examples of how to use functions that are included in `comphys.c` (see exercise 2.7). In this lecture, we are going to learn how to write our own functions to handle certain calculations or perform certain operations. As an example of how to introduce a user-written function into our program, let us modify our Planck function example from Lecture 2.

```
#include <stdio.h>
#include <math.h>

double planck(double wave, double T);

int main()
{
  double wave,T;
  double result;
  int ni;

  printf("\nEnter the temperature in Kelvin > ");
  ni = scanf("%lf",&T);

  wave = 500.0;

  while(wave <= 12000.0) {
    result = planck(wave,T);
    printf("The Planck function at %7.2f A and %7.2f K is %e\n",
            wave,T,result);
    wave += 500.0;
  }
  return(0);
}
```

```
double planck(double wave, double T)
{
  static double p = 1.19106e+27;
  double p1;

  p1 = p/(pow(wave,5.0)*(exp(1.43879e+08/(wave*T)) - 1.0));
  return(p1);
}
```

Notice in this example we have placed all the mathematical calculations in a "function" called `planck`. Note as well that we have added a declaration statement to the beginning of the program:

```
double planck(double wave, double T);
```

This statement performs two functions. First, it declares `planck` to be of type `double` which means that it returns a double-precision floating point number to the function that called it (in this case the `main` function). In addition, this statement informs the compiler that `planck` requires two parameters to be *passed* to it, both double-precision floating-point numbers (`wave` and `T`).

At the end of the C-program we have the actual code for the function `planck`, starting with what is essentially a repeat of the declaration statement. The actual function code is enclosed in brackets. Note that the code for `planck` uses the parameters passed to it by the `main` function, does a calculation, and then returns a number `p1` to the `main` function. In the `main` function, this number is placed in the variable `result` in the following statement:

```
result = planck(wave,T);
```

and thus you can see that `planck` can now be used in the same way as other math functions such as `sin` or `cos`.

Parameters can be passed to functions in two different ways. They can be *passed by value* or *passed by reference*. In the example above, they were passed by value. What this means is that the function receives its own copy of the parameters, which it can change and modify to its heart's content without changing the value of the parameters in the calling function. If we want the function to make permanent changes to these parameters, then we must pass the *addresses* of these parameters to the function. Then whatever changes the function makes to these parameters take effect in other functions as well. This is a way of enabling a function to "return" more than one value.

If we pass `wave` and `T` to `planck` by value, then we would use the statement

```
result = planck(wave,T);
```

On the other hand, if we wish to pass `wave` and `T` to `planck` so that `planck` can permanently modify them, then we must pass their addresses, such as in the following statement:

```
result = planck(&wave,&T);
```

In this case, `wave` and `T` must be referred to as `*wave` and `*T` in `planck`, and the declaration for `planck` would look like:

```
double planck(double *wave, double *T)
```

Another way to pass information to functions is through the medium of *external variables*. There are two types of variables in C, local and external variables. In the example above, `result` is a variable *local* to `main`; it cannot be "seen" or referenced by `planck`, for instance. The variables `p` and `p1` are local to `planck` and cannot be seen by `main`. If, however, we place the declarations for `wave` and `T` at the top of the program, before `main`, then they are *external* variables which can be seen (and modified) by both functions.

The next example shows a use of external variables:

```
#include <stdio.h>
#include <math.h>

double wave,T;      /* External Variables */
double planck();

int main()
{
  double result;
  int ni;

  printf("\nEnter the temperature in Kelvin > ");
  ni = scanf("%lf",&T);

  wave = 500.0;

  while(wave <= 12000.0) {
    result = planck();
    printf("The Planck function at %7.2f A and %7.2f K is %e\n",
            wave,T,result);
```

```
    wave += 500.0;
  }
  return(0);
}

double planck()
{
  static double p = 1.19106e+27;
  double p1;

  p1 = p/(pow(wave,5.0)*(exp(1.43879e+08/(wave*T)) - 1.0));
  return(p1);
}
```

Note the changes in the declaration of `planck` and how `planck` is referenced in the main program.

Exercise 3.1

Write a program that will pass a variable `double T = 100.0` by reference to a function (declare it `void doit(double *T)`) which will then multiply `T` by 2.5 and pass it back to `main`. Print the variable `T` out in `main` to verify that it has indeed been changed to 250.

Exercise 3.2

Rewrite your program in Exercise 3.3 and pass `T` by *value*. Show that even though `T` is multiplied by 2.5 in the `doit` function, its value remains the same in the `main` function.

Exercise 3.3

Rewrite your program from Exercise 3.1, but now declare `T` as an external variable. Verify that after calling `doit` the value of `T` in `main` is 250.0.

Exercise 3.4

In our traditional mathematical world, we permit numbers with an infinite number of ndigits. Real-world arithmetic defines 1/3 as that unique positive number that, when multiplied by 3, yields the integer 1. However, in a computer, we have only a finite number of bits available to represent all numbers. So although the computer can not accurately represent 1/3, it approximates it so that when it is multiplied by 3, the result is so close to the integer 1 that it is acceptable. The computer *error* associated with approximation

arises from *truncation* or *rounding*. For example, the actual decimal value of 1/3 can be written as 0.33333333… where there are an infinite number of 3's in the mantissa; however, the computer approximation can only place 24 3's in the mantissa (for floating point numbers on a 32-bit machine) – the computer truncates the rest of the 3's.
Suppose that E_n represents the magnitude of the *absolute error* of an iterative algorithm after n iterations. Absolute error is just defined as the absolute value of the difference between an actual value, p, and an approximated value, p^*: $E = |p - p^*|$. After n iterations, if $E_n = CnE_0$, where E_0 is the initial error, and C is a constant independent of n, then the error is said to be *linear*. If $E_n = C^nE_0$, then the error is said to be *exponential*. Linear error growth is usually unavoidable and when C and E_0 are small, the results are generally acceptable. Exponential growth should be avoided. A system exhibiting linear error growth is known to be *stable*; a system exhibiting exponential growth is known to be *unstable*.
The Taylor polynomial for $f(x) = \sin x$ about $x_0 = 1.0$ is

$$p(x) = \sum_{i=1}^{N} (-1)^{i+1} \left(\frac{x^{2(i-1)+1}}{(2(i-1)+1)!} \right).$$

Our goal is to find out how many of the terms in the Taylor series we need to keep in order to stay within a user-defined error. Below (and provided) is a program that finds the minimum number of iterations, N, that satisfies the absolute error equation:

$$\left| \sin\left(\frac{\pi}{4.0}\right) - p\left(\frac{\pi}{4.0}\right) \right| < 10^{-5}.$$

Run this program using the command:

```
./program_name tolerance > ex34.out
```

where `program_name` is the name that you assign the executable program at compile time,

```
gcc -o program_name ex34.c -lm
```

and `tolerance` is either a decimal or scientific notation value. Plot the results using gnuplot:

```
gnuplot> plot "ex34.out" using 1:2 with lines
```

Modify the program to do the same thing for $\exp x$, where the Taylor series expansion is:

$$p(x) = \sum_{i=1}^{N} \frac{x^{i-1}}{(i-1)!}.$$

Also, make the passed variables *external* instead of being passed by value. How do the rates of error-reduction compare between the two functions? You may find that this program may be of value to you in the future - add to to your code library.

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

double getp(int i, double x);
double geterror(double p,double pstar);
double factorial(int i);

const double pi=3.141592653589793;

int main(int argc, char *argv[])
{
  int i,N,ni;
  int M=50;  //set the maximum number of allowable steps
  double tolerance=0.0;
  double error=0.0;
  double p=0.0;
  double pstar;
  double x;

  x=pi/4.0;

  while (tolerance <= 0.0) {
    if (argc != 2) {
      printf("\nEnter the tolerance (scientific notation, e.g. 1.5e-13) > ");
      ni = scanf("%le",&tolerance);
    } else tolerance=atof(argv[1]);

    if (tolerance == 0.0) {
      printf("\nTolerance can not be zero or negative, try again...\n\n");
      return(0);
    }
  }

  pstar=sin(x);

  for (i=1;i<=M;i++) {
    p=p+getp(i,x);
    error=geterror(p,pstar);
    printf("%d  %le\n",i,error);
    if (error<tolerance) {
      break;
    }
    if (i==M) {
      printf("\nThe number of iterations exceeds allowable maximum, retry\n");
      printf("\nwith new tolerance or change max limit.\n");
      break;
    }
  }
  return(0);
}

double getp(int i, double x)
```

Read: *The C Programming Language*: Sections 4.1 – 4.7

3.2 File Input and Output

So far we have printed the output from our programs to the screen. This is useful only if there is a relatively small amount of output, but if we have pages and pages, it is more desirable to output to a file. This file can then be used by other programs, or we can import it into a graphics program such as gnuplot and plot our results. How do we do file input and output?

We must first define a "file pointer" and then use the `fopen` function. Consider this fragment of code:

```
FILE *in,*out;

in = fopen("input.dat","r");
out = fopen("output.dat","w");
```

here we have defined two file pointers `*in` and `*out` and then used the `fopen` function to open the file `input.dat` for reading (`r`), and the file `output.dat` for writing (`w`). The file pointers "point" to "streams" `in` and `out` which can be used to input data from and output data to, respectively, the files `input.dat` and `output.dat`. We can use the functions `fscanf` and `fprintf` to read and write data. Consider the following code fragment:

```
#include <stdlib.h>  /* Needed for exit() */

double x,y;
int i,ni;
FILE *in,*out;

if((in = fopen("input.dat","r")) == NULL) {
  printf("\nCannot open file for input\n");
  exit(1);
}

if((out = fopen("output.dat","w")) == NULL) {
  printf("\nCannot open file for output\n");
  exit(1);
}

for(i=0;i<10;i++) {
  ni = fscanf(in,"%lf",&x);
```

```
  y = x*1.34e+22;
  fprintf(out,"%e",y);
}
fclose(in);
fclose(out);
```

Notice that we have embedded the two `fopen` statements in `if` statements that check to see if the function `fopen` returns in either case a `NULL`. If `fopen` does return a `NULL`, then this means that something has prevented the program from opening the file. In the case of a file opened for reading, this might happen if the file does not exist, or if the file "permissions" do not allow reading. If `fopen` returns a `NULL`, then the program exits gracefully using the `exit(1)` statement. It is important to include such checks in your program. If you don't, and `fopen` returns a `NULL` then nonsense will be input into your program, and you will get garbage out! Notice that at the end we closed both streams using the `fclose` function. Every `fopen` call should be accompanied by an `fclose` call. Let us modify our Planck function program to output the data to a file:

```
#include <stdio.h>
#include <stdlib.h>  /* Needed for exit() */
#include <math.h>

double planck(double wave, double T);

int main()
{
  double wave,T;
  double result;
  int ni;
  char outfile[80];
  FILE *out;

  printf("\nEnter the temperature in Kelvin > ");
  ni = scanf("%lf",&T);

  printf("\nEnter the name of the output file > ");
  ni = scanf("%s",outfile);

  if((out = fopen(outfile,"w")) == NULL) {
   printf("\nCannot open %s for writing\n",outfile);
   exit(1);
  }
```

```
  wave = 500.0;

  while(wave <= 12000.0) {
    result = planck(wave,T);
    fprintf(out,"%7.1f   %e\n",wave,result);
    wave += 500.0;
  }

  fclose(out);

  return(0);
}

double planck(double wave, double T)
{
  static double p = 1.19106e+27;
  double p1;

  p1 = p/(pow(wave,5.0)*(exp(1.43879e+08/(wave*T)) - 1.0));
  return(p1);
}
```

i Exercise 3.5

Use your editor to create a file called `inputwave.dat` in your working directory. The contents of this file should be:

```
500
1000
1500
2000
.
.
.
12000
```

i.e. with all the wavelengths between 500 and 12000 Å (please fill in the dots appropriately!!). Modify the above program so that it reads in one line of this file for every passage through the loop. You can do this by changing the condition in the `while` statement to something like this:

```
while(fscanf(in,"%lf",&wave) != EOF) {
```

This will read in one line per passage through the loop and exit the loop when the End Of the File (`EOF`) is encountered.

Read: *The C Programming Language*: Sections 7.5 – 7.7

3.3 An Aside: Incorporating GNUPLOT in your program

So far we have employed gnuplot as a stand alone program, but it is possible to use gnuplot commands in your program and thus incorporate graphics into your program. As an example, let us modify the last example program to do this. Modify the line `FILE *out;` to `FILE *out,*rsp;` and then, after the `fclose(out)` statement, add the following code:

```
if((rsp = fopen("gnuplot.rsp","w")) == NULL) {
  printf("\nCannot open gnuplot.rsp for writing\n");
  exit(1);
}
fprintf(rsp,"plot '%s' using 1:2 with lines\n",outfile);
fprintf(rsp,"pause mouse\n");
fprintf(rsp,"replot\n");
fclose(rsp);

if(system("gnuplot gnuplot.rsp") == -1) {
  printf("\nCommand could not be executed\n");
  exit(1);
}
return(0);
```

3.4 More on Vectors and Arrays

When we introduced the concept of variable (part I), we mentioned that arrays of variables could be declared easily, using statements like:

```
double r[10];
int x[200],y[3][5];
```

The entries of `r` then run as `r[0]`, `r[1]`, ..., `r[9]`, the entries of `x` as `x[0]`, `x[1]`, ..., `x[199]` and the entries of `y` as

```
y[0][0],y[0][1],y[0][2],y[0][3],y[0][4],
y[1][0],y[1][1],y[1][2],y[1][3],y[1][4],
y[2][0],y[2][1],y[2][2],y[2][3],y[2][4]
```

This way of *allocating* memory for vectors and arrays is useful if 1) the vector or array is not very large or, 2) one knows the size of the vector or array before the program is run. If the program needs to be run with different sized vectors or arrays each time it is run (for instance, you might need to analyze different data sets with different numbers of data points), it is useful to *dynamically* allocate memory for your vector or array at runtime. This can be done using the built-in C-function `calloc`, but as we mentioned in Lecture I, two very useful functions `vector` and `matrix` have been provided for you in `comphys.c`. The `vector` and `matrix` functions can be used in the following way: Let us suppose that we want to dynamically allocate space for vectors `r`, `x`, and array `y` so that they have the same dimensions as in the example above. We can do it as follows

```
double *r;
int *x,**y;
   .
   .
   .
r = dvector(0,9);
x = ivector(0,199);
y = imatrix(0,2,0,4);
```

First, notice that there are a number of versions of `vector` and `matrix`. To allocate space for an integer vector, use `ivector`, a floating point vector, `vector` and for a double precision floating point vector, `dvector`. The same goes for `matrix`. Notice as well that the functions `vector` and `matrix` give us the ability to define vectors and matrices which are not *zero-offset*, i.e. whose first index is not 0. For instance, if we want `x` to go from `x[1]` to `x[200]`, we could use the statement

```
x = ivector(1,200);
```

Unit offset vectors can be quite convenient. To use the functions `vector` and `matrix` in our programs, we must include the following statements right at the beginning of our programs:

```
#include "comphys.h"
#include "comphys.c"
```

Indeed, as we go through the semester, we will use a number of functions from `comphys.c`. Most of these functions are from the book *Numerical Recipes in C*, Second Edition by Press et

al., published by Cambridge University Press. This is one of the best books ever written, and should be in the library of all physicists and astronomers.

We have already seen in Lecture I how vectors can be initialized. We can similarly initialize arrays. For instance, the array `y` can be initialized with values in the following way:

```
int y[3][5] = {{ 1, 8, 3, 5, 2},
               { 0, 1, 4, 4, 2},
               { 99,456, 23, 12, 5}};
```

Unfortunately, it is not so easy to initialize a vector or matrix which has been defined using the functions `vector` or `matrix`. One way of doing so is to stuff the vector or matrix with data read in from a file (see Exercise 3.6).

One does not have to use `vector` to create a unit offset vector. For instance, let us suppose we have declared and initialized a vector `c` in the following way:

```
double c[4] = {1.865,3.449,1.23e+04,0.0045};
```

which clearly goes from `c[0]` to `c[3]`. Let us suppose we want to define a vector that has the same entries, but is unit offset. This is easy to arrange, and can be done as follows

```
double c[4] = {1.865,3.449,1.23e+04,0.0045};
double *C;

C = c-1;
```

The vector `C` will now go from `C[1]` to `C[4]` with the same entries as `c`, i.e., `C[1] = 1.865`, `C[2] = 3.449`, etc.

One can in principle do the same thing for matrices, but the implementation of the offset is quite a bit more complex.

> **Exercise 3.6**
>
> Modify your program from Exercise 3.5 to read the values from the file `inputwave.dat` into the vector `wave` which has been allocated space using the function `dvector`. Then modify the *while loop* to use these values in the computation of the Planck function.

Read: *The C Programming Language*: Chapter 5

4 Interpolation

4.1 Linear Interpolation

A common computational problem in physics involves determining the value of a particular function at one or more points of interest from a tabulation of that function. For instance, we may wish to calculate the index of refraction of a type of glass at a particular wavelength, but be faced with the problem that that particular wavelength is not explicitly in the tabulation. In such cases, we need to be able to interpolate in the table to find the value of the function at the point of interest. Let us take a particular example.

BK-7 is a type of common optical crown glass. Its index of refraction n varies as a function of wavelength; for shorter wavelengths n is larger than for longer wavelengths, and thus violet light is refracted more strongly than red light, leading to the phenomenon of dispersion. The index of refraction is tabulated in Table 4.1.

Let us suppose that we wish to find the index of refraction at a wavelength of 5000. Unfortunately, that wavelength is not found in the table, and so we must estimate it from the values in the table. We must make some assumption about how n varies between the tabular values. Presumably it varies in a smooth sort of way and does not take wild excursions between the tabulated values. The simplest and quite often an entirely adequate assumption to make is that the actual function varies linearly between the tabulated values. This is the basis of **linear interpolation**.

> **Exercise 4.1**
>
> Determine, by hand, the value of the index of refraction of BK7 at 5000 using linear interpolation.

How do we carry out linear interpolation on the computer? Let us suppose that the function is tabulated at N points and takes on the values $y_1, y_2, y_3 ... y_N$ at the points $x_1, x_2, x_3 ... x_N$, and that we want to find the value of the function y at a point x that lies someplace in the interval between x_1 and x_N.

The first thing that we must do is to bracket x, that is we must find a j such that $x_j < x \leq x_{j+1}$. This can be accomplished by the following code fragment:

Table 4.1: Refractive Index for BK7 Glass

$\lambda()$	n	$\lambda()$	n	$\lambda()$	n
3511	1.53894	4965	1.52165	8210	1.51037
3638	1.53648	5017	1.5213	8300	1.51021
4047	1.53024	5145	1.52049	8521	1.50981
4358	1.52669	5320	1.51947	9040	1.50894
4416	1.52611	5461	1.51872	10140	1.50731
4579	1.52462	5876	1.5168	10600	1.50669
4658	1.52395	5893	1.51673	13000	1.50371
4727	1.52339	6328	1.51509	15000	1.5013
4765	1.5231	6438	1.51472	15500	1.50068
4800	1.52283	6563	1.51432	19701	1.495
4861	1.52238	6943	1.51322	23254	1.48929
4880	1.52224	7860	1.51106		

```
for(i=1;i<N;i++) {
  if(xn[i] < x && xn[i+1] >= x) {
    j = i;
    break;
  }
}
```

where the `xn`'s are the tabulated points. When the `if` statement is satisfied, `j` is assigned the value of `i` and the procedure drops out of the loop. Please note that this is not the most efficient way to accomplish this task, especially if N is very large. We will look at a more efficient way later on.

Once we have bracketed x, we can find the equation of the line between the points (x_j, y_j) and (x_{j+1}, y_{j+1}). This equation will be of the form $y = mx + b4where\text{m\$}$ is the slope and b is the y-intercept. As we all know, the slope is given by

$$m = \frac{y_{j+1} - y_j}{x_{j+1} - x_j}$$

and the intercept can be found by substituting one point, say, (x_j, y_j) into the resulting equation. Thus,

$$b = y - mx = y_j - \frac{y_{j+1} - y_j}{x_{j+1} - x_j} x_j$$

yielding for the equation of the line, after some rearrangement,

$$y == y_j + \left(\frac{y_{j+1} - y_j}{x_{j+1} - x_j}\right)(x - x_j)$$

It is left to the student to show (for future reference) that this equation may be rewritten

$$y = Ay_j + By_{j+1}$$

where

$$A = \frac{x_{j+1} - x}{x_{j+1} - x_j}$$

and

$$B = \frac{x - x_j}{x_{j+1} - x_j}$$

i Exercise 4.2

Write a C-function that will linearly interpolate the tabular data for the index of refraction of BK-7 and return a value for n for wavelengths between 3511 and 23254. Write a driver program that will use this function to prompt the user for a wavelength and then print to screen the corresponding value of n.

i Exercise 4.3

The file `data/boiling.dat` contains data in two columns for the boiling point of water at different atmospheric pressures. The first column is the pressure in millibars, the second is the corresponding boiling point temperature in degrees Celsius. Write a C-function that initializes two vectors, `P` and `T` with the data in that data file (don't read in the datafile – hardwire the data into your program), accepts the pressure as a double floating-point parameter, and returns the value of the temperature of the boiling point at that pressure. You should also write a driver program that will prompt the user for an atmospheric pressure, check whether it is within the limits of the data ($50 \leq P \leq 2150$), calls your C-function, and prints to the screen the boiling point of water at that pressure.

4.2 Polynomial Interpolation

Linear interpolation is good enough for government work, and it is even better than that. Because it is simple and makes the simplest possible assumption about the data, it should be employed in all cases except where it is manifestly inadequate. There are such cases. Sometimes the function being interpolated is very non-linear or has been tabulated at such wide intervals

that linear interpolation would lead to large errors. Some applications demand more than simply the functional values at the interpolated points; sometimes the derivative of the function is required as well. With linear interpolation, the derivative is a constant between the tabulated points, and may actually be undefined at the tabulated points!

For such applications it may be best to interpolate using a polynomial interpolating function or functions. It can be shown that the following poly- nomial $P(x)$ of degree $N-1$ will exactly pass through the N tabulated points of the function $y = f(x)$:

$$\begin{aligned} P(x) = & \frac{(x-x_2)(x-x_3)\cdots(x-x_N)}{(x_1-x_2)(x_1-x_3)\cdots(x_1-x_N)}\, y_1 \\ & + \frac{(x-x_1)(x-x_3)\cdots(x-x_N)}{(x_2-x_1)(x_2-x_3)\cdots(x_2-x_N)}\, y_2 \\ & + \cdots \\ & + \frac{(x-x_1)(x-x_2)\cdots(x-x_{N-1})}{(x_N-x_1)(x_N-x_2)\cdots(x_N-x_{N-1})}\, y_N \end{aligned}$$

The problem with the direct application of the polynomial $P(x)$ is that for tabulations with many points, it can lead to very high degree polynomials. For instance, if a function is tabulated at 100 points, the above equation would yield a polynomial of degree 99! Such a polynomial could potentially fluctuate wildly between the tabulated points and thus not be a good representation of the actual function. $P(x)$ is more usually applied to subsets of the tabulated points. For instance, if the polynomial is applied to subsets of 3 points, it yields parabolic interpolation, which can be much superior to linear interpolation if the function has a number of minima and maxima.

One problem with parabolic 3-point interpolation is that when it comes to bracketing x, there is an ambiguity – does one bracket between x_1 and x_2 or between x_2 and x_3? This is one reason why cubic 4-point interpolation is more commonly practiced – the bracketing is then between x_2 and x_3 with no ambiguity. Such interpolation is also called Lagrangian 4-point interpolation. The Lagrangian 4-point interpolation equation can be written:

$$L(x) = \frac{(x-x_2)(x-x_3)(x-x_4)}{(x_1-x_2)(x_1-x_3)(x_1-x_4)}\,y_1$$
$$+\frac{(x-x_1)(x-x_3)(x-x_4)}{(x_2-x_1)(x_2-x_3)(x_2-x_4)}\,y_2$$
$$+\frac{(x-x_1)(x-x_2)(x-x_4)}{(x_3-x_1)(x_3-x_2)(x_3-x_4)}\,y_3$$
$$+\frac{(x-x_1)(x-x_2)(x-x_3)}{(x_4-x_1)(x_4-x_2)(x_4-x_3)}\,y_4$$

which the student can easily verify by reference to the equation for $P(x)$.

To apply this interpolation equation, the user should first bracket x between x_j and x_{j+1} as before, but now identify x_j with x_2 in the above equation and x_{j+1} with x_3. It then follows that x_1 will be x_{j-1} and x_4 will be x_{j+2}. The perceptive student will see that this will lead to a problem at the endpoints. For instance, if x is situated between the first two tabulated points, x_{j-1} will be undefined. Likewise, if x is situated between the last two tabulated points, x_{j+2} will be undefined. Thus in those intervals, the user must either interpolate linearly, or, in the first case, use the polynomial that would be used for an x bracketed between the 2nd and 3rd tabulated points, and similarly for the last case.

Exercise 4.4

Write a C-function that will implement the 4-point Lagrangian interpolation formula above. For the endpoints, use the polynomial that would have been defined for the adjacent interval as described above. Modify the driver program in Exercise 4.2 (interpolation in a table of the wavelength and the index of refraction for BK-7 glass) to use this new C-function. Compare the results between the two programs.

Exercise 4.5

Write a C-function that will implement the 4-point Lagrangian interpolation formula above. For the endpoints, use the polynomial that would have been defined for the adjacent interval as described above. Modify the driver program in Exercise 4.3 (interpolation in a table of atmospheric pressure and the boiling point of water) to use this new C-function. Compare the results between the two programs.

We have only scratched the surface of the subject of interpolation. The subject of Extrapolation – finding a value for a function outside the range of the defined points – is much more dangerous. Interpolation schemes can be used for extrapolation, but only with great care!

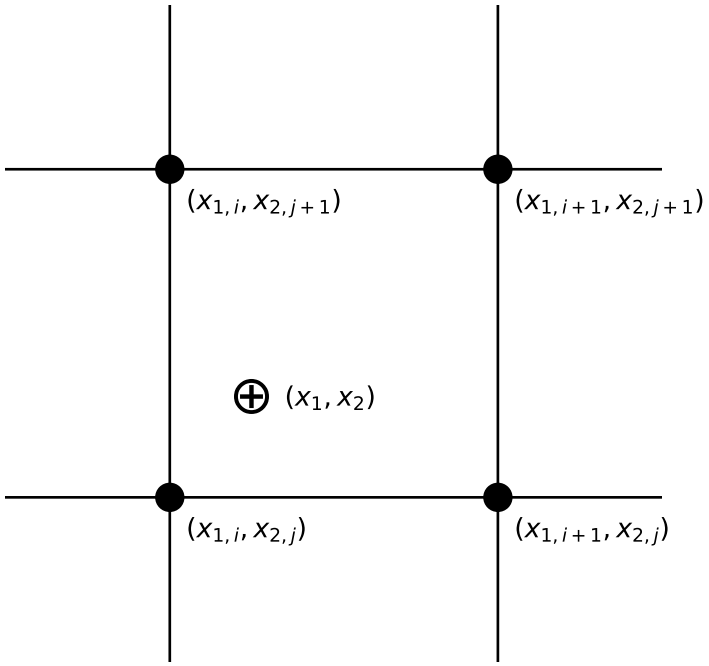
Before we leave the subject of interpolation, let us examine one further subject, that of efficiently bracketing x. If we have a set of x's (say in a vector `x[i]`) in numerical order that we must find interpolated values for, there is a simple time saving step that we can use, implemented in the following fragment of code (assume `x` is a vector of dimension `n`):

```
j = 1;
for(k=1;k<=n;k++) {
  for(i=j;i<N;i++) {
    if(xn[i] < x[k] && xn[i+1] >= x[k]) {
      j = i;
      break;
    }
  }
  here is your interpolation function code
}
```

Notice that the second `for` loop begins at `i=j` and not `i=1`; since the x_k's are in numerical order, if x_k is bracketed between `xn[j]` and `xn[j+1]`, there is no need to search beginning at `i=1` for x_{k+1} because it will either be bracketed between the same pair or later pairs (note that we are obviously also assuming that the `xn[j]`'s are in numerical order). This can save an enormous amount of time in a code that needs to interpolate in a large (say $N > 100$) table. Another useful trick is that of bisection. Further notes on bisection and other techniques for efficiently bracketing x can be found in *Numerical Recipes* (Press et al. 1992).

4.3 Interpolation in 2 dimensions

Some problems require interpolation in a two-dimensional grid of data. Let us suppose, for instance, that $y = y(x_1, x_2)$ where x_1 and x_2 are the two independent variables. The functional value y is tabulated on a Cartesian grid, and so the first thing the programmer must do is to bracket the desired point (x_1, x_2) in this grid (see figure below):



This bracketing can be done by bracketing in the two dimensions one at a time, using the technique discussed earlier.

The simplest interpolation technique in two dimensions is *bilinear interpolation*. If we define

$$\begin{aligned} y_1 &= y(x_{1,i}, x_{2,j}) \\ y_2 &= y(x_{1,i+1}, x_{2,j}) \\ y_3 &= y(x_{1,i+1}, x_{2,j+1}) \\ y_4 &= y(x_{1,i}, x_{2,j+1}) \end{aligned}$$

i.e., working our way counterclockwise around the above figure, then the interpolation formulae are:

$$\begin{aligned} t &= (x_1 - x_{1,i})/(x_{1,i+1} - x_{1,i}) \\ u &= (x_2 - x_{2,j})/(x_{2,j+1} - x_{2,j}) \end{aligned}$$

which make both t and u lie between 0 and 1. Then,

$$y(x_1, x_2) = (1-t)(1-u)y_1 + t(1-u)y_2 + tuy_3 + (1-t)uy_4$$

where (x_1, x_2) are the coordinates of the desired point.

Exercise 4.6

Exercise 4.6: A good example of the need to carry out interpolation in two dimensions is found in the calculation of partition functions. In certain plasma-physics contexts, it is necessary to calculate the partition functions of atoms and ions.
A partition function, U, is essentially an overall "statistical weight" for the atom, calculated by carrying out a weighted sum — weighted according to the populations of the levels — of the statistical weights for all of the energy levels in the atom.
At low temperatures, the partition function is simply the statistical weight of the ground level of the atom, but at higher temperatures, the partition function becomes larger, as the populations in the excited levels become significant.
The partition function is also a function of density, as at high densities in the plasma, the outermost energy levels are effectively "stripped off," resulting in a lowering of the ionization energy, usually denoted as ΔE. Thus, $U = U(T, \Delta E)$.
Because the calculation for a partition function can be very complex, they are usually calculated and tabulated so that users need not carry out the full calculation. The following table is a tabulation for the partition function for the hydrogen atom:

Table 4.2: The Hydrogen Partition Function

T (K)	ΔE = 0.10	ΔE = 0.50	ΔE = 1.00	ΔE = 2.00
3250	2.000	2.000	2.000	2.000
10083	2.000	2.000	2.000	2.000
14188	2.025	2.006	2.005	2.004
15643	2.068	2.016	2.012	2.009
17246	2.168	2.037	2.027	2.020
19014	2.384	2.080	2.058	2.040
20963	2.814	2.162	2.114	2.078
23111	3.610	2.308	2.213	2.142
25480	4.991	2.551	2.377	2.246

Write a function that will perform a 2-D interpolation in this table, and use it to determine the partition function for Hydrogen at the following points $(T, \Delta E)$: (16000, 0.25), (18500, 1.50), (19000, 0.15), (25023, 1.99).

Exercise 4.7

The study of plasmas is an important field of physics, and is essential in the understanding of the interiors of stars and the functioning of fusion reactors.
Hydrogen becomes increasingly ionized (hydrogen loses its electron when ionized) with increasing temperature, but the density of electrons, N_e, also plays an important role.

The following table gives the ratio of ionized hydrogen atoms to all forms of hydrogen (neutral + ionized) as a function of both T (in kelvins) and N_e (number of electrons per cubic centimeter).
Write a C-function and driver that will interpolate in this table. The driver should prompt the user for T (in kelvins) and N_e. Note that the table is tabulated in terms of $\log N_e$.

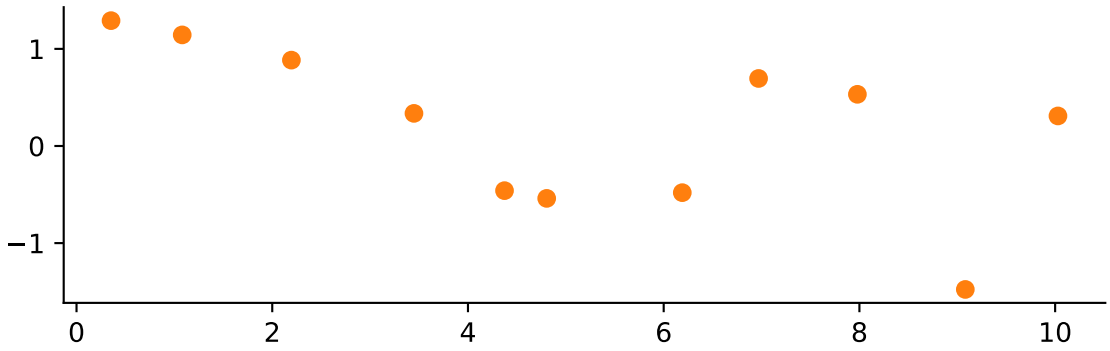
Table 4.3: Hydrogen Ionization: Ratio of Ionized to Total

T(K)	10.0	11.0	12.0	13.0	14.0	15.0	16.0	17.0
1000.0	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.000	0.0000
2000.0	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.000	0.0000
3000.0	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.000	0.0000
4000.0	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.000	0.0000
5000.0	0.0017	0.0002	0.0000	0.0000	0.0000	0.0000	0.000	0.0000
6000.0	0.2980	0.0407	0.0042	0.0004	0.0000	0.0000	0.000	0.0000
7000.0	0.9582	0.6961	0.1864	0.0224	0.0023	0.0002	0.000	0.0000
8000.0	0.9979	0.9791	0.8241	0.3191	0.0448	0.0047	0.000	0.0000
9000.0	0.9998	0.9980	0.9804	0.8335	0.3335	0.0477	0.005	0.0005
10000.0	1.0000	0.9997	0.9971	0.9713	0.7719	0.2529	0.032	0.0034
11000.0	1.0000	0.9999	0.9994	0.9939	0.9425	0.6211	0.140	0.0161
12000.0	1.0000	1.0000	0.9998	0.9984	0.9841	0.8606	0.381	0.0581
13000.0	1.0000	1.0000	0.9999	0.9995	0.9948	0.9503	0.656	0.1607
14000.0	1.0000	1.0000	1.0000	0.9998	0.9980	0.9807	0.835	0.3373
15000.0	1.0000	1.0000	1.0000	0.9999	0.9992	0.9917	0.922	0.5448

4.4 Interpolation in Python

Let's say you have run an experiment and have some data.

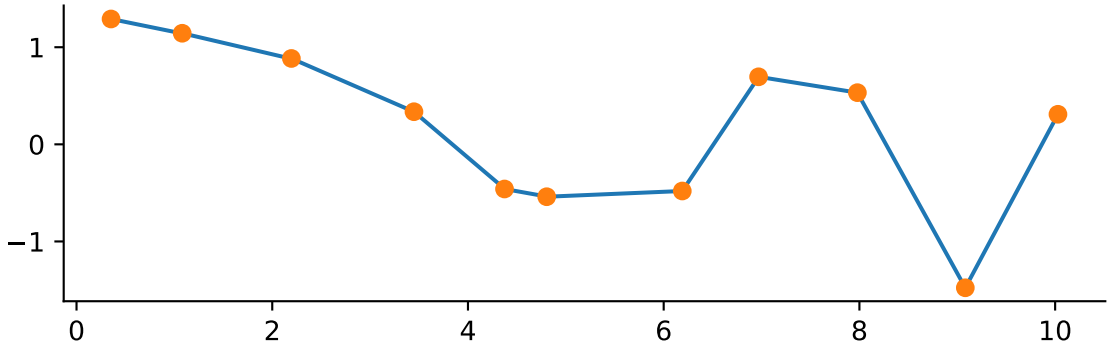
```
# Pretend you don't know the functional form of the data.
num_samples = 11
x_data = np.linspace(0, 10, num=num_samples) + np.random.normal(scale=.2, size=num_samples)
y_data = np.cos(-x_data**2 / 9.0)            + np.random.normal(scale=.2, size=num_samples)

fig, ax = plt.subplots()
ax.plot(x_data, y_data, 'o', color='C1')
```



Now you want to connect the dots … I mean interpolate your data.

Numpy has a method (`numpy.interp`) for simple linear interpolation. The method takes in an array of new `x`'s where you want to calculate the new `y`'s. The original `x` and `y` points are also needed.

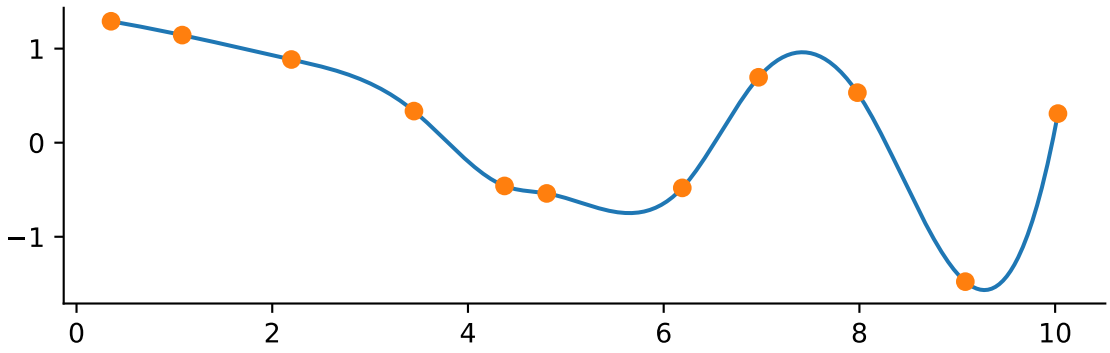
```
x_interp = np.linspace(x_data.min(), x_data.max(), num=1001)
y_interp = np.interp(x_interp, x_data, y_data)

fix, ax = plt.subplots()
ax.plot(x_interp, y_interp)
ax.plot(x_data, y_data, 'o')
```



If you want a fancier smooth line SciPy has got you covered! The `interpolate` subpackage contains many methods to suit your needs. These methods follow a different pattern than the Numpy approach. When using the SciPy `interpolate` methods you provide the original data points and are returned a function you can call to interpolate new values.

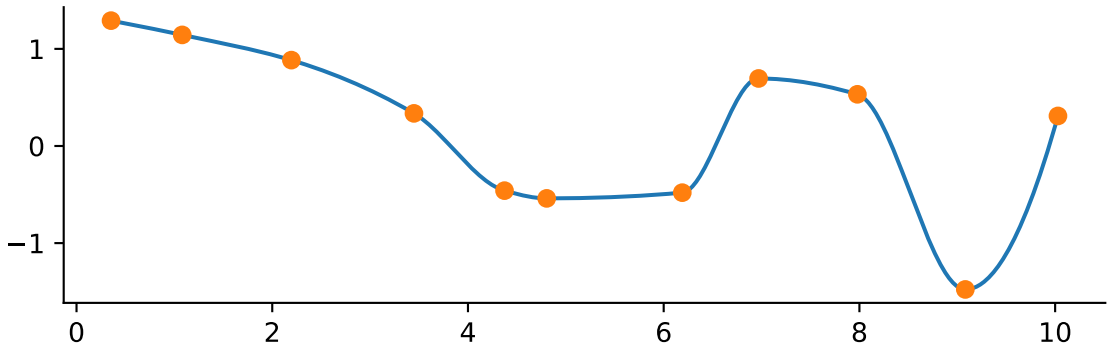
As an example consider the `CubicSpline` method

```
from scipy.interpolate import CubicSpline

f = CubicSpline(x_data, y_data)

y_interp = f(x_interp)

fix, ax = plt.subplots()
ax.plot(x_interp, y_interp)
ax.plot(x_data, y_data, 'o')
```



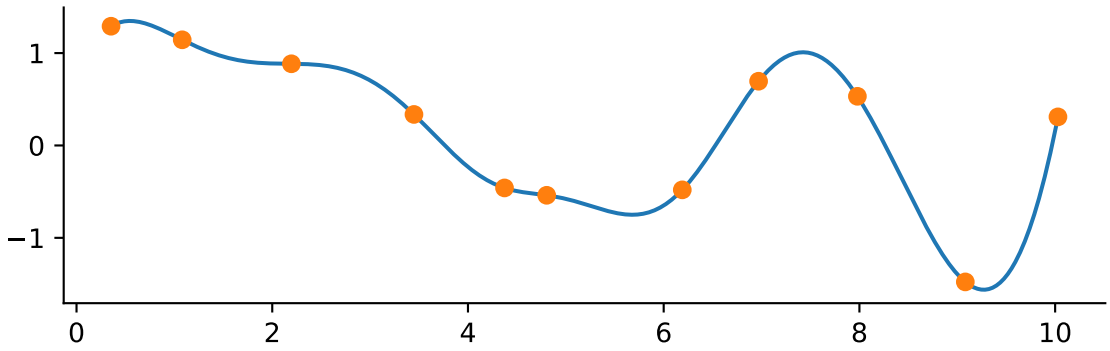
Quick and easy! There are other interpolation methods. For instance, maybe you are concerned with the line overshooting the data. An alternative is to use a so-called *monotone* cubic interpolant which attempts to preserve the local shape implied by the data. An example is the `PchipInterpolator`.

```
from scipy.interpolate import PchipInterpolator

f = PchipInterpolator(x_data, y_data)

y_interp = f(x_interp)

fix, ax = plt.subplots()
ax.plot(x_interp, y_interp)
ax.plot(x_data, y_data, 'o')
```



These cubic methods are typically fine for most application. There is also a generalized method for making interpolating splines of any degree but use with caution. You may introduce unexpected wiggles!

```
from scipy.interpolate import make_interp_spline

f = make_interp_spline(x_data, y_data, k=5)

y_interp = f(x_interp)

fix, ax = plt.subplots()
ax.plot(x_interp, y_interp)
ax.plot(x_data, y_data, 'o')
```



There are other methods for 2-D interpolation and smoothing as well. Have a look at the SciPy Interpolation User Guide for more details.

5 Finding the Root of a Function

The need to determine the roots of a function – i.e. where that function crosses the x-axis – is a recurring problem in practical and computational physics. For example, the density of matter in the interior of a star may be approximately represented by a type of function called a polytropic function. The edge or surface of the star occurs where the appropriate polytropic function goes to zero. There are many other examples. All of you know how to find analytically the roots of a parabola. For more complex functions, it may not be possible or easy to find the roots analytically, and so it is necessary to use numerical methods. The numerical method that is used depends upon whether or not it is possible to determine the derivative of the function. The function, for instance, may be given to us as a "black box" – it may be in tabular form, or given to us as a "C" function, or it may be known only through a recursion equation. In such a case, the appropriate method would be to more and more closely bracket the root until we know it to the precision that we desire. Alternately, we may be able to differentiate the function. In that case, we can use the Newton-Raphson method (or similar methods) to zero in on the root.

5.1 The Method of Bracketing and Bisection

The first thing that we need to know in root-finding is an approximate location of the root we are interested in. How can we determine this? The easiest way is to use a plotting program, such as `gnuplot`, `EXCEL`, etc., to plot the function. Another way is to calculate the function at a number of points. Then, unless the function is pathological in some way, we can say that a root exists in the interval (a, b) if $f(a)$ and $f(b)$ have opposite signs. An exception to this is a function with a singularity in this interval. Suppose c is in the interval (a, b), and the function is given by

$$f(x) = \frac{1}{x - c}$$

Then, $f(x)$ has a singularity at c and not a root. There are other pathological functions such as $\sin(1/x)$, which has infinitely many roots in the vicinity of $x = 0$. So, the user must be aware of such problems, as such pathologies will give problems to even the most clever programs.

Let us assume that our function is fairly free of pathologies and that we know that there is a root in the interval (a, b) and that we want to find it more exactly.

The simplest way to proceed is using the method of *bisection*. This method is straightforward. If we know that a root is in (a, b) because $f(a)$ has a different sign from $f(b)$, then evaluate the

function f at the midpoint of the interval, $(a+b)/2$, and examine its sign. Replace whichever limit (e.g. a or b) that has the same sign. Repeat the process until the interval is so small that we know the location of the root to the desired accuracy.

The "desired" accuracy must be within limits. As we have discussed before in class and lab, since computers use a fixed number of binary digits to represent floating-point numbers, there is a limit to the accuracy with which computers can represent numbers. To be precise, the smallest floating-point number which, when added to the floating-point number 1.0, produces a floating-point number different from 1.0, is termed the *machine accuracy*, ϵ_m. Using single precision, most machines have $\epsilon_m \approx 1 \times 10^{-8}$. Using double precision, $\epsilon_m \approx 1 \times 10^{-16}$. It is always a good idea to back off from these numbers, and thus setting ϵ_m to 10^{-6} and 10^{-14} respectively are practical limits. Sometimes even those limits are too optimistic. Let us suppose that our $(n-1)^{\text{th}}$ and n^{th} evaluations of the midpoint are x_{n-1} and x_n, and that the root was initially in the interval (a, b). We would then be justified in stopping our search if $|x_n - x_{n-1}| < \epsilon_m |b - a|$.

> **Exercise 5.1**
>
> The polynomial $f(x) = 5x^2 + 9x - 80$ has a root in the interval $(3, 4)$. Find the root to a precision of 10^{-6} using the *bisection* method. Verify your root analytically.

Plotting Functions in `gnuplot`

When finding roots, it is a good idea to plot the function to get an idea of the position of the roots. This can be done in `gnuplot`. To plot the function of Exercise 5.1, enter the following commands:

```
gnuplot> f(x) = 5*x**2 + 9*x - 80
gnuplot> plot f(x)
gnuplot> g(x) = 0
gnuplot> replot g(x)
```

Note that exponentiation in `gnuplot` is accomplished with "`**`". Notice in the plot that $f(x)$ has a root at about -5 and another between 3 and 4.

> **Exercise 5.2**
>
> Bessel functions $J_0(x)$, $J_1(x)$, etc. often turn up in mathematical physics, especially in the context of optics and diffraction. The file `comphys.c` has a canned version of the Bessel function $J_0(x)$. The function declaration (contained in the file `comphys.h`) for this function is

```
float bessj0(float x)
```

Modify your program from Exercise 5.1 to find the first two positive roots of $J_0(x)$.

Exercise 5.3

The viscosity of air (in micropascal seconds) is given to good accuracy by the following polynomial (valid between $T = 100\,\mathrm{K}$ and $600\,\mathrm{K}$):

$$\eta = 3.61111 \times 10^{-8} T^3 - 6.95238 \times 10^{-5} T^2 + 0.0805437\,T - 0.3$$

Use this polynomial to find the temperature T at which $\eta = 20.1\,\mu\mathrm{Pa} \cdot \mathrm{s}$.

Exercise 5.4

An atomic state decays with two time constants τ_1 and τ_2 and can be described by the equation:

$$N = \frac{N_0}{2}\left(e^{-t/\tau_1} + e^{-t/\tau_2}\right)$$

Find, by the method of bisection, the time at which $N = N_0/2$, i.e., the half-life of the state. Let $\tau_1 = 5.697$ and $\tau_2 = 9.446$. The program should first display (with `gnuplot`) the function (use $N_0 = 100.0$) and then prompt the user for an initial bracket. Hints: `gnuplot` does not recognize the variable `t`, so use `x` instead. You may wish to add the command `set xrange [0:15]` just before the `plot f(x)` command to give the plot a nice scaling. Use `exp(x)` for e^x in `gnuplot`.

5.2 The Newton-Raphson Method

If we can calculate the first derivative of our function, then we can use a speedier (although somewhat more dangerous) method called the Newton-Raphson method.

Let us suppose that we have a function as illustrated below, which has a root at $x = x_r$, and that we have a rough estimate for that root of $x = x_0$. Let us evaluate the derivative of this function at x_0, $f'(x_0)$. This derivative will be the slope of the tangent line to the function at the point $(x_0, f(x_0))$, and you can see that where it crosses the x-axis, x_1, is a much better estimate to the root of the function than x_0.

The process can be repeated to give an even better estimate, and so on. The equation of the first tangent line is (the student should verify):

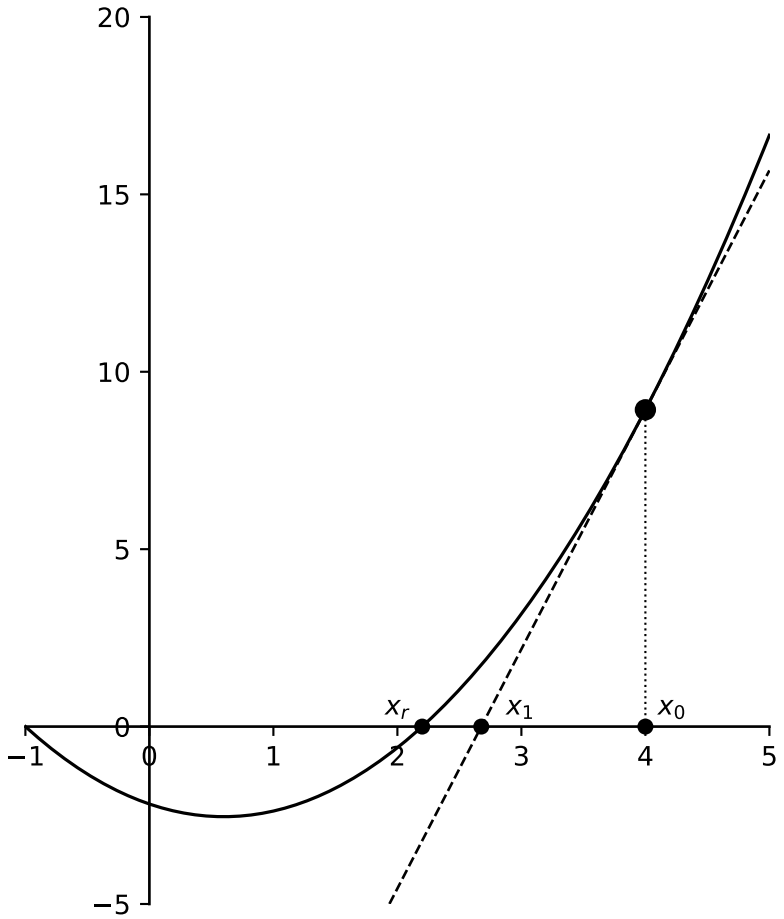
$$y = f'(x_0)(x - x_0) + f(x_0)$$

If we set $y = 0$ to find the root of this line, we get

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

From this, we can derive the Newton–Raphson formula for the n^{th} estimate of the root:

$$x_n = x_{n-1} - \frac{f(x_{n-1})}{f'(x_{n-1})}$$



Exercise 5.5

Write a program to find the root for the polynomial of Exercise 5.1 using the Newton–Raphson method. Begin with an estimate $x_0 = 4$. See what happens if you use a different starting point. Explore both positive and negative values for the starting point.

Exercise 5.6

Write a program to solve the problem of Exercise 5.3 using the Newton–Raphson method. Begin with an estimate of 200 K.

Exercise 5.7

Write a program to solve the problem of Exercise 5.4 using the Newton–Raphson method.

An obvious danger to the Newton-Raphson method is that if there is a minimum or a maximum between the root and your starting point, the method will either not converge to the desired root at all, but to another one, or wander out to $\pm\infty$ at an ever accelerating pace. In addition, if you happen by bad luck to choose your beginning point at a maximum or minimum point, your program will never converge, but will blow up! Hence, as always, you should have a pretty good idea of the nature of your function before you attempt to find and "polish" roots.

Exercise 5.8

Use the fact that

$$\frac{dJ_0(x)}{dx} = -J_1(x)$$

to calculate the first two roots of $J_0(x)$.
The code for $J_1(x)$ is in the file `comphys.c`, and the function definition is

```
float bessj1(float x);
```

which is contained in `comphys.h`.

Exercise 5.9 An Iterative Problem

In ballistics, it is common to solve the equations of motion of a particle shot at an angle, θ, above a horizontal surface with an initial velocity v_0. The effects of air resistance play an important role that is often neglected in introductory or intermediate coursework. If the force of air resistance can be modeled as:

$$F_x = -km\dot{x} \tag{1}$$

$$F_y = -km\dot{y} \tag{2}$$

then the $x(t)$ and $y(t)$ solutions are:

$$x(t) = \frac{v_0 \cos\theta}{k}\left(1 - e^{-kt}\right) \tag{3}$$

$$y(t) = -\frac{gt}{k} + \frac{kv_0 \sin\theta + g}{k^2}\left(1 - e^{-kt}\right) \tag{4}$$

The time of flight of the projectile (the time elapsed before the projectile lands) is given as:

$$T = \frac{v_0 \sin\theta + g}{gk}\left(1 - e^{-kT}\right) \tag{5}$$

Notice that T appears on both sides of equation (5). This is known as a "transcendental equation" which can be solved numerically using iteration. Note that this equation collapses to the simple expression for T when resistance is neglected ($k = 0$):

$$T(k = 0) = \frac{2v_0 \sin\theta}{g} \tag{6}$$

The range of motion (the distance traveled by the projectile before landing) is:

$$R = x(t = T) \tag{7}$$

In the exercises below, use $\theta = 60^\circ$, $v_0 = 600\,\mathrm{m/s}$, and $|g| = 9.8\,\mathrm{m/s^2}$.

a. Solve for T (equation 5) numerically using iteration. In the first iteration, use the non-resistive expression for T (equation 6). Iterate until the solution converges to within a user-defined tolerance (error) – e.g., ΔT over successive iterations is less than the tolerance. Use a value of $k = 0.005$ for this part only.

b. Find the value of k that allows the projectile to travel 1500 meters ($R = 1500\,\mathrm{m}$).

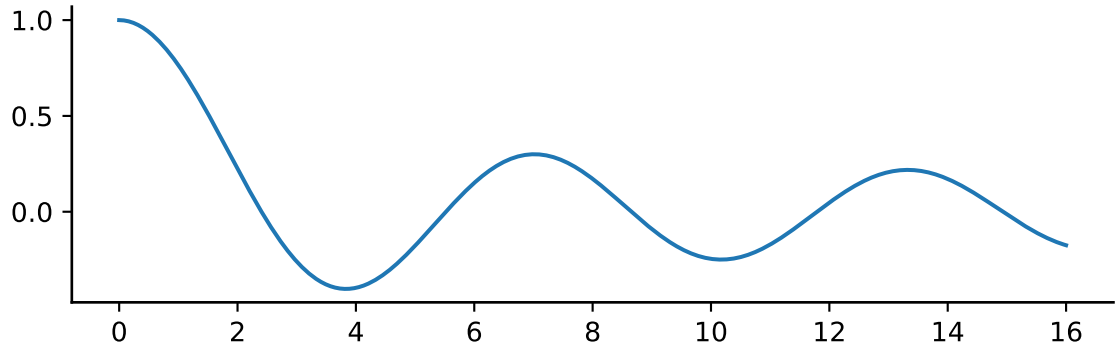
c. [**GRAD STUDENTS ONLY**] Plot y vs. x using gnuplot for k values of 0.0, 0.005, 0.01, 0.02, 0.04, and 0.08. Also, show analytically how to get equation (6) from equation (5) (*Hint: Use perturbation methods – i.e., expand* $1 - e^{-kT}$ *in a power series expansion, and keep only the first few terms*).

5.3 Root Finding in Python

In SciPy all of the root finding methods are in the `optimize` subpackage.

To show off how it is done consider this suspiciously familiar function

```
import numpy as np
from scipy.special import j0
import matplotlib.pyplot as plt
plt.rcParams.update({
  'figure.figsize': (7, 2),
  'axes.spines.top': False,
  'axes.spines.right': False
  })

x = np.linspace(0, 16, 400)
fig, ax = plt.subplots()
ax.plot(x, j0(x))
```

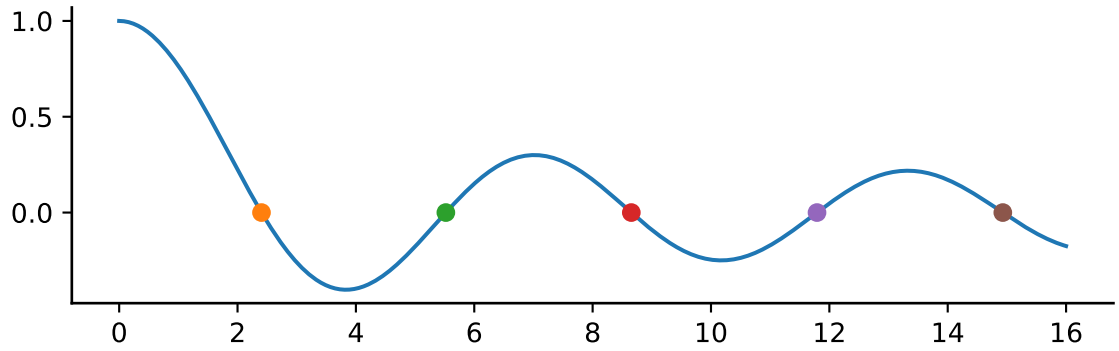


All of the root finding algorithems for scalar functions can be accessed via a single method `root_scalar`.

Let's use a different method for each root! We need to supply a bracket for where to limit our search and a method name.

```
from scipy.optimize import root_scalar

method = ['bisect' ,'brentq' ,'brenth' ,'ridder' ,'toms748']
bracket = [[2,4], [5,7], [8, 10], [11,13], [14, 16]]
results = []

for m, b in zip(method, bracket):
```

```
    results.append(root_scalar(j0, bracket=b, method=m))

fig, ax = plt.subplots()
ax.plot(x, j0(x))

for result in results:
    ax.plot(result.root, 0, 'o')
```



A call to `root_scaler` retuns a `RootResults` object which contains more information than only the root value.

```
results[0]
```

```
      converged: True
           flag: converged
 function_calls: 42
     iterations: 40
           root: 2.404825557694494
         method: bisect
```

For a full overview of the supported methods see the `root_scalar` documentation.

Optimization is a vast subject of which we will cover more in the next chapter.

6 Minimization and Maximization of Functions

The determination of the minima or the maxima of functions, including 1-D and multi-dimensional functions, is an important problem not only in physics but throughout quantitative science. For instance, in physics and astronomy, many quantities come to a maximum at an intermediate value of a certain parameter. In the spectra of stars, the hydrogen lines come to their maximum strength at a temperature of about 10,000 K. Stars with temperatures both lower and higher than this have relatively weak hydrogen lines. The investigation of this phenomenon can lead to insights into the physics of stars. The theory of orbits in classical mechanics leads to the concept of an *effective potential*. Depending upon the nature of the central force and the angular momentum of the particle, the effective potential can have a minimum at a certain radius. A stable circular orbit is then possible at that radius. Around black holes, this effective potential has not only a minimum but also a maximum. At the minimum, stable circular orbits are possible. Inside of the maximum, the particle is irretrievably drawn into the black hole. Knowledge of the location of this maximum is of obvious importance if you are orbiting around a black hole!

The determination of a minimum of a multi-dimensional function is related to the question of optimization. For instance, a common question to ask in physics is: "What combination of parameters yields the best fit of a model to the experimental or observational results?" "

That this is a problem related to minimization becomes clear when one considers a function $f(x_1, x_2, x_3, ...)$ which is equal to the sum of the squared differences between the theoretical model and the experimental results. The variables $x_1, x_2, x_3, ...$ are the model parameters, and we can consider them to define an n-dimensional space. The function f then takes on a value at each point in this n-dimensional space, and the problem reduces to finding the global minimum of f.

Finding the maximum of a function is equivalent to finding a minimum, because if $f(x)$ has a maximum at x_m, then $-f(x)$ has a minimum at the same point x_m. Thus, in what follows, we will consider algorithms to determine the *minimum* of a function only.

Let us first begin with 1-D functions, and then move on to multi-dimensional functions.

6.1 The Minimization of 1-D Functions

Analogous to Chapter 5, in which we considered how to find the root of a 1-D function, we can divide the problem into functions for which we can determine the first derivative and those for

which we cannot. Let us first consider the case for which we cannot determine the derivative, or are too lazy to do so.

6.1.1 Bracketing a Minimum in 1-D

In Section 5.1 we used the method of bisection to zero in on the root of a 1-D function. We can use a similar method here, but we must first define what we mean by "bracketing" a minimum. A minimum can be bracketed only by a triplet of points $a < b < c$ such that $f(b)$ is less than both $f(a)$ and $f(c)$. Only then do we know that the function has a minimum in the interval (a, c) (see Figure 6.1). To zero in on the minimum, we choose a new point x, which is either between a and b, or between b and c. Let us suppose x is between b and c. We evaluate $f(x)$. If $f(b) < f(x)$, then we replace c with x. If, however, $f(b) > f(x)$, then the new bracketing triplet is (b, x, c).
We continue until we know the position of the minimum to a precision consistent with both our needs and the machine precision.

How do we choose x? There are a number of ways to choose x, but a straightforward way is to take x halfway through the larger of the two intervals (a, b) and (b, c).

We use the following algorithm:

- **If** the interval (a, b) is larger than (b, c):
  Take $x = (a + b)/2$ and find $f(x)$.
  - If $f(b) < f(x)$, the new interval is (x, b, c).
  - If $f(b) > f(x)$, the new interval is (a, x, b).
- **If** the interval (b, c) is larger than (a, b):
  Take $x = (b + c)/2$ and find $f(x)$.
  - If $f(b) < f(x)$, the new interval is (a, b, x).
  - If $f(b) > f(x)$, the new interval is (b, x, c).

Exercise 6.1

Using the above algorithm, find the minimum of the polynomial

$$f(x) = x^3 - 24x^2 + 6x + 15$$

using the beginning bracket $(11, 15, 20)$. Print out intermediate results so that you know how many iterations the algorithm went through.

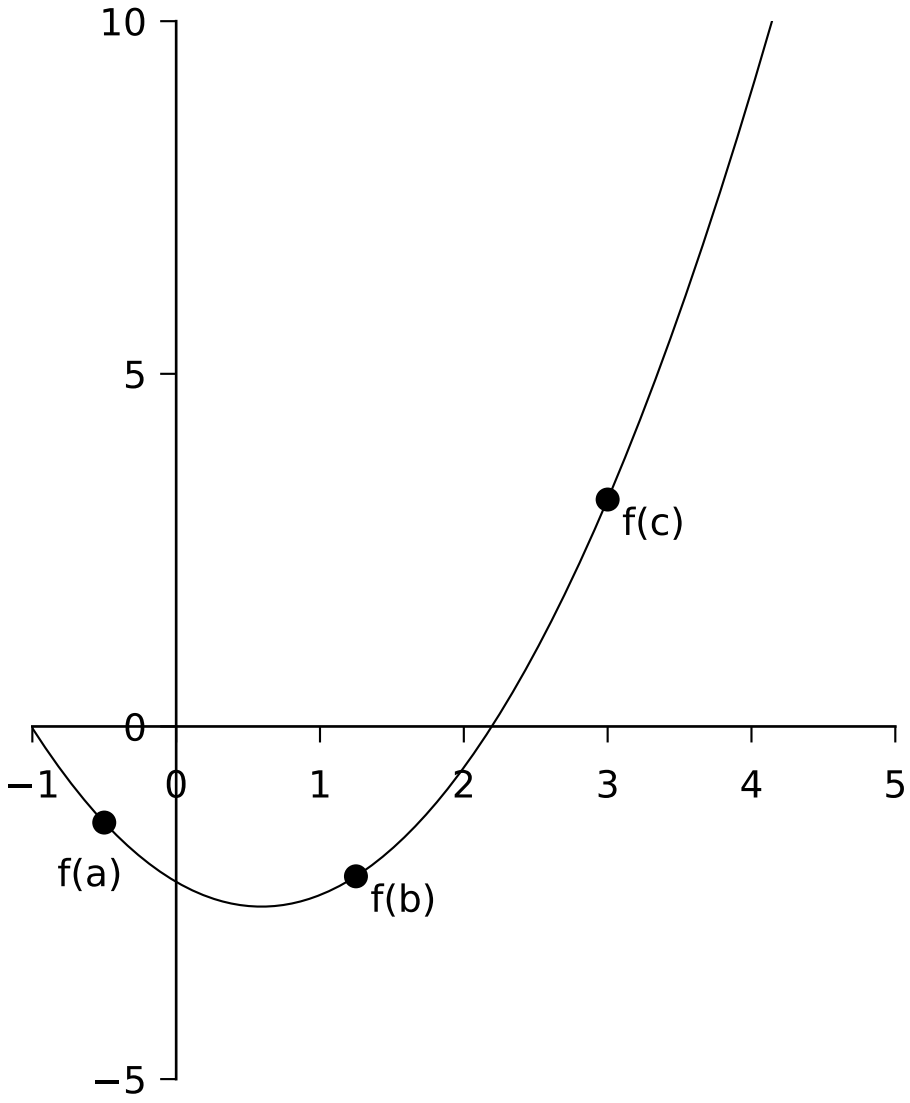


Figure 6.1: A parabola we want to minimize

Exercise 6.2

Legendre polynomials appear in the solutions to Laplace's equation in electrostatics, particularly for systems with spherical symmetries. For example, the general solution for the potential in 3-space, arrived at via separation of variables, takes the form:

$$V(r,\theta) = \sum_{n=0}^{\infty} \left(A_n r^n + \frac{B_n}{r^{n+1}} \right) P_n(\cos\theta)$$

where the coefficients A_n and B_n are determined by boundary conditions, and r and θ are the radial distance and polar angle, respectively, in spherical coordinates.
P_n is the Legendre polynomial for the nth harmonic, defined via the Rodrigues Formula as:

$$P_n(x) = \frac{1}{2^n n!} \left(\frac{d}{dx} \right)^n (x^2 - 1)$$

Write a C program to find the minimum of the $n = 3$ polynomial

$$P_3(x) = \frac{1}{2}(5x^3 - 3x)$$

using the initial bracket $(0.0, 0.5, 1.0)$. Print out intermediate results so that you know how many iterations the algorithm went through.

Exercise 6.3

Recall that the Bessel function $J_0(x)$ is included in `comphys.c` as:

```
float bessj0(float x)
```

Write a program that first invokes gnuplot to plot $J_0(x)$ between $x = 0$ and $x = 15$, and then prompts the user to enter a triplet (`a, b, c`) to bracket one of the displayed minima of $J_0(x)$. The program should then use bracketing and bisection to find the x value of that minimum to a tolerance of 1×10^{-5}. Hint: You can get gnuplot to plot $J_0(x)$ with the command:

```
gnuplot> plot besj0(x)
```

6.1.2 Bracketing a Minimum in 1-D with Derivatives

In the previous section, we cornered the minimum of our function by successively reducing the size of the bracketing interval until we knew the location of the minimum to the desired precision. We did this by arbitrarily selecting a new evaluation point in the center of the *largest* interval, (a, b) or (b, c). Now, this is inefficient, as the minimum may not be in the largest interval, and so our algorithm actually wasted nearly 50% of its steps. If we can calculate the derivative of the function, we can use this information to determine which interval the minimum is actually in and thus avoid wasting steps. Furthermore, if we retain the derivatives of the function at all bracketing points a, b, and c, then we can utilize linear *extrapolation* to predict where the derivative of the function goes to zero — i.e., the location of the minimum. This scheme should zero in on the minimum with far fewer steps than the previous algorithm.

This is how we proceed:

1. Evaluate the function and its derivative at each of the bracketing points a, b, and c.
2. Examine $f'(b)$:
   - If $f'(b) > 0$, the minimum is in the interval (a, b).
   - If $f'(b) < 0$, the minimum is in (b, c). See Figure 6.2.
3. If $f'(b) < 0$, use $f'(a)$ and $f'(b)$ to extrapolate the derivative to 0.
   The straight line that passes through the points $(a, f'(a))$ and $(b, f'(b))$ is given by:

   $$y' = f'(a) + \frac{f'(b) - f'(a)}{b - a}(x - a)$$

   (The student should verify this). This can be used to extrapolate to $y' = 0$ by setting $y' = 0$ and solving for x, yielding:

   $$x = a - \frac{f'(a)(b - a)}{f'(b) - f'(a)}$$

   We then calculate $f(x)$ and $f'(x)$, and the new interval is (b, x, c).
4. If $f'(b) > 0$, use $f'(b)$ and $f'(c)$ to extrapolate the derivative to 0.
   The straight line that passes through the points $(b, f'(b))$ and $(c, f'(c))$ is:

   $$y' = f'(b) + \frac{f'(c) - f'(b)}{c - b}(x - b)$$

   (The student should verify this.)

   This can be used to extrapolate to $y' = 0$ by setting $y' = 0$ and solving for x, yielding:

$$x = b - \frac{f'(b)(c-b)}{f'(c) - f'(b)}$$

We then calculate $f(x)$ and $f'(x)$, and the new interval is (a, x, b).

5. Go back to step (2) and continue until the minimum is isolated to the desired accuracy.

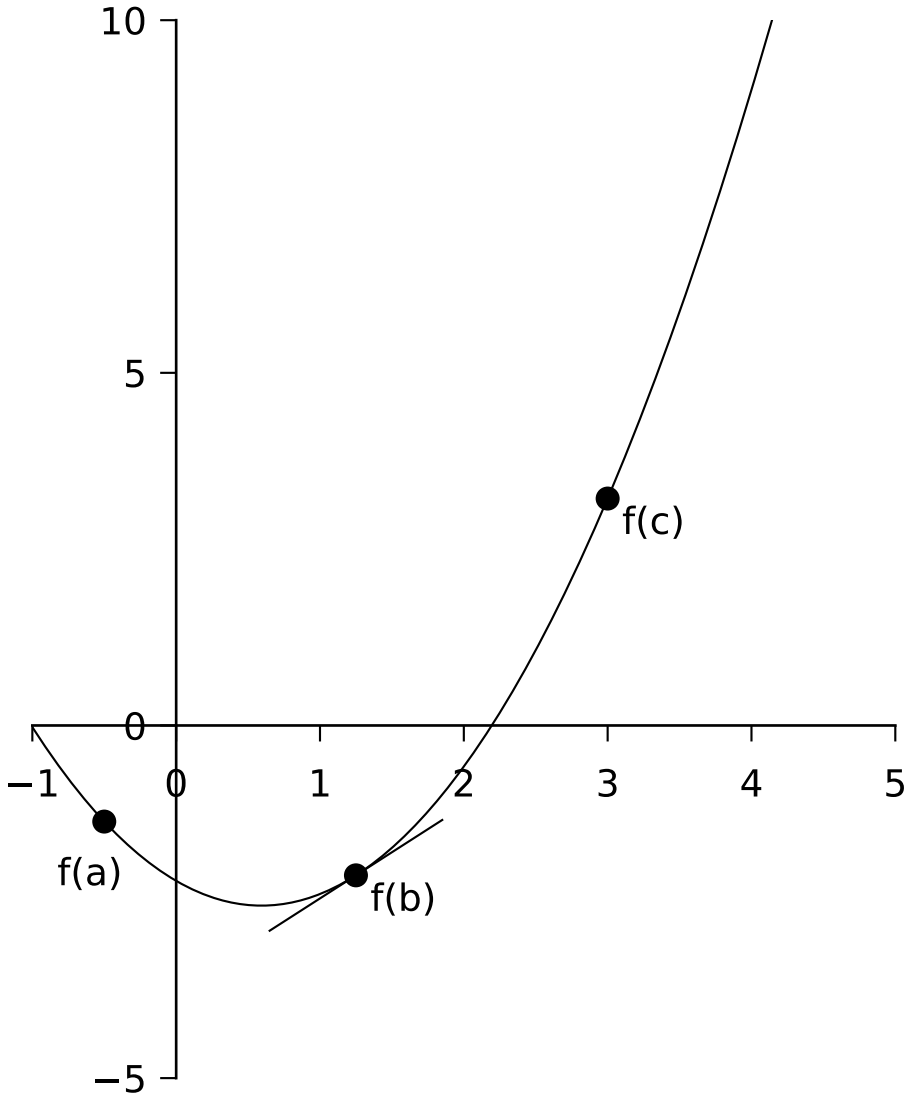


Figure 6.2: A parabola with derivative information

Exercise 6.4

Modify your program for **Exercise 6.1** to include derivative information.
Does the modified program take fewer steps to converge on the minimum?
Does it do a better job at finding the minimum? How do you know?

Exercise 6.5

Re-do **Exercise 6.2** using derivative information.

Exercise 6.6

Re-do **Exercise 6.3** using derivative information. Recall that:

$$\frac{dJ_0(x)}{dx} = -J_1(x)$$

Exercise 6.7 — The Derivation of Wien's Law

Wien's law is a property of the blackbody spectrum and can be written as:

$$\lambda_{\max}(\text{Å}) = \frac{2.9 \times 10^7 \ \text{ÅK}}{T(\text{K})}$$

meaning that the wavelength of the maximum of the blackbody (Planck) spectrum is inversely proportional to the temperature.
The flux emitted by a blackbody (in units of erg s ^{1} cm ^{2} Å ^{1}) is given by:

$$B(\lambda) = \frac{1.19106 \times 10^{27}}{\lambda^5 \left(e^{1.43879 \times 10^8/(\lambda T)} - 1\right)}$$

This function is coded in the C function `planck` found in **comphys.c**. The function definition is:

```
float planck(float wave, float T);
```

where wave is the wavelength in Ångströms and T is the temperature in Kelvin.
Use the method of this section to determine the wavelength of the maximum of $B(\lambda)$ for the following temperatures: $T = 2000\text{K}, 5000\text{K}, 8000\text{K}, 11000\text{K}, 14000\text{K}$ and use gnuplot to plot these wavelengths against $1/T$ and show that the slope of the line is $2.9 \times 10^7, \text{ÅK}$. To learn how to get gnuplot to do this for you, see the "aside" below. *Hint*: keep in mind that $B(\lambda)$ has a *maximum*, so determine the minimum of $-B(\lambda)$. Another question to consider — how will you find the initial values of (a, b, c) which bracket the minimum of $-B(\lambda)$? This is an important question — you will need to devise a C function that will come up with these three numbers.

6.1.3 An Aside - How to get Gnuplot to fit a line to data

Let us suppose that we have some data in the file `data.dat` and we want to get gnuplot to fit a regression line to those data. This is how to do it manually – the same commands can be used in "response" mode. Carry out the following steps:

```
gnuplot > plot "data.dat" using 1:2
gnuplot > f(x) = a + b*x
gnuplot > fit f(x) "data.dat" using 1:2 via a,b
gnuplot > replot f(x)
```

Gnuplot will calculate the regression line, and then display the values of the coefficients a and b along with associated errors.

How does one plot, say, the reciprocal of the first column of `data.dat` against the square of the second column? Use the following gnuplot command:

```
gnuplot > plot "data.dat" using (1/$1):($2*$2)
```

6.2 The Minimization of Multi-dimensional Functions

The subject of the minimization of multidimensional functions, i.e. functions of the form $y = f(x_1, x_2, x_3, ...)$ is a huge subject, and we will only be able to scratch the surface.
We begin with the so-called *downhill simplex method*.

6.2.1 The Multidimensional Downhill Simplex Method

A *simplex* is a geometrical figure consisting, in N dimensions, of $N+1$ vertices and all their interconnecting line segments, faces, etc. For $N = 2$, the simplex is a triangle; for $N = 3$, it is a tetrahedron, etc. In 1-D minimization, it is possible to bracket the minimum, but in multidimensional space, this is impossible. What we can do, however, is to *surround* the minimum with a *simplex* and then shrink that *simplex* until we know the position of the minimum to the desired precision. The way the *simplex* method works is as follows. Let us suppose we begin with a rough estimate of the position of the minimum in N-dimensional space. This point $(x_1, x_2, \ldots, x_N)$ makes up one of the vertices of the simplex. We then supply the remaining vertices of the simplex by adding a small value to each of the coordinates, thus:

$$
\begin{aligned}
&(x_1 + \Delta x_1,\ x_2,\ \ldots,\ x_N),\\
&(x_1,\ x_2 + \Delta x_2,\ \ldots,\ x_N),\\
&\ldots,\\
&(x_1,\ x_2,\ \ldots,\ x_N + \Delta x_N)
\end{aligned}
$$

The Δx_j's can all be the same, or they can be different, according to the needs of the problem. These $N+1$ points define the initial simplex.

The downhill simplex method now takes a number of steps in the "downhill" direction in an attempt to find the minimum of the function. Most of these steps are in the form of reflections, in which the "highest" vertex of the simplex (i.e. the vertex at which the function evaluates to the highest value) is reflected through the opposite face of the simplex to a lower point. Other steps (see Figure 6.3) involve reflection and expansion, or contraction. The goal is to surround the minimum and then contract on it until we know the minimum to the required precision. If you could watch the simplex in action, it would appear somewhat like an amoeba, first pursuing its prey and then engulfing it.

The C function `amoeba` — from *Numerical Recipes* (Press et al. 1992) — implements the downhill simplex method. Since this algorithm may be a bit difficult for you to code yourself, we have given you this routine in `comphys.c`, and you will explore a number of applications of this routine.

The function definition for amoeba (contained in `comphys.h`) is:

```
void amoeba(float **p, float y[], int ndim, float ftol,
            float (*funk)(float []), int *nfunk)
```

Here are the specific instructions for the use of `amoeba`, taken from *Numerical Recipes* (Press et al. 1992):

> Multidimensional minimization of the function `funk(x)` where `x[1...ndim]` is a vector in `ndim` dimensions, by the downhill simplex method of Nelder and Mead. The

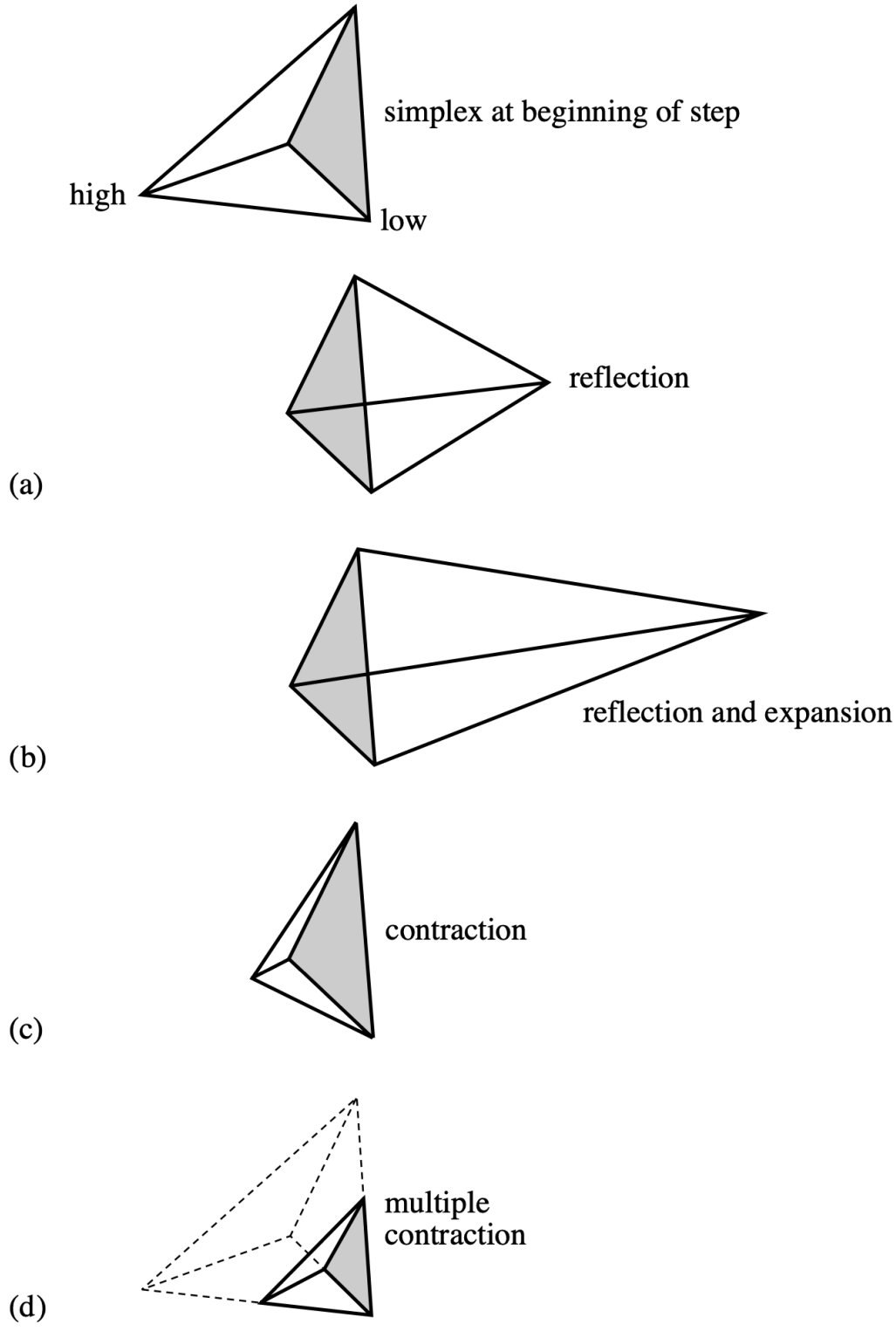


Figure 6.3: Possible outcomes for a step in the downhill simplex method

matrix `p[1..ndim+1][1..ndim]` is input. Its `ndim + 1` rows are `ndim`-dimensional vectors which are the vertices of the starting simplex. Also input is the vector `y[1..ndim+1]`, whose components must be preinitialized to the values of `funk` evaluated at the `ndim + 1` vertices (rows) of `p`; and `ftol`, the fractional convergence tolerance to be achieved in the function value. On output, `p` and `y` will have been reset to `ndim + 1` new points all within `ftol` of a minimum function value, and `nfunk` gives the number of function evaluations taken.

This certainly sounds complicated, but it is fairly easy to use in practice.
For an example, let us look at a C-program that will implement `amoeba` to solve a simple minimization problem where we find the minimum of the function

$$f(x, y) = x^2 + (y - 1)^2 + 1.0$$

```
#include <stdio.h>
#include <math.h>
#include "comphys.h"
float f(float x[]);

int main()
{
  float **p,*y;
  int nfunk,i;
  float x0,y0;

  /* Allocate memory for the p matrix and the y vector */

  p = matrix(1,3,1,2);
  y = vector(1,3);

  /* initial point */

  x0 = 5.0;
  y0 = 5.0;

  /* define the vertices of the simplex - in this case a triangle */

  p[1][1] = x0;
  p[1][2] = y0;
  p[2][1] = x0 + 1.0;
  p[2][2] = y0;
  p[3][1] = x0;
```

```
  p[3][2] = y0 + 1.0;

  /* initiate y[i] */

  for(i=1;i<=3;i++) y[i] = f(p[i]);

  /* invoke amoeba with ftol = 0.001 */

  amoeba(p,y,2,0.001,f,&nfunk);

  /* the best value for the minimum is obtained by averaging the final
  p's. */

  x0 = (p[1][1]+p[2][1]+p[3][1])/3.0;
  y0 = (p[1][2]+p[2][2]+p[3][2])/3.0;

  printf("\nThe minimum is found at x0 = %f, y0 = %f\n",x0,y0);
  printf("The number of function evaluations is %d\n",nfunk);
  return(0);
}

/* here is our function that we want to miminize */

float f(float x[])
{
  float z;
  z = x[1]*x[1] + (x[2]-1.0)*(x[2]-1.0) + 1.0;
  /* print out intermediate values so we can see what is happening */
  printf("\nx[1] = %f x[2] = %f z = %f",x[1],x[2],z);
  return(z);
}
```

The initial simplex looks like this:

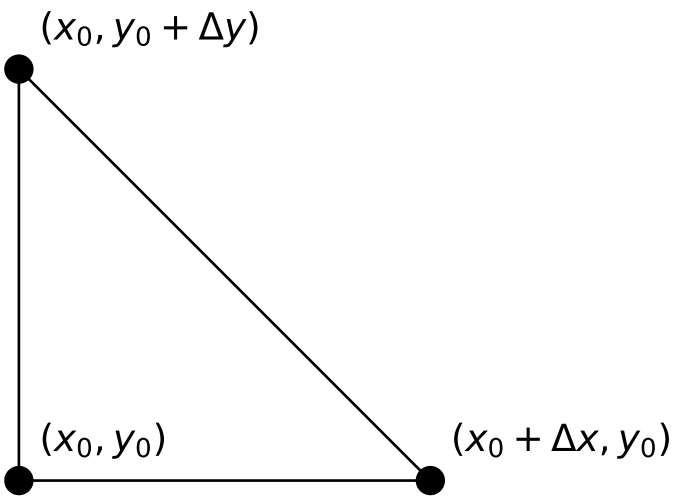
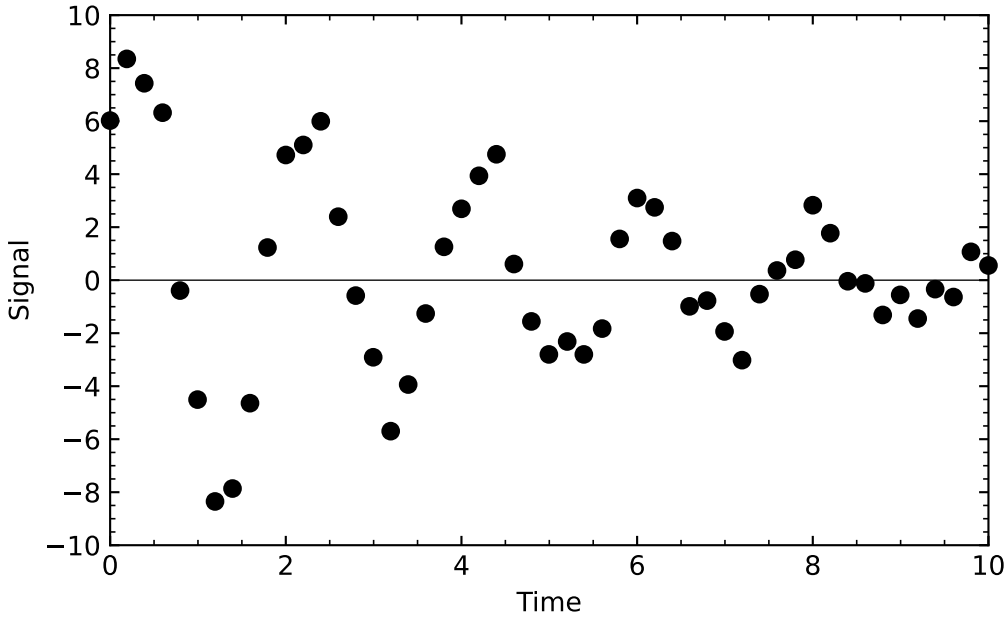


Figure 6.4: A 3D simplex creates a triangle to encapulate the minima

In this case, both Δx and Δy are 1.

Exercise 6.8

In this exercise, we explore a very useful application of function minimization — optimizing a fit to a model. Consider the following data, which come from an exponentially decaying oscillating system:



The data are slightly noisy, but we can model these data with the equation

$$f(t) = Ae^{-\gamma t}\sin(Bt + C)$$

We can consider the parameters A, γ, B, and C as variables, and ask our simplex algorithm `amoeba` to roam around in this four-dimensional space seeking the best fit (i.e., the fit with the smallest residual or χ^2) between this model and the data in the above figure. These data are contained in the file `data/decay.out`. The first thing you must do is to read these data into your program. Consider the following code fragment, which can serve as the beginning of your program:

```
#include <stdio.h>
#include <math.h>
#include "comphys.h"
float *t,*yi;
int k;
float f(float x[]);

int main()
{
  float **p,*y;
  int nfunk,i;
  float A,gam,B,C;
  FILE *in;

  p = matrix(1,5,1,4);
  y = vector(1,5);
  t = vector(0,200);
  yi = vector(0,200);

  if((in = fopen("decay.out","r")) == NULL) {
    printf("\nCannot find input file\n");
    exit(1);
  }

  k = 0;
  while(fscanf(in,"%f %f",&t[k],&yi[k]) != EOF) k++;

  fclose(in);

  // ...

}
```

What this code fragment does is to define two external vectors, `*t` and `*yi`, and then allocate space for them in the lines:

```
t = vector(0,200);
yi = vector(0,200);
```

The next lines in the code open the file `decay.out` and then read the data from `decay.out` into the vectors `t` and `yi`. The file handle `in` is then closed.

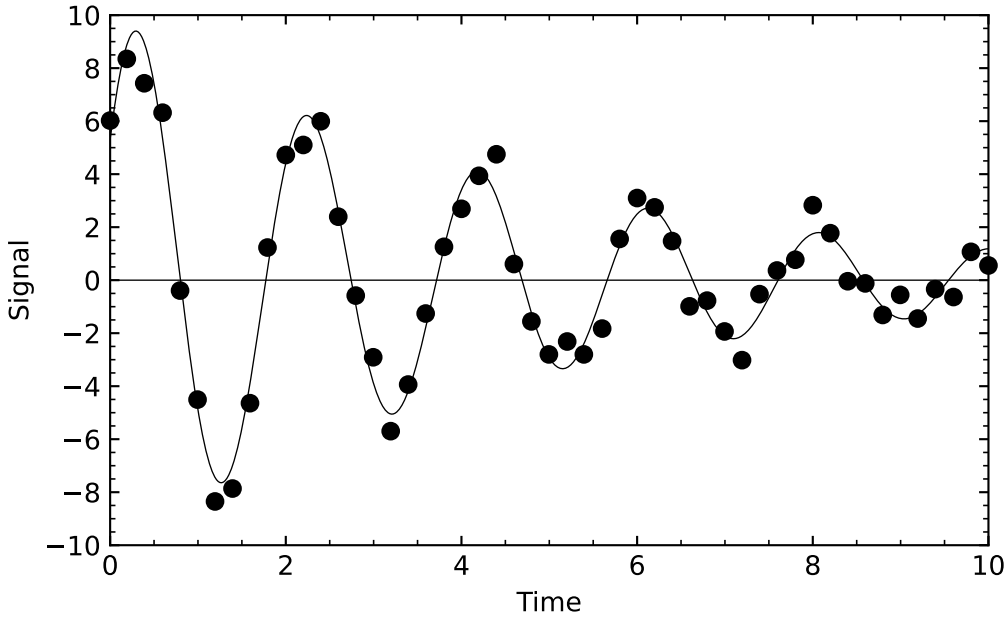
The reason why we used external vectors was so that our function `f`, which is used by `amoeba`, would have access to these vectors. The function `f` calculates the residual or χ^2 between the model and the data in `decay.out` using:

$$\chi^2 = \sum_{i=0}^{k} (y_i - f(t_i))^2$$

This value for χ^2 is then returned. This is accomplished in the code:

```
float f(float x[])
{
  float y1,chi2;
  int i;

  chi2 = 0.0;
  for(i=0;i<k;i++) {
    y1 = x[1]*exp(-x[2]*t[i])*sin(x[3]*t[i] + x[4]);
    chi2 += (y1 - yi[i])*(y1 - yi[i]);
  }
  printf("\nchi2 = %f",chi2);
  return(chi2);
}
```

What remains for you to do is to figure out reasonable starting values for A, γ, B, and C, use them to initialize the components of p, and then to invoke amoeba. The next figure shows a plot of what I got.



Exercise 6.9

The restricted 3-body problem of classical mechanics leads to a non-dimensionalized potential energy surface given by the equation:

$$V(x,y) = \left\{ \frac{\xi_2}{[(x-\xi_1)^2+y^2]^{1/2}} - \frac{\xi_1}{[(x-\xi_2)^2+y^2]^{1/2}} - \frac{1}{2}(x^2+y^2) \right\}$$

where $0 < \xi_1 < 1$ and $\xi_2 = \xi_1 - 1$. It turns out that this potential surface has 5 extrema, three minima (L_1, L_2, and L_3), which all lie along the x-axis, and two maxima (L_4 and L_5). For small values of ξ_1, L_4 will be in the second quadrant and approximately located at a point 1 unit away from the origin along a line inclined 60° to the $-x$ axis. As ξ_1 increases, L_4 moves closer to the y-axis and lies on the y-axis for $\xi_1 = 0.5$. L_5 is located similarly in the third quadrant. Write a program that will prompt the user for ξ_1, check that $\xi_1 \leq 0.50$, and then find the coordinates of L_4 using the simplex method.

6.2.2 Powell's Method

The downhill simplex method has the virtue that it is easy to implement, and often will do the job as well as other much more sophisticated methods. A problem with the simplex method is that it uses more function evaluations than other multidimensional minimization techniques, and if evaluating your function is computationally expensive, then the simplex method can be

very slow. So, if you are dealing with a complicated, computationally expensive function, it is worthwhile to investigate other methods.

One straightforward way to minimize a multidimensional function is to use the methods of Section 6.1 to minimize first along one direction, then along the next and so on until you spiral into the minimum — i.e., if $y = f(x_1, x_2, x_3, ...)$, hold $x_2, x_3, ...$ constant and minimize f along x_1, then hold $x_1, x_3, ...$ constant and minimize along x_2, and so on, cycling through the directions until you know the minimum to the desired precision. This is actually a very good algorithm to follow in most situations, especially if the minimum of your function is nice and symmetric, as is quite often the case. However, the method becomes very inefficient if the minimum is found at the bottom of a long, narrow valley. Then the method will have to cycle through the directions many, many times, bouncing against the sides of this valley to get to the minimum.

The key is to abandon the coordinate directions $x_1, x_2, x_3, ...$ and instead use a new basis set of "conjugate directions", with the idea that the first of these directions will be along the direction of maximum descent — i.e., along the direction of that long narrow valley. The algorithm moves as far as it can along this direction; it then abandons that direction and finds the next direction of maximum descent, and so on, until the actual function minimum is found. This is the basis of Powell's method, which is implemented in the C-function `powell` found in `comphys.c`. The function definition is

```
void powell(float p[], float **xi, int n, float ftol,
int *iter, float *fret, float (*func)(float []))
```

The *Numerical Recipes* instructions for this function are as follows:

> Minimization of a function `func` of `n` variables. Input consists of an initial starting point `p[1...n]`; an initial matrix `xi[1..n][1..n]`, whose columns contain the initial set of directions (usually the n unit vectors); and `ftol`, the fractional tolerance in the function value such that failure to decrease by more than this amount on one iteration signals doneness. On output, `p` is set to the best point found, `xi` is the then-current direction set, `fret` is the returned function value at `p`, and `iter` is the number of iterations taken.

Let us use Powell's method to minimize the function $f(x, y) = x^2 + (y-1)^2 + 1.0$ of the previous section. The following program does the job:

```
#include <stdio.h>
#include <math.h>
#include "comphys.h"
float f(float x[]);

int main()
```

```
{
  float *p,**xi,fret;
  int iter=0;
  int i,j;
  float x0,y0;

  /* Allocate memory for the p vector and the xi matrix */
  p = vector(1,2);
  xi = matrix(1,2,1,2);

  /* initial point */
  x0 = 5.0;
  y0 = 5.0;

  p[1] = x0;
  p[2] = y0;

  /* Initial directions */
  for(i=1;i<=2;i++) {
    for(j=1;j<=2;j++) {
        if(i == j) xi[i][j] = 1.0;
        else xi[i][j] = 0.0;
    }
  }

  /* invoke powell with ftol = 0.001 */

  powell(p,xi,2,0.001,&iter,&fret,f);

  printf("\nThe minimum is found at x = %f, y = %f",p[1],p[2]);
  printf("\nThe value of f at the minimum is %f\n",fret);
  return(0);
}

/* here is our function that we want to minimize */

float f(float x[])
{
  char tmp[10];
  float z;
  z = x[1]*x[1] + (x[2]-1.0)*(x[2]-1.0) + 1.0;
  /* print out intermediate values so we can see what is happening */
```

```
  printf("\nx[1] = %f x[2] = %f z = %f",x[1],x[2],z);
  return(z);
}
```

> **Exercise 6.10**
>
> Repeat Exercise 6.8 using Powell's method.

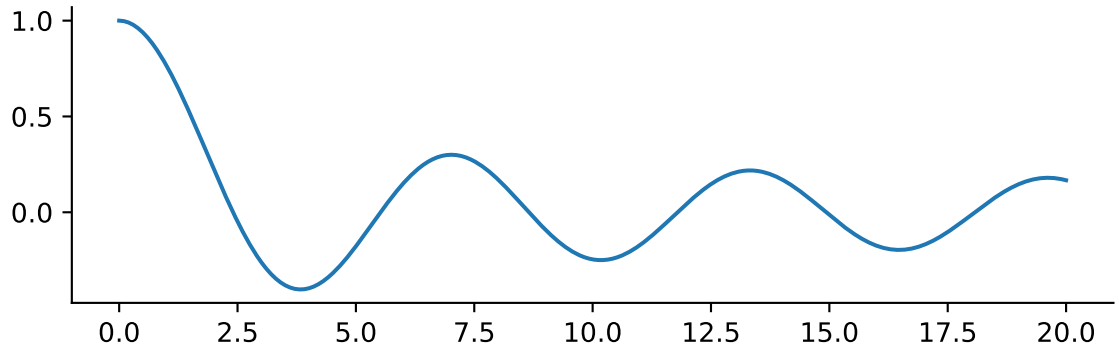
> **Exercise 6.11**
>
> Repeat Exercise 6.9 using Powell's method.

6.3 Optimization in Python

Optimization is a vast subject. SciPy has got you covered with the optimize subpackages, which we previously met in Chapter 5 for all of our root finding needs.

For scalar functions the `minimize_scalar` method is used. As an example we will return to our favorite wiggle

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import j0
from scipy.optimize import minimize_scalar
plt.rcParams.update({
  'figure.figsize': (7, 2),
  'axes.spines.top': False,
  'axes.spines.right': False
  })

x = np.linspace(0, 20, 400)
plt.plot(x, j0(x))
```



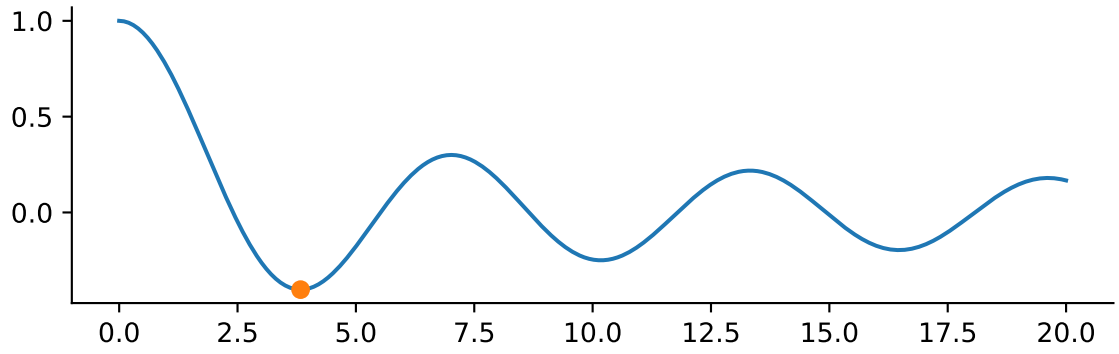
The call to `minimize_scalar` returns an object with other information about the result.

```
result = minimize_scalar(j0)
result
```

```
 message:
          Optimization terminated successfully;
          The returned value satisfies the termination criteria
          (using xtol = 1.48e-08 )
 success: True
     fun: -0.4027593957025531
       x: 3.8317059554863437
     nit: 9
    nfev: 13
```

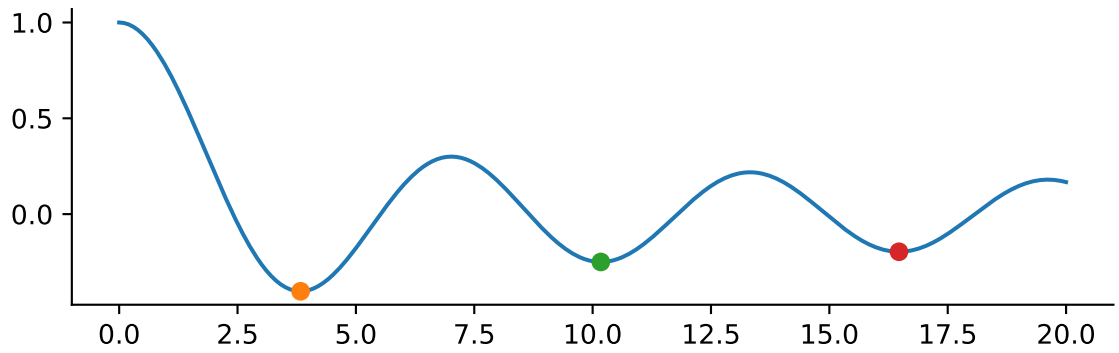
Notice there are many function evaluations. If your function is expensive to evaluate, minimization can be expensive.

```
plt.plot(x, j0(x))
plt.plot(result.x, result.fun, 'o')
```



Note this is a local minium. Passing a bracket we can find multiple local minima

```
results = []
bracket = [[2,4], [8, 10], [14, 16]]

for b in bracket:
    results.append(minimize_scalar(j0, bracket=b))

fig, ax = plt.subplots()
ax.plot(x, j0(x))

for mini in results:
    ax.plot(mini.x, mini.fun, 'o')
```



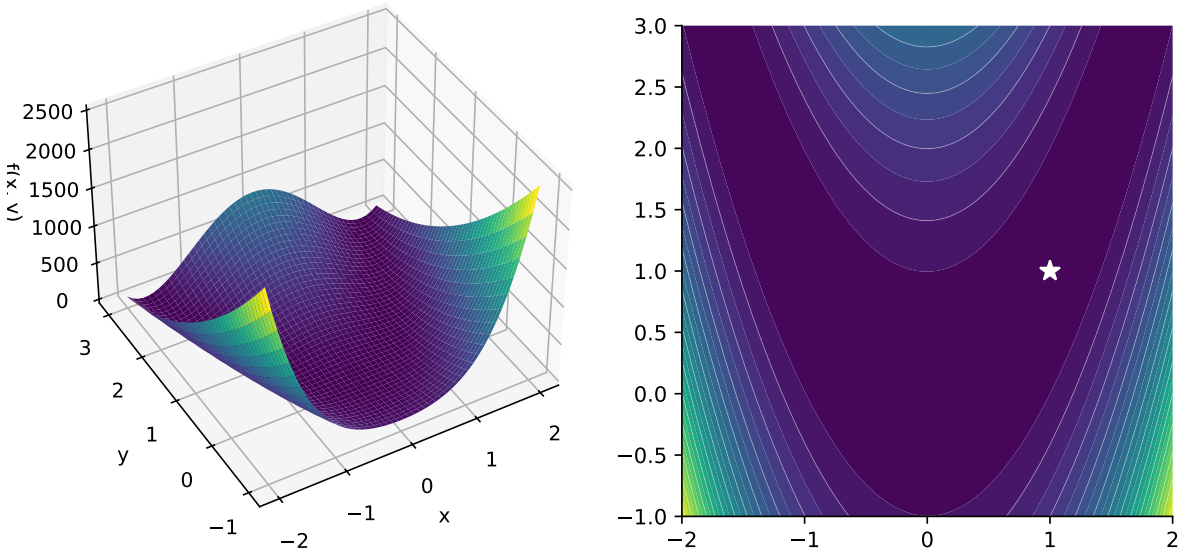
For a multi-dimensional example consider the example of the Rosenbrock function

$$f(\mathbf{x}) = \sum_{i=1}^{N-1} 100\left(x_{i+1} - x_i^2\right)^2 + \left(1 - x_i\right)^2$$

Don't worry too much about the exact form of the equation. The Rosenbrock function is a horseshoe type valley shown in Figure 6.5a. The Rosenbrock function is used as a benchmark for optimization techniques as it can be difficult for methods to navigate the shallow curved valley.

The Rosenbrock function as a minimum at $f(x, y) = f(1, 1) = 0$ shown in Figure 6.5b.

The Rosenbrock function is so commonly used in optimization that SciPy provides an method `rosen`.



(a) The funtion forms a shallow horseshoe valley (b) The minimum of the fucntion is at $(x, y) = (1, 1)$

Figure 6.5: The Rosenbrock function

```
from scipy.optimize import rosen

print(rosen([1, 2]))
print(rosen([-15, 25]))
print(rosen([1,1]))
```

```
100.0
4000256.0
0.0
```

Let's say we start at a point $(-1.5, 2.5)$ and we want to minimize the function. For multivariet minimization, SciPy provides the `minimize` method.

```
from scipy.optimize import minimize
x0 = np.array([-1.5, 2.5])
result = minimize(rosen, x0)
result
```

```
  message: Optimization terminated successfully.
  success: True
```

```
   status: 0
      fun: 2.008444162310784e-11
        x: [ 1.000e+00  1.000e+00]
      nit: 36
      jac: [-3.771e-07  1.852e-07]
 hess_inv: [[ 5.000e-01  1.000e+00]
            [ 1.000e+00  2.005e+00]]
     nfev: 144
     njev: 48
```

Again as with the root finding methods, the results contain more than just the minimum. In this case we also get the number of function evaluates and an estimate of the jacobian and hessian. The defauly method is `'BFGS'` but you can pass which method you want to use such as `Nelder-Mead`

```
minimize(rosen, x0, method='Nelder-Mead')
```

```
       message: Optimization terminated successfully.
       success: True
        status: 0
           fun: 6.446298475670153e-10
             x: [ 1.000e+00  1.000e+00]
           nit: 97
          nfev: 182
 final_simplex: (array([[ 1.000e+00,  1.000e+00],
                        [ 1.000e+00,  9.999e-01],
                        [ 1.000e+00,  1.000e+00]]), array([ 6.446e-10,  1.465e-
09,  1.635e-09]))
```

in which case we are returned the final simplex. Notice there were a few more function evaluations but not so bad compared to the default.

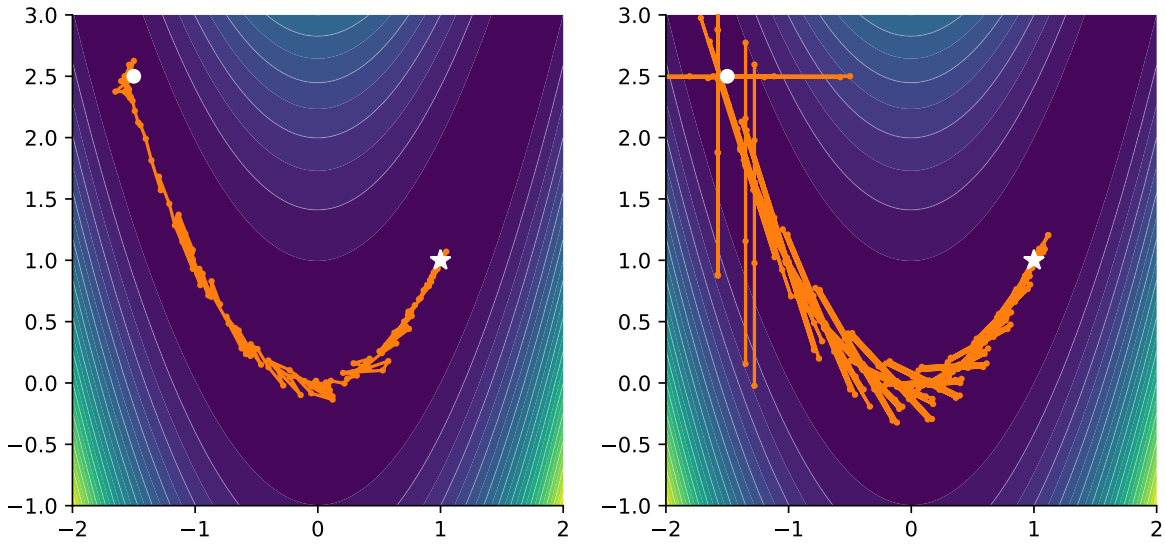
Consider our good old friend `Powell`

```
minimize(rosen, x0, method='Powell')
```

```
 message: Optimization terminated successfully.
 success: True
  status: 0
     fun: 9.860761315262648e-30
       x: [ 1.000e+00  1.000e+00]
```

```
  nit: 26
direc: [[ 4.521e-02  9.181e-02]
        [ 1.075e-07  1.814e-07]]
 nfev: 686
```

and notice how poorly it performs on this function. Navigating around the valley resulted in many more function evaluations. If can be instructive to view both methods side by side and see what path they take.



(a) Nelder-Mead: Takes a rough path but gets there in due time

(b) Powell: Really struggles with the diagonal route.

Figure 6.6: Comparing optimization traces

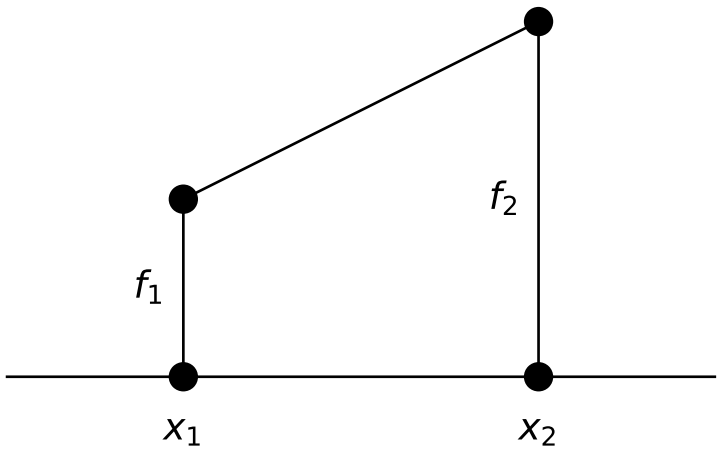
7 Numerical Integration

7.1 Elementary Algorithms

Let us suppose we are confronted with a function $f(x)$ tabulated at the points $x_1, x_2, x_3, \ldots, x_n$, not necessarily equally spaced. We require the integral of $f(x)$ from x_1 to x_n. How do we proceed? The simplest algorithm we can use, valid both for equally and unequally spaced points, is the *trapezoidal rule*. The trapezoidal rule assumes that the function is linear between the tabulated points. With this assumption, it can be seen that the integral from x_1 to x_2 is given by

$$\int_{x_1}^{x_2} f(x)\,dx \approx \frac{h}{2}(f_1 + f_2)$$

where $h = x_2 - x_1$.



This equation can be extended to n points.

For equally spaced points

$$\int_{x_1}^{x_n} f(x)\,dx \approx h\left(\frac{1}{2}f_1 + f_2 + f_3 + ... + \frac{1}{2}f_n\right) \tag{1}$$

and

$$\int_{x_1}^{x_n} f(x)\,dx \approx \frac{1}{2}((x_2 - x_1)(f_1 + f_2) + (x_3 - x_2)(f_2 + f_3) + ... + (x_n - x_{n-1})(f_{n-1} + f_n))$$

for unevenly spaced points.

Exercise 7.1

Use the trapezoidal rule to evaluate the integral

$$\int_1^2 x \sin x\,dx$$

with 10 points. Double the number of points and re-evaluate the integral. The "exact" value of this integral is 1.440422. Rewrite your program to evaluate the integral at an arbitrary number of points input by the user. Are 1000 points better than 100 points?

For equally spaced points, the trapezoidal rule has an error $O(h^3)$, which means that the error for each interval in the rule is proportional to the cube of the spacing. This means, of course, that if the interval size is reduced, the error decreases. In terms of the total number of points (N) used, the total error for the trapezoidal rule can be expressed as $O(1/N^2)$ — i.e., if we double the number of points, the error goes down by a factor of 4. Eventually, however, we run into the law of diminishing returns — either we hit the limit of machine accuracy, or the cumulative error involved in using many tiny intervals begins to build and becomes unacceptably high. In addition, if we use very small intervals, we pay dearly in computer time for the many function evaluations. If the function is complex and thus expensive to compute, this can become prohibitive.

Despite these disadvantages, the trapezoidal rule is adequate for most applications, and it is the only choice if your function is tabulated at unequally spaced points. For those applications which require more accuracy than the trapezoidal rule permits, or which are attempting to integrate very complex functions, "higher order" integration schemes are available. The most popular is *Simpson's rule* which is based on parabolic interpolation. The formula for Simpson's rule, a three-point formula, is:

$$\int_{x_1}^{x_3} f(x)\,dx \approx \frac{h}{3}\left(\frac{1}{3}f_1 + \frac{4}{3}f_2 + \frac{1}{3}f_3\right) + O(h^5) \tag{2}$$

The "extended" version of this formula over N steps (note that N must be odd) is:

$$\int_{x_1}^{x_N} f(x)\,dx \approx \frac{h}{3}(f_1 + 4f_2 + 2f_3 + 4f_4 + ... + 2f_{N-2} + 4f_{N-1} + f_N) + O\left(\frac{1}{N^4}\right)$$

Exercise 7.2

Calculate the integral of Exercise 7.1 with Simpson's rule using 9 points in the interval $(1, 2)$, and then repeat the exercise with 19 points. Compare with the results from Exercise 7.1 for 10 and 20 points respectively.

Note that if you have a tabulated function with an *even* number of points, you can still use Simpson's rule. Simply evaluate the integral over the first interval using the trapezoidal rule and then use Simpson's rule for the remaining odd number of points.

Even higher-order formulae can be used to carry out numerical integration, but in practice, these are hardly ever used. There are advanced techniques, which we will not go into, that can extract virtually any desired precision from the trapezoidal and Simpson's rules. Thus, it is hardly ever necessary to utilize higher-order formulae.

Exercise 7.3

To explore one of these "advanced techniques", go back to the program you wrote for Exercise 7.1 and rewrite it to calculate the value of the integral with $N = 30$ and $N = 80$ points. Setting $x = 1/N^2$ and $y =$ the value of the integral for each value of N defines two points. The program should find the equation of the line between these two points. The y-intercept of this equation should be a very accurate approximation to the integral. Why? Since $x = 1/N^2$[1], the y-intercept (where $x = 0$) corresponds to $N = \infty$. The program should print out the value of the integral and its deviation from the "exact" value 1.440422 for $N = 30$, 80, and "∞" points.

7.2 Gaussian Integration

In the previous section, we found that we could obtain significantly higher accuracy in numerical integration using the same number of points in a given interval by changing the *weighting* used for the points. For instance, in trapezoidal integration, each point (except for the end points) enters into the formula (Equation 1) with equal weights. In Simpson's rule (Equation 2) the points are not given equal weights. Note that the odd *abscissas* are given weights of 2/3, whereas the even abscissas are given weights of 4/3.

[1]Why do we set $x = 1/N^2$? Because the error in the trapezoidal method goes as $O(1/N^2)$, and thus the error in the determination of the integral should scale linearly with $1/N^2$.

It turns out that if we relax the restriction that the abscissas should be evenly spaced in the interval, we can then choose abscissas and corresponding weights that yield very high accuracies. This is the basis of *Gaussian integration*. We do not have the time to go into this method in great detail, but it is important that you are at least acquainted with this method of integration, as it is commonly and widely used.

Gaussian integration is based on integrals of the form

$$\int_a^b W(x)\, f(x)\, dx$$

which can be approximated by the sum

$$\sum_{i=1}^{N} w_i\, f(x_i)$$

The weights (w_i) and the abscissas (x_i) can be chosen, for a given $W(x)$, to make the approximation *exact* if $f(x)$ is a polynomial. If $f(x)$ can be "well approximated" by a polynomial, then the integration formula above yields very high accuracy. Different $W(x)$'s, of course, lead to different choices for the abscissas and weights.

For example:

- $W(x) = 1$ leads to **Gauss–Legendre** integration
- $W(x) = (1 - x^2)^{-1/2}$ leads to **Gauss–Chebyshev** integration

We will concentrate here on the simplest and most useful case, **Gauss–Legendre integration**.

For a given number of abscissas, it is possible to compute the x_i's and the w_i's for Gauss–Legendre integration. We won't go into how this is actually done; details can be found in *Numerical Recipes*. Below is a table listing the abscissas and weights for several values of N. These abscissas are based on the interval $[-1, 1]$:

N	x_i	w_i
2	±0.57735	1.000000
3	0	0.888889
	±0.774597	0.555556
4	±0.339981	0.652145
	±0.861136	0.347855
5	0	0.568889
	±0.538469	0.478629

N	x_i	w_i
	± 0.90618	0.236927

If your integration is not over the limits $[-1, 1]$, then the abscissas must be transformed accordingly. For an interval $[a, b]$, the new abscissas t_i are given by:

$$t_i = \frac{a+b}{2} + \frac{b-a}{2} x_i$$

and note that

$$dt = \frac{b-a}{2} dx$$

Hence,

$$\int_a^b f(t)\, dt = \int_{-1}^{1} f\left(\frac{a+b}{2} + \frac{b-a}{2} x\right) \frac{b-a}{2}\, dx$$

Thus,

$$\int_a^b f(t)\, dt \approx \frac{b-a}{2} \sum_{i=1}^{N} w_i\, f\left(\frac{a+b}{2} + \frac{b-a}{2} x_i\right)$$

where the weights and abscissas are as displayed in the table above, and we are assuming that $W(x) = 1$.

> **Exercise 7.4**
>
> Repeat Exercise 7.1 using Gauss–Legendre integration with 5 abscissas. Compare the accuracy with your results from Exercises 7.1 and 7.2.

7.3 Multidimensional Integrals

Numerical integrations of functions of several variables over regions with dimensions greater than one are very difficult to carry out. First, to get sufficient precision, the numerical integration may involve literally tens of thousands of function evaluations. Second, if the boundary of the integration region is complex, this can greatly complicate the procedure. Thus, if possible, before resorting to multidimensional techniques, try to reduce the dimension of your integral.

Sometimes this may be done by exploiting the symmetries of the integrand. For instance, if you are integrating a spherically symmetric function over a spherical region, switch to spherical coordinates. The integral will reduce to a one-dimensional integral! If this does not work, and the function is otherwise fairly well behaved (for instance, is not strongly peaked in a certain region), and you don't require great accuracy, explore *Monte Carlo* techniques (see Section 7.4). Otherwise, a multidimensional integral can be attacked using one-dimensional techniques in the following way:

Let us suppose we have the following integral:

$$I = \int_{x_1}^{x_2} dx \int_{y_1(x)}^{y_2(x)} dy \int_{z_1(x,y)}^{z_2(x,y)} f(x, y, z)\, dz$$

where the limits on the integrals define the region of integration in the usual way. If we let

$$G(x, y) = \int_{z_1(x,y)}^{z_2(x,y)} f(x, y, z)\, dz$$

and

$$H(x) = \int_{y_1(x)}^{y_2(x)} G(x, y)\, dy$$

then

$$I = \int_{x_1}^{x_2} H(x)\, dx$$

The third integral for I can be evaluated using one of the one-dimensional techniques discussed in §7.1. However, in this procedure $H(x)$ will have to be evaluated at a number of values of x, and this must be done by integrating $G(x, y)$ again using a one-dimensional technique for each of those values of x. $G(x, y)$, however, can only be evaluated by carrying out the first integral for each value of x and y encountered in the evaluation of the second and third integrals. Thus, while the third integral is evaluated only once, the second integral is evaluated many times and the third integral many, many times leading to a great number of function evaluations.

Example

Evaluate the integral

$$\int_{-1}^{1} dx \int_{-\sqrt{1-x^2}}^{\sqrt{1-x^2}} (x^2 + y^2)\, dy$$

using two applications of the trapezoidal method. *Note: This integral can be reduced to a single integral in polar coordinates. The exact value of this integral is* $\pi/2 = 1.570796$. Here is some code that will do the job:

```
/*
Functions employed:
trap: carries out 1-D trapezoidal integration
integrand: evaluates the integrand
l1, l2: the limits
twoDtrap: implements 2-D trapezoidal integration
H: Carries out integration of the integrand for a given value of x
x: to make things easy, x is declared as an external variable, so that
we don't have to pass it to each function

*/
#include <stdio.h>
#include <math.h>

double trap(double a, double b, double (*func)(double), int N);
double integrand(double y);
double l1();
double l2();
double twoDtrap(double a, double b,int N);
double H(double y1, double y2);
double x;
int N;

int main()
{
  double a = -1.0;
  double b = 1.0;
  double result;

  printf("\nEnter number of points for both x and y > ");
  scanf("%d",&N);
```

```
  result = twoDtrap(a,b,N);

  printf("\nIntegral = %e\n",result);

  return(0);
}

/* twoDtrap carries out trapezoidal integration on H between the limits
   y1 = l1(x), y2 = l2(x) and x = a and x = b */

double twoDtrap(double a, double b, int N)
{
  double h,integral;
  int i;

  h = (b-a)/(double)(N-1);

  integral = 0.0;
  x = a;
  integral += 0.5*H(l1(),l2());
  x = b;
  integral += 0.5*H(l1(),l2());
  x = a+h;
  for(i=2;i<N;i++) {
    integral += H(l1(),l2());
    x += h;
  }
  integral *= h;
  return(integral);
}

double H(double y1, double y2)
{
  return(trap(y1,y2,integrand,N));
}

/* Remember that x is external, so it need not be passed to l1 and l2 */
double l1()
{
  return(-sqrt(1-x*x));
}
```

```
double l2()
{
  return(sqrt(1-x*x));
}

/* Only y is passed to integrand, as x is external */

double integrand(double y)
{
  return(x*x + y*y);
}

/* Note that trap is written generally, so that any function (func) can
   be passed to it. Study carefully how a function is passed to another
   function, and how the passed function is referred to inside the second
   function. Note that in this case, only one variable can be passed to
   func in this case y.  */

double trap(double a, double b, double (*func)(double), int N)
{
  double h,integral,x;
  int i;

  if(a == b) return(0.0);
  h = (b-a)/(double)(N-1);

  integral = 0.0;
  integral += 0.5*((*func)(a) + (*func)(b));

  x = a+h;
  for(i=2;i<N;i++) {
    integral += (*func)(x);
    x += h;
  }
  integral *= h;
  return(integral);
}
```

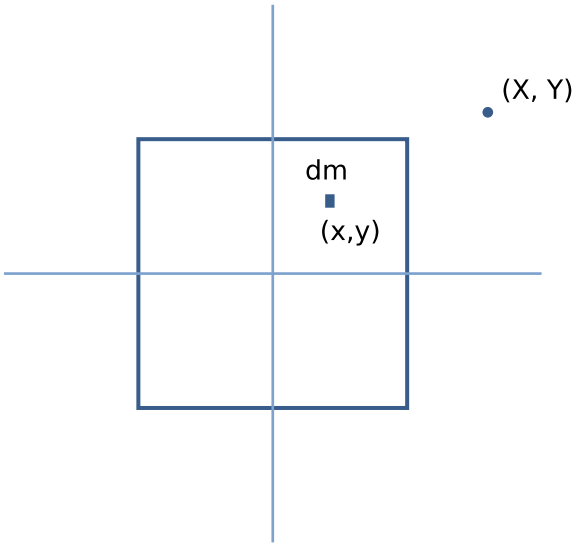
Exercise 7.5

Evaluate the integral

$$\int_{-1}^{1} dx \int_{-\pi/2}^{\pi/2} x^2 \cos y \, dy$$

using two applications of the trapezoidal method. The exact value of this integral is $4/3$.

Exercise 7.6



The gravitational potential at a point (X, Y) due to an infinitesimal mass element dm at position (x, y) is given by

$$dV = -G\frac{dm}{r}$$

where r is the distance between (x, y) and (X, Y). Calculate the gravitational potential V for the square mass shown in the figure above at the point (X, Y), which lies in the plane of the square and outside of it, by integrating the above expression for dV over the mass. Note that the surface density σ of the square is a function of position: $\sigma = (x^2 + y^2)$ kg/m^2 and that $dm = \sigma \, dx \, dy$ The square has dimensions $2\,\text{m} \times 2\,\text{m}$ and is centered on the origin. Prompt the user for the (X, Y) coordinates, and reprompt if (X, Y) lies inside the square.

7.4 Monte Carlo Integration

This technique of numerical integration is named after the famous casino in Monte Carlo not because it is risky or chancy, but because it uses random numbers to carry out an integration. How does this work? Suppose we have an integral of a function f over a given volume V, and we choose N random points in that volume. Then,

$$\iiint_V f\,dV \approx V\langle f\rangle \pm V\sqrt{\frac{\langle f^2\rangle - \langle f\rangle^2}{N}} \tag{3}$$

where $\langle f\rangle$ is the arithmetic mean of f over the N random points:

$$\langle f\rangle = \frac{1}{N}\sum_{i=1}^{N} f(x_i)$$

and

$$\langle f^2\rangle = \frac{1}{N}\sum_{i=1}^{N} f^2(x_i)$$

Note that the error term in equation (3) is proportional to $1/\sqrt{N}$, which is one of the great disadvantages of the Monte Carlo method of numerical integration; the error reduces very slowly with increasing N, and thus many random points are needed for high accuracy. Nonetheless, Monte Carlo integration is sometimes *"the only way to go"*, especially if the region of integration is complicated.

Before we proceed any further with Monte Carlo integration, we should discuss in a bit more detail the generation of random numbers by a computer. All "C" compilers come with a pair of library routines.

```
void srand(unsigned seed);
int rand(void);
```

The `rand` function is the actual random number generator, but the random number generator must be initialized by `srand` by supplying `srand` with an arbitrary unsigned integer, the *seed*. Note that the same initializing value of "seed" will always return the same random number sequence. The function `rand` returns, upon each call, a random number in the range 0 to the largest positive value the computer can produce for variables of type `int`; this number is available in `stdlib.h` as `RAND_MAX`. Thus, if we want a random floating point number in the range 0.0 – 1.0 we can use the code fragment:

```
x = (float)rand()/(RAND_MAX+1.0);
```

Unfortunately, `rand` is not a good enough random number generator, even for government work. It does not give truly random sequences, and this can be fatal for Monte Carlo integrations. It is much better to use the function `float ran1(long *idum)` (see *Numerical Recipes* for details). This function is available in `comphys.c` and defined in `comphys.h` and, as we saw in Chapter 2, should be initialized by setting `idum` to a negative number, thus:

```
long idum;
float rn;
/* Initialize ran1 */
idum = -1;
rn = ran1(&idum);
/* Use ran1 */
rn = ran1(&idum);
```

The function `ran1`, which returns a random floating point number between 0.0 and 1.0 is good enough for government work and is much superior to `rand`. Note, however, that if you use the same negative number seed `idum` each time, you will get the same sequence of random numbers. It is thus better to use the scheme outlined in Chapter 2 to set `idum = -1*now`, where `now` is a time variable.

Now, back to Monte Carlo integration. Suppose we want to integrate a function g over an irregular region W that is difficult to sample randomly. What do we do? We find a region V that contains W and which *is* easy to sample randomly — say, for instance, a rectangle or a cube. We then define $f = g$ at points inside of W but $f = 0$ at points inside of V but outside of W. It is in our own interest to make V fit around W as snugly as possible, as the zero values of f will increase the factor $\langle f^2 \rangle - \langle f \rangle^2$ in the error term of equation (3) and will reduce the effective value of N.

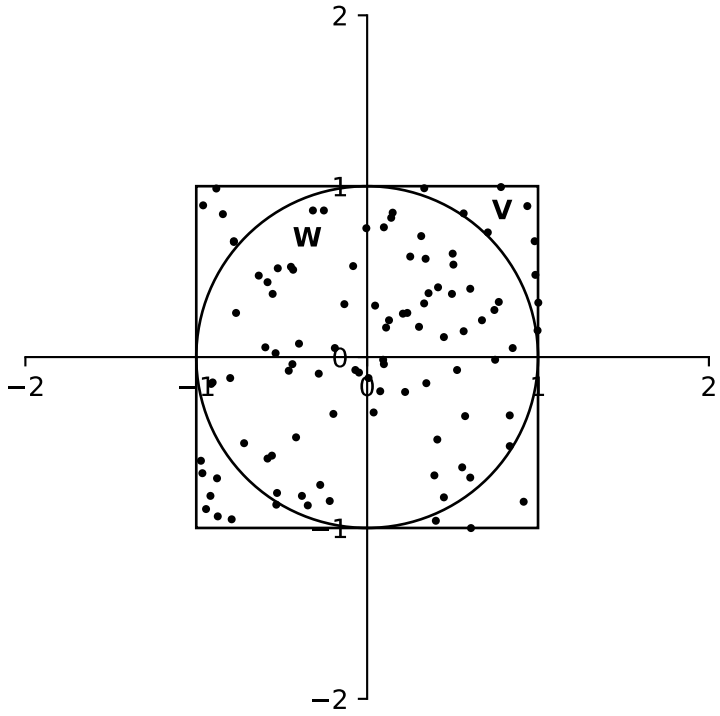
For an example, let us take the integral in the example in §7.3. The region of interest W is a circle with radius 1 and center at $(0, 0)$. We take the region V to be a square with the same center. The Monte Carlo integration is carried out by the following code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "comphys.h"
#include "comphys.c"
#define N 100

int main()
```

```
{
  float x,y,rn,r2,f,integral,V,sum,av,sum2,av2;
  float error;
  long i;
  long idum = -1;
  time_t now;

  /* Use the time function to supply a different seed to the
     random number generator every time the program is run */

  now = time(NULL);

  idum = -1*now;

  /* Initialize random number generator */

  rn = ran1(&idum);

  /* Our region V that encloses W, a circle of unit radius,
     is a square with sides of length 2.  Thus, the area of
     this square is 4 */

  V = 4.0;
  sum = 0.0;
  sum2 = 0.0;
  for(i=0;i<N;i++) {
    /* Following gives x and y between -1.0 and 1.0 */
    x = 2.0*ran1(&idum) - 1.0;
    y = 2.0*ran1(&idum) - 1.0;
    r2 = x*x + y*y;

    /* Is the point in the region W? */
    if(r2 <= 1.0) f = r2;
    else f = 0.0;

    /* Evaluate sums */
    sum += f;
    sum2 += f*f;
  }
  /* Calculate Integral and error */
  av = sum/(float)N;
  av2 = sum2/(float)N;
```

```
  integral = av*V;
  error = V*sqrt((av2-av*av)/(float)N);

  printf("\nIntegral = %f +/- %f\n",integral,error);
  return(0);
}
```

The result of the first 10 runs of this program (your numbers will be different) with $N = 100$ are: 1.825, 1.640, 1.414, 1.700, 1.532, 1.571, 1.413, 1.600, 1.519, 1.745 each with a calculated error of about ± 0.130. The mean of these determinations is 1.596 ± 0.043 (where the error is the standard error of the mean). If we increase N to 1000, we get the following run: 1.593, 1.574, 1.614, 1.589, 1.545, 1.531, 1.592, 1.570, 1.627, 1.579 each with a calculated error of about ± 0.042. The mean of these determinations is 1.581 ± 0.009.



Notice that increasing N by a factor of 10 decreased the error by a factor of only about 3. A single run with $N = 100000$ gives 1.575 ± 0.004. Recall that the exact value of this integral is $\pi/2 = 1.571$.

The moral of the story is — run your Monte Carlo program many times as the individual results are scattered around the true value.

Exercise 7.7

Use Monte Carlo integration to evaluate the integral in Exercise 7.5. The program should prompt the user for N, the number of random points used in the integration.

Exercise 7.8

Using Monte Carlo integration, find the center of mass of a 2-D right triangle with base of length 1 and height of length 2. With the right angle positioned at $(0, 0)$, the density varies as $\rho = y \sin x^2 + 2$. Recall that the center of mass of a planar object is given by

$$X = \frac{1}{M} \iint x\rho \, dS$$

$$Y = \frac{1}{M} \iint y\rho \, dS$$

where

$$M = \iint \rho \, dS$$

Exercise 7.9

Repeat the problem of Exercise 7.6 using Monte Carlo integration.

Exercise 7.10

The Ising Model: Monte Carlo methods may be used in applications other than integration. Consider the following from atomic physics. An Ising chain is a 1-D array made of N particles with spin. The spin (s) of an individual particle can be either up (+1) or down (-1), and the total energy of the chain is given by

$$E = -J \sum_{i=1}^{N-1} s_i s_{i+1}$$

The energy is a minimum when all the spins are the same. Consider an event that randomly flips the spin of one of the particles in the array. Thermodynamically, if that change results in a lower energy, it is highly probable. If the change results in a higher energy, it will have a probability given by the Boltzmann distribution

$$R = \exp(-\Delta E/kT)$$

Write a program that will implement the following algorithm:

1) Generate a trial spin configuration consisting of N particles with random spins s_i.

2) Pick a particle j randomly, flip its spin, and calculate the resulting new energy E_{new}.

3) Accept the new configuration if $E_{\text{new}} \leq E_{\text{old}}$. If $E_{\text{new}} > E_{\text{old}}$, then assign a relative probability R given by the above equation. Generate a random number $0 < r < 1$. If $R > r$ accept the new configuration. If not, restore the old configuration as the accepted configuration and go back to step 2.

The above steps $2 \rightarrow 3$ should be incorporated into a for loop which will run from `l=1` to 5000. Print out `l` and the energy resulting from the configuration decided upon in step 3 for each passage through the loop. Take the following values for the constants: $N = 50$, $J = 2.0$, $k = 0.001$. Use Gnuplot to plot out l versus E. Note that for low temperatures ($T < 1000$), the system rapidly declines in energy and reaches equilibrium within the 5000 iterations. For high temperatures, the energy simply fluctuates and does not reach equilibrium. The Ising model can be used to simulate many quantum physical systems. One example is magnetism. In a ferromagnetic material, individual atoms have a net magnetic moment. Magnetization will occur when those magnetic moments line up, and this represents the minimum energy configuration. In the presence of an external magnetic field, a macroscopic sample of the material can be magnetized, but as the temperature increases, the alignment of the magnetic moments becomes more chaotic. At sufficiently high temperatures, no magnetization will occur.

8 Numerical Integration of Ordinary Differential Equations

8.1 Introduction

Most ordinary differential equations of mathematical physics are second-order equations. Examples include the equation of motion of the simple pendulum

$$\frac{d^2\theta}{dt^2} + \frac{g}{L}\sin\theta = 0,$$

Van der Pol's equation, which arises from the behavior of a three-element vacuum tube

$$\frac{d^2y}{dt^2} - \epsilon(1-y^2)\frac{dy}{dt} + ay = 0,$$

Emden's equation, which can be used to calculate the thermal behavior of a spherical cloud of gas acting under self gravity and the laws of thermodynamics (i.e. a stellar interior)

$$\frac{d^2y}{dx^2} + \frac{2}{x}\frac{dy}{dx} + y^n = 0,$$

and a closely related equation, which describes the interior of a white dwarf star

$$\frac{1}{x^2}\frac{d}{dx}\left(x^2\frac{dy}{dx}\right) + (y^2 - C)^{3/2} = 0$$

Any second-order differential equation can be expressed as a system of two first-order equations. For instance, if we take Van der Pol's equation, we can replace it by the system

$$\frac{dy}{dt} = z, \quad \frac{dz}{dt} = \epsilon(1-y^2)z - ay.$$

> **Exercise 8.1**
>
> Express the differential equations for the simple pendulum, Emden's equation and the white dwarf equation as systems of two first-order differential equations.

While some of the differential equations of mathematical physics can be solved analytically, others can be solved only by using numerical methods. Numerical methods for integrating differential equations are designed to operate on first-order differential equations, for reasons that will become clear later.

A problem involving an ordinary differential equation is not completely specified by the equation alone. The *boundary conditions* must also be specified. For second-order ordinary differential equations there are essentially two different types of boundary conditions:

- Initial value problems, in which values for y and its derivative dy/dx are given at an initial point, $x = x_0$ and
- Two-point boundary value problems in which values for y are given at an initial x_0 and a final x_f point.

These two types of problems require different approaches. We concentrate first on initial-value problems.

8.2 Initial Value Problems

8.2.1 Difference Equations and Euler's Method

The principle underlying the numerical integration of ordinary differential equations is that an approximate solution may be obtained by replacing the infinitessimal differentials dy and dx by finite differences Δy and Δx. Thus, the differential equation

$$\frac{dy}{dx} = f(x, y)$$

may be expressed as a *difference equation* as follows:

$$\Delta y = f(x, y)\Delta x.$$

Using this equation (or a more sophisticated version of it), we may "step" through a solution using step sizes of Δx, beginning with the initial value $y = y_0$ at $x = x_0$ for the first step

$$\begin{aligned} y_1 &= y_0 + f(x_0, y_0)\Delta x \\ x_1 &= x_0 + \Delta x \end{aligned}$$

or, in general, for the $i+1$th step

$$y_{i+1} = y_i + f(x_i, y_i)\Delta x \tag{8.1}$$

$$x_{i+1} = x_i + \Delta x \tag{8.2}$$

It should be realized that Equation 8.1 and Equation 8.2, known as the *simple Euler Method*, can be improved upon, as $f(x_i, y_i)$ is the derivative at the *beginning* of the interval from x_i to x_{i+1} whereas we should use an *average* value of $f(x, y)$ over this interval. Indeed, most of the advances in the development of numerical integration techniques can be summarized in terms of the discovery of better and more efficient expressions for the determination of the average value of the derivative, $f(x, y)$, in the interval (x_i, x_{i+1}).

The simple Euler method may be immediately improved by replacing $f(x_i, y_i)$ in Equation 8.1 with $\frac{1}{2}[f(x_i, y_i) + f(x_{i+1}, y_{i+1})]$ to yield

$$y_{i+1} = y_i + \frac{1}{2}[f(x_i, y_i) + f(x_{i+1}, y_{i+1})]\Delta x, \tag{8.3}$$

but this is an *implicit* method rather than an *explicit* method, as y_{i+1} must be known before $f(x_{i+1}, y_{i+1})$ can be determined. This implicit equation may be made explicit by using the simple Euler method to first *predict* a value for y_{i+1}, call it y^1_{i+1}, and then Equation 8.3 to *correct* the value for y_{i+1}, thus:

$$\begin{aligned} y^1_{i+1} &= y_i + f(x_i, y_i)\Delta x \\ y_{i+1} &= y_i + \frac{1}{2}[f(x_i, y_i) + f(x_{i+1}, y^1_{i+1})]\Delta x \end{aligned}$$

This method, a simple example of a *predictor-corrector* technique (see below), is called the *Improved Euler Method*.

Example: Use the improved Euler Method to integrate the differential equation

$$\frac{dy}{dx} = y \sin x$$

from $x = 0$ to $x = 1$ using a step size of 0.1 and the initial conditions $y = e = 2.718281828...$ when $x = 0$. The following code will do the job:

```c
#include <stdio.h>
#include <math.h>

int main()
{
  double x1,y1,yp1,x2,y2;
  double dx = 0.1;

  /* initial conditions */
  x1 = 0.0;
  y1 = exp(1.0);
  printf("x = %f y = %f\n",x1,y1);

  /* Integrate using Improved Euler from x=0 to x=1 */
  while(x1 < 1.0) {
    yp1 = y1 + y1*sin(x1)*dx;
    y2 = y1 + 0.5*(y1*sin(x1) + yp1*sin(x1+dx))*dx;
    x2 = x1 + dx;
    if(x2 > 1.0) break;
    printf("x = %f y = %f\n",x2,y2);
    x1 = x2;
    y1 = y2;
  }
  return(0);
}
```

The result is that when $x = 1$, $y = 4.301422$, a fairly good approximation of the "exact" result, $y = 4.304658$.

Exercise 8.2

Integrate the differential equation

$$\frac{dy}{dx} = (x-1)y^2$$

from $x = 0$ to $x = 1$ using the initial condition $y = 1$ when $x = 0$. Use a step size of 0.1. Attempt to integrate the equation analytically (hint: use separation of variables) and compare your answer with the exact result. Try a step size of 0.01 to see if you get a better result.

The simple Euler method has an error per step that is proportional to Δx^2, e.g. $O(\Delta x^2)$. The improved Euler method improves this to $O(\Delta x^3)$. Let us now examine a method which has a much better error, $O(\Delta x^5)$. This method is the *fourth-order Runge Kutta method*.

8.2.2 The Runge Kutta Method

In our discussion in the previous section, we stated that the history of the development of techniques for the numerical integration of differential equations can be viewed as a quest for better and more efficient methods for the determination of the average value of the derivative $f(x, y)$ in the interval (x_i, x_{i+1}). Let us develop this idea further. Consider the set of equations:

$$\begin{aligned} k_1 &= f(x_n, y_n)\Delta x \\ k_2 &= f(x_n + \frac{1}{2}\Delta x, y_n + \frac{1}{2}k_1)\Delta x \\ y_{n+1} &= y_n + k_2 \end{aligned}$$

The first equation assigns the derivative of y at the beginning of the interval to $k_1/\Delta x$. The second equation then uses k_1 to predict the value of y at the middle of the interval to be $y_n + \frac{1}{2}k_1$, and uses this value to evaluate the derivative at that point and assigns it to $k_2/\Delta x$. The third equation uses this "improved" value of the derivative to determine y_{n+1}. This method of numerical integration is called the *midpoint method*, and it is also referred to as the *second-order Runge-Kutta method*. The error in this method is $O(\Delta x^3)$.

A substantial improvement in this scheme can be obtained by using k_2 to predict a better value of y at the center of the interval and then using this to determine a better value of the derivative at the center of the interval, $k_3/\Delta x$. The value of k_3 can then be used to determine a value for the derivative at the end of the interval, $k_4/\Delta x$, and then all four derivatives (one at the beginning of the interval, two estimates at the center of the interval and one at the end) can be used in a weighted average to give an estimate for an overall average value of the derivative for the entire interval. This can be accomplished with the following series of steps:

$$\begin{aligned} k_1 &= f(x_n, y_n)\Delta x \\ k_2 &= f(x_n + \frac{1}{2}\Delta x, y_n + \frac{1}{2}k_1)\Delta x \\ k_3 &= f(x_n + \frac{1}{2}\Delta x, y_n + \frac{1}{2}k_2)\Delta x \\ k_4 &= f(x_n + \Delta x, y_n + k_3)\Delta x \\ y_{n+1} &= y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{aligned}$$

This method, because its error is $O(\Delta x^5)$, is known as the *fourth-order Runge Kutta method*. The "magic" of Runge Kutta derives from the fact that the particular weighting used for the derivatives (1,2,2,1) *exactly* cancels errors of $O(\Delta x^4)$, thus making the error for the entire method $O(\Delta x^5)$.

Example: Let us integrate the differential equation of the example of the previous section, but this time using the fourth-order Runge Kutta method. The following code performs the trick:

```
#include <stdio.h>
#include <math.h>

int main()
{
  double x1,y1,x2,y2;
  double k1,k2,k3,k4;
  double dx = 0.1;

  /* initial conditions */
  x1 = 0.0;
  y1 = exp(1.0);

  /* Integrate using fourth-order Runge Kutta from x=0 to x=1 */
  while(x1 < 1.0) {
    printf("x = %3.1f y = %f\n",x1,y1);
    if(x1 == 1.0) break;
    k1 = y1*sin(x1)*dx;
    k2 = (y1+0.5*k1)*sin(x1+0.5*dx)*dx;
    k3 = (y1+0.5*k2)*sin(x1+0.5*dx)*dx;
    k4 = (y1+k3)*sin(x1+dx)*dx;
    y2 = y1 + (k1 + 2.0*k2 + 2.0*k3 + k4)/6.0;
    x2 = x1 + dx;
    x1 = x2;
    y1 = y2;
  }
  return(0);
}
```

This code gives the result $y(1) = 4.304658$, considerably better than before, but without very much more effort!

> **Exercise 8.3**
>
> Integrate the differential equation of Exercise 8.2 using the Runge Kutta method. Do you get a better result?

It turns out that it is possible to come up with higher order Runge Kutta schemes. However, in practice, almost everyone uses the fourth-order Runge Kutta method. The reason for this is

that, coupled with an *adaptive stepsize routine*, the fourth-order Runge Kutta method will give almost any accuracy desired.

8.2.3 A Brief Aside: Adaptive Stepsize Control for the Runge Kutta Method

One of the great advantages of the Runge Kutta technique (as opposed to predictor-corrector techniques) is that it is possible to vary the step size from one step to the next. This is an advantage because one can then modify the stepsize in response to the error. In regions where the function is changing rapidly, we need to use small step sizes to keep good accuracy. In other regions where the function is hardly changing, we can take big strides. Such an "adaptive stepsize" routine can greatly increase the efficiency of the Runge Kutta technique. This technique is usually implemented by *step doubling*. In step doubling we take each integration step twice, once as a full step of length $2\Delta x$, then as two steps, each of length Δx. Let us denote the exact solution for an advance from x to $x+2\Delta x$ by $y(x+2\Delta x)$. With step doubling, we get two numerical approximations to $y(x+2\Delta x)$, $y_{(1)}$ and $y_{(2)}$:

$$y(x+2\Delta x) \approx y_{(1)} + (2\Delta x)^5\phi$$
$$y(x+2\Delta x) \approx y_{(2)} + 2(\Delta x)^5\phi$$

where the last term in both equations involving ϕ (which is assumed to be the same in both equations) gives the error for that step. For the top equation, we are taking a step size of $2\Delta x$, in the bottom, two steps of Δx. The difference $\epsilon = |y_{(2)} - y_{(1)}|$ we can take as a good estimate of the error for a step of size Δx. Clearly, $\epsilon \propto (\Delta x)^5$. Hence, if the desired error per step is ϵ_0, we may, after taking our trial steps (as in the above two equations), adjust the step size Δx either upwards or downwards according to

$$\Delta x_{\text{new}} = \Delta x_{\text{old}} \left|\frac{\epsilon_0}{\epsilon}\right|^{1/5}$$

and then use this new step size to redo the integration step starting at x.

> **Exercise 8.4**
>
> Integrate the differential equation of Exercises 8.2 & 8.3 using adaptive stepsize control with the Runge Kutta method. Begin with $\Delta x = 0.1$ and $\epsilon_0 = 1.0 \times 10^{-6}$. Print out the intermediate results so that you can see what your algorithm is doing. Remember that the integration must stop at $x = 1.0$. Experiment with different values of ϵ_0 to see how close you can get to the "exact" solution.

8.2.4 The Integration of Systems of Differential Equations

Most differential equations of mathematical physics are second-order differential equations. As we saw in the introduction, a second-order ordinary differential equation can always be written as a system of two first-order differential equations. How can we integrate numerically a system of two equations? We need to set up two parallel, interleaved Runge Kutta routines. The best way to illustrate this is to look at an example.

Example: Suppose we have the second-order differential equation (a form of Van der Pol's equation)

$$\frac{d^2y}{dx^2} = (1-y^2)\frac{dy}{dx} - y$$

which we wish to integrate subject to the initial conditions $y(0) = 2$, $y'(0) = 0$. This can be written as the following system of two first-order differential equations:

$$\begin{aligned}\frac{dy}{dx} &= z \\ \frac{dz}{dx} &= (1-y^2)z - y\end{aligned}$$

We can set up the following Runge Kutta scheme to integrate this equation:

$$\begin{aligned}k_1 &= z_n\Delta x \\ l_1 &= [(1-y_n^2)z_n - y_n]\Delta x \\ k_2 &= (z_n + \frac{1}{2}l_1)\Delta x \\ l_2 &= [(1-(y_n + \frac{1}{2}k_1)^2)(z_n + \frac{1}{2}l_1) - (y_n + \frac{1}{2}k_1)]\Delta x \\ k_3 &= (z_n + \frac{1}{2}l_2)\Delta x \\ l_3 &= [(1-(y_n + \frac{1}{2}k_2)^2)(z_n + \frac{1}{2}l_2) - (y_n + \frac{1}{2}k_2)]\Delta x \\ k_4 &= (z_n + l_3)\Delta x \\ l_4 &= [(1-(y_n + k_3)^2)(z_n + l_3) - (y_n + k_3)]\Delta x \\ y_{n+1} &= y_n + (k_1 + 2k_2 + 2k_3 + k_4)/6 \\ z_{n+1} &= z_n + (l_1 + 2l_2 + 2l_3 + l_4)/6\end{aligned}$$

Note that the k's are the derivatives of y, whereas the l's are the derivatives of z. Note as well that we cannot calculate all the k's first and then the l's, because the k's depend on the l's and vice-versa (notice that k_2, for instance, depends on l_1, and l_2 depends on k_1 as well as l_1).

This integration scheme can be implemented in the following code:

```
#include <stdio.h>

int main()
{
  double x1,y1,z1,x2,y2,z2;
  double k1,k2,k3,k4,l1,l2,l3,l4;
  double dx = 0.1;

  /* initial conditions */
  x1 = 0.0;
  y1 = 2.0;
  z1 = 0.0;

  /* Integrate using fourth-order Runge Kutta from x=0 to x=1 */
  while(x1 < 1.0) {
    printf("x = %3.1f y = %f z = %f\n",x1,y1,z1);
    if(x1 == 1.0) break;
    k1 = z1*dx;
    l1 = ((1-y1*y1)*z1-y1)*dx;
    k2 = (z1+0.5*l1)*dx;
    l2 = ((1-(y1+0.5*k1)*(y1+0.5*k1))*(z1+0.5*l1)-(y1+0.5*k1))*dx;
    k3 = (z1+0.5*l2)*dx;
    l3 = ((1-(y1+0.5*k2)*(y1+0.5*k2))*(z1+0.5*l2)-(y1+0.5*k2))*dx;
    k4 = (z1+l3)*dx;
    l4 = ((1-(y1+k3)*(y1+k3))*(z1+l3)-(y1+k3))*dx;
    y2 = y1 + (k1 + 2.0*k2 + 2.0*k3 + k4)/6.0;
    z2 = z1 + (l1 + 2.0*l2 + 2.0*l3 + l4)/6.0;
    x2 = x1 + dx;
    x1 = x2;
    y1 = y2;
    z1 = z2;
  }
  return(0);
}
```

The output from this program is

```
x = 0.0 y = 2.000000 z = 0.000000
x = 0.1 y = 1.990931 z = -0.172638
x = 0.2 y = 1.966948 z = -0.300697
```

```
x = 0.3 y = 1.931828 z = -0.397370
x = 0.4 y = 1.888175 z = -0.472827
x = 0.5 y = 1.837717 z = -0.534503
x = 0.6 y = 1.781553 z = -0.587734
x = 0.7 y = 1.720322 z = -0.636360
x = 0.8 y = 1.654338 z = -0.683230
x = 0.9 y = 1.583660 z = -0.730560
x = 1.0 y = 1.508149 z = -0.780208
```

Exercise 8.5

Integrate the equation for the simple pendulum

$$\frac{d^2y}{dt^2} + \sin y = 0$$

with the initial conditions $y(0) = \pi/4$ and $y'(0) = 0$ and carry the integration out over two complete oscillations using a step size of $\Delta t = 0.1$.

8.2.5 Predictor-Corrector Methods

The Runge Kutta method is like a 4-wheel drive vehicle – it will succeed in integrating a differential equation even under the most difficult conditions, but it may not be the speediest technique on the block. The real speedsters are the *Predictor-Corrector* methods. These methods use determinations of the derivative, $f(x, y)$, at previous steps to first *predict* (by polynomial extrapolation) and then *correct* (by polynomial interpolation) the derivative at the current step. There are many predictor-corrector schemes, but here is a commonly used one, called the *Adams-Bashforth-Moulton* scheme:

$$\text{predictor}: \quad y_{n+1} = y_n + \frac{\Delta x}{12}(23y'_n - 16y'_{n-1} + 5y'_{n-2}) + O(\Delta x^4)$$
$$\text{corrector}: \quad y_{n+1} = y_n + \frac{\Delta x}{12}(5y'_{n+1} + 8y'_n - y'_{n-1}) + O(\Delta x^4)$$

The problem with predictor-corrector methods is that they are *multi-step* techniques and thus to start the integration, we need more than just the initial conditions. Note that for the predictor step, we need to know the derivative at the $(n-2)$nd, $(n-1)$st and the nth steps. How do we do this? One way to provide these values is to set up a Runge-Kutta routine to prime the pump, so to speak. But this seems like overkill. Another way to proceed is to determine these values from the differential equation itself by deriving a power series expansion for $y(x)$.

Let us suppose that we have the differential equation

$$\frac{dy}{dx} = f(x, y)$$

subject to the initial condition $y(0) = y_0$. Then,

$$\left.\frac{dy}{dx}\right|_{x=0} = f(0, y_0)$$

and

$$\left.\frac{d^2y}{dx^2}\right|_{x=0} = \frac{df(0, y_0)}{dx} + \frac{df(0, y_0)}{dy} \left.\frac{dy}{dx}\right|_{x=0}$$

and so on. Then, we may write:

$$y(x) = y(0) + \left.\frac{dy}{dx}\right|_{x=0} x + \frac{1}{2} \left.\frac{d^2y}{dx^2}\right|_{x=0} x^2 + ...$$

which is the equation for the Maclaurin series expansion. Obviously, if the initial condition is given at some other point than $x = 0$, the Taylor series expansion could be used.

To make all this clear, let us consider the following example:

Example: Integrate the differential equation

$$\frac{dy}{dx} = y \sin x$$

using the Adams-Bashforth-Moulton predictor-corrector scheme and a step size of 0.1. The initial condition is $y = e$ when $x = 0$. Prime the pump by expanding y in a Maclaurin series.

Now,

$$\left.\frac{dy}{dx}\right|_{x=0} = e \sin 0 = 0,$$

$$\frac{d^2y}{dx^2} = y \cos x + \sin x \frac{dy}{dx}$$

yielding

$$\left.\frac{d^2y}{dx^2}\right|_{x=0} = e$$

and

$$\frac{d^3y}{dx^3} = -y\sin x + 2\cos x\frac{dy}{dx} + \sin x\frac{d^2y}{dx^2}$$

giving

$$\left.\frac{d^3y}{dx^3}\right|_{x=0} = 0$$

and, finally (because we are too lazy to derive any more than one more term),

$$\frac{d^4y}{dx^4} = -y\cos x - 3\sin x\frac{dy}{dx} + 3\cos x\frac{d^2y}{dx^2} + \sin x\frac{d^3y}{dx^3}$$

giving

$$\left.\frac{d^4y}{dx^4}\right|_{x=0} = 2e$$

which gives the Maclaurin expansion:

$$y(x) = e + \frac{1}{2}ex^2 + \frac{1}{12}ex^4 + ...$$

and differentiating,

$$y'(x) = ex + \frac{1}{3}ex^3 + ...$$

This gives $y(0) = 2.718282$, $y(0.1) = 2.731896$, $y(0.2) = 2.773010$ and $y'(0) = 0.000000$, $y'(0.1) = 0.272734$, $y'(0.2) = 0.550905$. These values can be used to prime the pump if we identify $y'(0)$ with y'_{n-2}, $y'(0.1)$ with y'_{n-1}, and $y'(0.2)$ with y'_n in the Adams-Bashforth-Moulton routine. The following code carries out the numerical integration:

```
#include <stdio.h>
#include <math.h>
double yfunc(double x);
double yderiv(double x);

/* Implements the Adams-Bashforth-Moulton routine */
int main()
```

```
{
  double xm2,xm1,x0,x1,ym2,ym1,y0,y1;
  double ypm2,ypm1,yp0,yp1;
  double dx = 0.1;

  /* Prime the Pump! */
  ym2 = yfunc(0.0);
  ypm2 = yderiv(0.0);
  ym1 = yfunc(0.1);
  ypm1 = yderiv(0.1);
  y0 = yfunc(0.2);
  yp0 = yderiv(0.2);

  xm2 = 0.0;
  xm1 = 0.1;
  x0 = 0.2;
  x1 = 0.3;

  printf("x = %3.1f y = %f\n",xm2,ym2);
  printf("x = %3.1f y = %f\n",xm1,ym1);
  printf("x = %3.1f y = %f\n",x0,y0);

  while(x1 < 1.0) {
    /* Predictor step */
    y1 = y0 + (23.0*yp0 - 16.0*ypm1 + 5.0*ypm2)*dx/12.0;

    /* Calculate yp1 */
    yp1 = y1*sin(x1);

    /* Corrector step */
    y1 = y0 + (5.0*yp1 + 8*yp0 - ypm1)*dx/12.0;

    /* Recalculate yp1 - this step is optional! */
    yp1 = y1*sin(x1);

    printf("x = %3.1f y = %f\n",x1,y1);
    if(x1 == 1.0) break;

    /* Update variables */
    xm2 += dx;
    xm1 += dx;
    x0 += dx;
```

```
    x1 += dx;
    ym2 = ym1;
    ym1 = y0;
    y0 = y1;
    ypm2 = ypm1;
    ypm1 = yp0;
    yp0 = yp1;
  }
  return(0);
}

double yfunc(double x)
{
  static double e = 2.7182818285;
  return(e + 0.5*e*x*x + e*pow(x,4.0)/12.0);
}

double yderiv(double x)
{
  static double e = 2.7182818285;
  return(e*x + e*pow(x,3.0)/3.0);
}
```

If this all looks like too much trouble, you are probably right. I almost always stick with the Runge Kutta routine for numerical integration of differential equations. However, note that this method uses only two function evaluations per step, whereas the Runge Kutta technique uses four. As a matter of fact, one can toss out the second function evaluation in the Adams-Bashforth-Moulton routine with only a small sacrifice in accuracy. This means that the Adams-Bashforth-Moulton routine can be four times faster than the Runge Kutta routine. This can be very important if the function evaluation is computationally expensive.

Exercise 8.6

Integrate the differential equation of Exercise 8.2 using the Adams-Bashforth-Moulton technique.

Exercise 8.7

Integrate the second-order differential equation (Emden's equation)

$$\frac{d^2y}{dx^2} + \frac{2}{x}\frac{dy}{dx} + y^n = 0,$$

with $n = 3$ and initial conditions $y = 1$, $y' = 0$ when $x = 0$ using the Adams-Bashforth-Moulton technique. The solution will give you the interior structure of a star that is completely convective throughout (red dwarf). Note that this is a second-order differential equation, so two parallel interleaved Adams-Bashforth-Moulton routines will have to be used. Prime the pump by using the following power series expansion for $y(x)$ (difficult to derive!):

$$y = 1 - \frac{x^2}{3!} + n\frac{x^4}{5!} + (5n - 8n^2)\frac{x^6}{3 \cdot 7!} + ...$$

Show that $y(x) = 0$ at $x = 6.89685$. Stop the integration when $y(x)$ crosses the x-axis.

8.3 Two-Point Boundary-Value Problems

Ordinary differential equations that are required to satisfy boundary conditions at two points instead of one are called *two-point boundary-value problems*. These are more difficult to solve than *initial-value problems* because one usually must make a number of trial integrations before one is found that satisfies both boundary conditions.

This comes about for the following reason. For a second-order differential equation, two boundary conditions are required to fully specify the solution. In initial-value problems, both of these conditions are specified at the start of the integration, and so the solution is fully constrained at the outset. In two-point boundary-value problems generally only one boundary condition is specified at the initial point; the second is applied (usually) to the end point of the integration. This means that the solution is not fully constrained at the beginning of the integration and there are literally an infinite number of solutions of the differential equation that satisfy the single boundary condition at the initial point. To solve the problem, one must find the (single) solution that satisfies both boundary conditions out of this infinite set.

This might sound almost impossible to accomplish, but it turns out that there are a number of iterative algorithms one can employ to find the correct solution. The simplest to understand, and the only one we will consider in these notes is called the "shooting method". Another more sophisticated method is the "relaxation method".

Perhaps the best way to explain the shooting method is by way of an example. Let us integrate the equation of Van der Pol

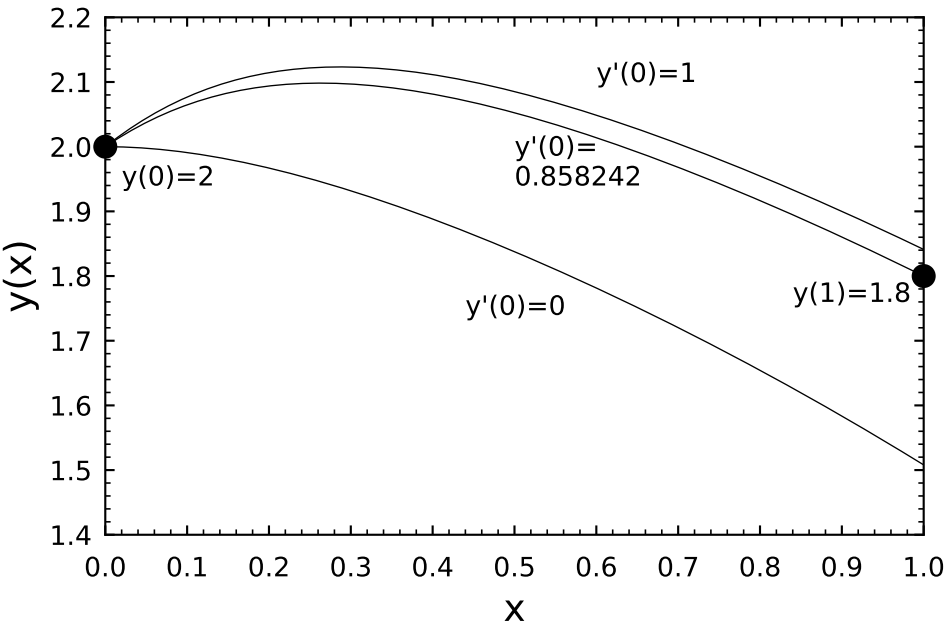
$$\frac{d^2y}{dx^2} = (1 - y^2)\frac{dy}{dx} - y$$

subject to the boundary conditions $y(0) = 2$, $y(1) = 1.8$. In our earlier example, we specified both y and y' at the initial point $x = 0$. Now we are specifying only y at the initial point. We must therefore find $y'(0)$ such that the solution goes through $y(1) = 1.8$. How do we proceed?

Let us begin with two trial solutions (or "shots"), one with $y'(0) = 0.0$ and the other with $y'(0) = 1.0$, and integrate both from $x = 0$ to $x = 1$. Neither of these will likely pass through $y(1) = 1.8$, but we can use linear interpolation (or extrapolation) to find, from these two trial solutions, a better starting value for $y'(0)$ (to improve our aim, so to speak). A new integration with this starting value will enable us to find an even better starting value, and so on, until we have found a solution which satisfies the boundary conditions to within the desired precision. The following code accomplishes this task for the equation of Van der Pol:

```
#include <stdio.h>
#include <math.h>
int flag = 0;

double integrate(double x1,double y1,double z1,double y11);

int main()
{
  double x1,y1,z1,z2,z3,y11,r1,r2,r3;
  double k1,k2,k3,k4,l1,l2,l3,l4,r;
  double dx = 0.1;

  /* initial conditions: Note the z's are trial y'(0) 's */
  x1 = 0.0;
  y1 = 2.0;
  z1 = 0.0;
  z2 = 1.0;

  /* Enter boundary condition at the end point and put into y11 */
  printf("\nEnter y(1) > ");
  scanf("%lf",&y11);

  /* Iterative loop - stop condition is when last two iterations
     are equal within machine precision */
  while(fabs(z1-z2) > 1.0e-06) {
    r1 = integrate(x1,y1,z1,y11);
    r2 = integrate(x1,y1,z2,y11);

    /* The interpolation (extrapolation) equation to find
       a better z = z3 */
    z3 = z1 - r1*(z2-z1)/(r2-r1);

    /* Replace the worst trial y'(0) with the
       best found so far */
    if(fabs(z3-z1) > fabs(z3-z2)) z1 = z3;
```

```
    else z2 = z3;
  }

  printf("y'(0) = %f\n",z1);
  flag = 1;
  printf("%3.1f %f %f\n",x1,y1,z1);
  r = integrate(x1,y1,z1,y11);
  return 0;
}

double integrate(double x1,double y1,double z1,double y11)
{
  /* Integrate Van der Pol equation using fourth-order Runge
     Kutta from x=0 to x=1.  The return value is y(1) - y11
     where y11 is the second boundary condition.  The correct
     solution will have y(1) - y11 = 0 */

  double dx = 0.1;
  double x2,y2,z2;
  double k1,k2,k3,k4,l1,l2,l3,l4;
  extern int flag;

  while(x1 < 1.0 - dx/2.0) {
    k1 = z1*dx;
    l1 = ((1-y1*y1)*z1-y1)*dx;
    k2 = (z1+0.5*l1)*dx;
    l2 = ((1-(y1+0.5*k1)*(y1+0.5*k1))*(z1+0.5*l1)-(y1+0.5*k1))*dx;
    k3 = (z1+0.5*l2)*dx;
    l3 = ((1-(y1+0.5*k2)*(y1+0.5*k2))*(z1+0.5*l2)-(y1+0.5*k2))*dx;
    k4 = (z1+l3)*dx;
    l4 = ((1-(y1+k3)*(y1+k3))*(z1+l3)-(y1+k3))*dx;
    y2 = y1 + (k1 + 2.0*k2 + 2.0*k3 + k4)/6.0;
    z2 = z1 + (l1 + 2.0*l2 + 2.0*l3 + l4)/6.0;
    x2 = x1 + dx;
    x1 = x2;
    y1 = y2;
    z1 = z2;
    if(flag == 1) printf("%4.2f %f %f\n",x1,y1,z1);
  }
  return(y1-y11);
}
```



Exercise 8.8

Find the solution of the differential equation

$$\frac{d^2y}{dx^2} = 2y^3 + xy$$

which satisfies $y(0) = 1.0$ and $y(1) = 2.0$.

9 The Modeling of Data

9.1 Introduction

Let us suppose that we have a set of experimental data in the form of data pairs $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$. When plotted, these data may appear to have a linear relationship, or a higher-order functional relationship may be evident. A reasonable next step is to try to "fit" an appropriate function to these data, as part of the analysis process. The purpose may simply be to condense and summarize the observations using a mathematical function, or it may be that a theory for the phenomenon represented by the data exists, and we are attempting to determine empirically certain parameters in this theory through the fitting process. Whichever is the case, we need to devise some criterion for selecting a *best-fit* function.

Traditionally, the procedure used to find this best-fit function is the process of least squares. Let us suppose that our model function is of the form:

$$y = f(x; a_1, a_2, \ldots, a_m) \tag{9.1}$$

where the a_i's are the fitting parameters. Then, the least squares technique finds the combination of the a_i's such that the sum

$$\sum_{i=1}^{N} [y_i - f(x_i; a_1, a_2, \ldots, a_m)]^2$$

is minimized. Why do we choose this expression to minimize? Clearly, one does not want negative deviations to cancel with positive deviations, but why do we take the sum of the *square* of the deviations and not, for instance, the *absolute value* of the deviations? This is an important question. The answer is that it turns out that if the deviations (i.e. the errors) in the y_i's are normally distributed (i.e. have a Gaussian distribution), then minimizing the sum of the square of the deviations leads to the choice for the a_i's that is *most likely* to be correct. This result is derived from the theory of *Maximum Likelihood Estimators*. More information can be found in the book *Numerical Recipes*.

What if our errors are not normally distributed? This is a fairly common occurrence in experimental physics. It turns out that if the distribution of errors is only approximately

Gaussian, least squares is still a good way to go. However, the least squares technique is very sensitive to data points that are very deviant (*outliers*). These very deviant points can exert enormous leverage over the sum of the squares of the deviations to the extent that they may even dominate the sum, leading to a fitted function which clearly does not represent the majority of the data points. In such a case, we should either eliminate the outliers using a valid (i.e. honest) statistical process, or use another fitting strategy. One such fitting strategy is called *robust fitting* in which it is the sum of the absolute value of the deviations which is minimized.

It should be noted that in the above discussion we have tacitly assumed that all of the errors are in the y_i's and not in the x_i's. This situation is often the case in experimental physics, but sometimes it turns out that both the x_i's and the y_i's have significant errors. In this case, we need to use a slightly different approach (see *Numerical Recipes* for more information).

Let us begin our discussion of the modeling of data with a review of using least squares to fit data to a straight line, and use that as a springboard to a discussion on fitting data to nonlinear functions.

9.2 Least Squares Fitting of Data to a Straight Line

Let us suppose that the y-value for each data point (x_i, y_i) has its own, known, standard deviation, σ_i. Then the expression that we wish to minimize becomes:

$$\chi^2 = \sum_{i=1}^{N} \left(\frac{y_i - f(x_i; a_1, a_2, \ldots, a_m)}{\sigma_i} \right)^2$$

Dividing each term in the sum by σ_i^2 *weights* each data point according to its error—those data points with the smallest errors contribute the most to the sum[1]. If our model function is a straight line,

$$f(x) = f(x; a, b) = a + bx$$

then the above expression becomes

$$\chi^2(a, b) = \sum_{i=1}^{N} \left(\frac{y_i - a - bx_i}{\sigma_i} \right)^2 \tag{9.2}$$

This sum must be minimized with respect to the two parameters a and b, which means that we must have

[1] If individual errors σ_i are not given or specified, one can set $\sigma_i = 1$ in this and all equations in this chapter.

$$0 = \frac{\partial \chi^2}{\partial a} = -2 \sum_{i=1}^{N} \frac{y_i - a - bx_i}{\sigma_i^2}$$

$$0 = \frac{\partial \chi^2}{\partial b} = -2 \sum_{i=1}^{N} \frac{x_i(y_i - a - bx_i)}{\sigma_i^2}$$

if we make the following substitutions

$$S \equiv \sum_{i=1}^{N} \frac{1}{\sigma_i^2} \qquad S_x \equiv \sum_{i=1}^{N} \frac{x_i}{\sigma_i^2} \qquad S_y \equiv \sum_{i=1}^{N} \frac{y_i}{\sigma_i^2}$$

$$S_{xx} = \sum_{i=1}^{N} \frac{x_i^2}{\sigma_i^2} \qquad S_{xy} = \sum_{i=1}^{N} \frac{x_i y_i}{\sigma_i^2}$$

we derive the following equations

$$aS + bS_x = S_y$$

$$aS_x + bS_{xx} = S_{xy}$$

which may be written as a matrix equation

$$\begin{pmatrix} S & S_x \\ S_x & S_{xx} \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} S_y \\ S_{xy} \end{pmatrix}$$

To solve this equation, we need to find the inverse of the matrix

$$\begin{pmatrix} S & S_x \\ S_x & S_{xx} \end{pmatrix}^{-1} = \begin{pmatrix} S_{xx}/\Delta & -S_x/\Delta \\ -S_x/\Delta & S/\Delta \end{pmatrix} \tag{9.3}$$

(where $\Delta = SS_{xx} - (S_x)^2$ is the determinant of the matrix) and multiply through by this inverse matrix. This leads to the well-known linear-least-squares equations:

$$a = \frac{S_{xx}S_y - S_x S_{xy}}{\Delta} \tag{9.4}$$

$$b = \frac{SS_{xy} - S_x S_y}{\Delta} \tag{9.5}$$

Before we can say that we are done, we must estimate the errors in the fitted parameters a and b. The theory of the propagation of errors tells us that the variance σ_f^2 in the value of any function f is given by

$$\sigma_f^2 = \sum_{i=1}^{N} \sigma_i^2 \left(\frac{\partial f}{\partial y_i} \right) \tag{9.6}$$

We may derive the following equations directly from the solution (Equation 9.4 and Equation 9.5)

$$\frac{\partial a}{\partial y_i} = \frac{S_{xx} - S_x x_i}{\sigma_i^2 \Delta}$$

$$\frac{\partial b}{\partial y_i} = \frac{S x_i - S_x}{\sigma_i^2 \Delta}$$

Summing over the points as in Equation 9.6, we derive for the variances:

$$\sigma_a^2 = S_{xx}/\Delta$$

$$\sigma_b^2 = S/\Delta$$

Note that the right-hand sides of these two expressions are the diagonal entries in the inverse matrix (Equation 9.3). To derive the standard errors for a and b, we must multiply the variances above by the factor

$$s^2 = \chi^2/(N-2)$$

where χ^2 is calculated from Equation 9.2 using the calculated values of a and b and the factor $N-2$ comes from the number of *degrees of freedom*; N is the total number of points used to determine the 2 parameters a and b. Thus, the standard errors for a and b will be given by:

$$\text{S.E.}(a) = s\sigma_a$$

$$\text{S.E.}(b) = s\sigma_b$$

Exercise 9.1

Write a program implementing the above equations to fit a straight line to the data in the datafile `data91.dat`. Your program should estimate the parameters a and b and their standard errors.

Exercise 9.2

Write a function which will multiply an $N \times M$ matrix with a vector of length M to give a vector of length N.

Exercise 9.3

Let us suppose we are confronted with a matrix equation of the form

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \tag{9.7}$$

where $\mathbf{A}$ is an $N \times N$ matrix, and $\mathbf{b}$ is an input vector of dimension $N \times 1$. The $\mathbf{x}$ vector is the solution to this matrix equation. To find $\mathbf{x}$, we must find $\mathbf{A}^{-1}$, and then we have

$$\mathbf{x} = \mathbf{A}^{-1} \cdot \mathbf{b}$$

Now, imagine that both $\mathbf{b}$ and $\mathbf{x}$ are $N \times M$ matrices, then we may regard Equation 9.7 as a set of M simultaneous matrix equations. There is a C-function, found in `comphys.c`, which will solve this equation for you. It is declared as

```
void gaussj(float **a, int n, float **b, int m)
```

and comes from the *Numerical Recipes* book. Its description is as follows:
"Linear equation solution by Gauss-Jordan elimination. `a[1..n][1..n]` is the input matrix. `b[1..n][1..m]` is input containing the m right-hand side vectors. On output, `a` is replaced by its matrix inverse, and `b` is replaced by the corresponding set of solution vectors."
Write a short program using `gaussj` which will solve the following matrix equation:

$$\begin{pmatrix} 1 & 5 & 3 \\ 2 & 1 & 9 \\ 3 & 8 & 8 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 5 \\ 2 \\ 1 \end{pmatrix}$$

9.3 General Linear Least Squares

To generalize our discussion in the last section, let us go back to Equation 9.1 and suppose that our model function f may be written as a linear combination of a set of m linearly independent functions[2] $X_i(x)$ such that

$$y(x) = \sum_{k=1}^{m} a_k X_k(x) \tag{9.8}$$

where the functions $X_i(x)$ are arbitrary fixed functions of x, called the *basis functions*. We note that the functions could be $1, x, x^2, \ldots, x^{m-1}$ in which case $f(x)$ is simply a polynomial of degree $m-1$ in x. However, the $X_i(x)$ can be any set of linearly independent functions of x, such as, for instance, sines and cosines.

Our linear least squares expression for χ^2 now becomes:

$$\chi^2 = \sum_{i=1}^{N} \left[\frac{y_i - \sum_{k=1}^{m} a_k X_k(x_i)}{\sigma_i} \right]^2$$

The problem of finding the set of a_k's such that they minimize the above expression for χ^2 is called the problem of *general linear least squares*, not because the $X_k(x)$'s are linear (they generally are not!), but because the a_k's appear in Equation 9.8 in a linear way.

We proceed to solve the problem of general linear least squares in a way analogous to section 9.2. We minimize χ^2 individually with respect to each of the a_k's, yielding the following equations:

$$0 = \sum_{i}^{N} \frac{1}{\sigma_i^2} \left[y_i - \sum_{j=1}^{m} a_j X_j(x_i) \right] X_k(x_i) \quad k = 1, \ldots, m \tag{9.9}$$

If we define a matrix $\mathbf{A}$ with entries

$$A_{kj} = \sum_{i=1}^{N} \frac{X_j(x_i) X_k(x_i)}{\sigma_i^2}$$

and a vector $\mathbf{b}$ with entries

[2] A set of m functions, $X_i(x)$ is called *linearly independent* if no function $X_j(x)$ in that set may be expressed as a linear combination of the other functions in that set. Thus, the set $1, x, x^2$ is a linearly independent set, as, for instance, x may not be expressed as a sum of 1 and x^2.

$$b_k = \sum_{i=1}^{N} \frac{y_i X_k(x_i)}{\sigma_i^2}$$

then Equation 9.9 may be written as

$$\mathbf{A} \cdot \mathbf{a} = \mathbf{b}$$

where $\mathbf{a}$ has the entries $a_1, a_2, \ldots a_m$. We may thus determine $\mathbf{a}$ by inverting $\mathbf{A}$.

Of course, we are not finished until we have determined the errors for each of the a_k's. It turns out, in analogy with section 9.2, that the variances for the a_k's are given by the diagonal entries of the inverse matrix $\mathbf{A}^{-1}$, also called the *covariant matrix*. To derive the standard errors of the coefficients a_k, we must multiply these variances by the factor

$$s^2 = \chi^2/(N - M)$$

where M is the number of coefficients, and then take the square root. The number $N - M$ is the number of degrees of freedom.

Exercise 9.4

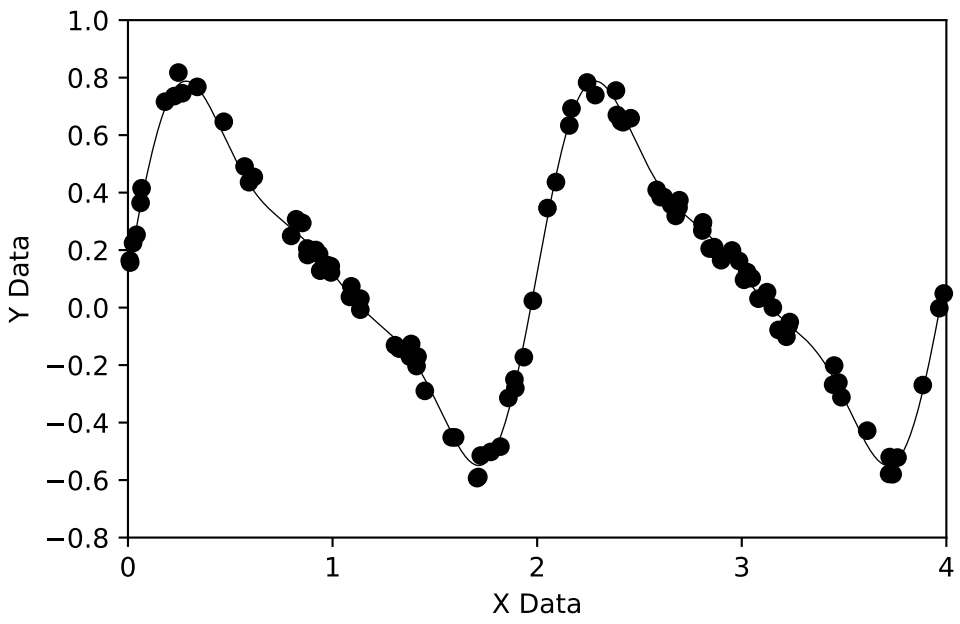
The datafile `data94.dat` contains x, y data which can be fit with a polynomial. Write a program which will fit these data to a polynomial of degree $M - 1$, where $M - 1$ is a user input. The program should output the M coefficients and their errors. Determine the best value of the degree of the polynomial for this dataset by trial and error. Hint: the best polynomial will have a small value for χ^2, but not necessarily the smallest value. If increasing the degree of the polynomial only marginally improves the value for χ^2, it may not represent a better fit. You also need to look at the standard errors of the coefficients. If the absolute value of a coefficient is no greater than 3 times its error, it is not statistically significant, i.e. it is not statistically different from zero.

Exercise 9.5

The datafile `data95.dat` contains x, y data which can be fit with a harmonic sine series of the form:

$$y = a_1 + \sum_{n=2}^{4} a_n \sin[(n-1)\pi x]$$

Determine the a_n's and their errors for this dataset.



9.4 Nonlinear Least Squares (Levenberg-Marquardt)

Forget about least squares for a moment, and think about a function f, defined in an m-dimensional space, which has a minimum. If we consider some point $\mathbf{P}$, and take that point as the origin of the coordinate system, then we can expand this function f about that point in a Taylor series:

$$f(\mathbf{x}) = f(\mathbf{P}) + \sum_i \frac{\partial f}{\partial x_i} x_i + \frac{1}{2} \sum_{i,j} \frac{\partial^2 f}{\partial x_i \partial x_j} x_i x_j + ...$$

where $\mathbf{x}$ is the vector $(x_1, x_2, \dots, x_m)$ and the derivatives are evaluated at point $\mathbf{P}$. We note the above expression may be written in the form

$$f(\mathbf{x}) \approx c - \mathbf{b} \cdot \mathbf{x} + \frac{1}{2} \mathbf{x} \cdot \mathbf{A} \cdot \mathbf{x}$$

where

$$c \equiv f(\mathbf{P}) \quad \mathbf{b} \equiv -\nabla f|_{\mathbf{P}} \quad [\mathbf{A}]_{ij} \equiv \left. \frac{\partial^2 f}{\partial x_i \partial x_j} \right|_{\mathbf{P}}$$

The matrix $\mathbf{A}$ is called the *Hessian matrix* of the function at $\mathbf{P}$. This mathematical expression is called a *quadratic form.*

Now, in this approximation, it is easy to show that

$$\nabla f = \mathbf{A} \cdot \mathbf{x} - \mathbf{b}$$

but, of course, at the minimum of this function, this gradient vanishes, and thus the position of the minimum is given by solving the matrix equation

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$$

Now, shift your mind back to the problem of Least Squares. Let us suppose that we have a function which depends on parameters $a_1, a_2, \ldots a_m$ in a non-linear way. Unfortunately, we cannot proceed in precisely the way we did in section 9.3, but we can still write down the same expression for χ^2 which we wish to minimize, that is,

$$\chi^2 = \sum_{i=1}^{N} \left(\frac{y_i - f(x_i; a_1, a_2, \ldots, a_m)}{\sigma_i} \right)^2$$

where the x_i's and the y_i's are, of course, the experimental data points we wish to fit with the function f. We may regard χ^2 as a non-linear function of the a_i's and write, according to the short discussion above,

$$\chi^2(\mathbf{a}) \approx \gamma - \mathbf{d} \cdot \mathbf{a} + \frac{1}{2} \mathbf{a} \cdot \mathbf{D} \cdot \mathbf{a}$$

If this is a good approximation, we may jump from a set of trial parameters $\mathbf{a}_{\text{cur}}$ to the minimum in one step using

$$\mathbf{a}_{\text{min}} = \mathbf{a}_{\text{cur}} + \mathbf{D}^{-1} \cdot [-\nabla \chi^2(\mathbf{a}_{\text{cur}})]$$

as we saw above. If, however, the quadratic form is *not* a good approximation to χ^2, then we can at least take a step downhill along the gradient toward the minimum (this is called the *steepest descent method*):

$$\mathbf{a}_{\text{next}} = \mathbf{a}_{\text{cur}} - \text{constant} \times \nabla \chi^2(\mathbf{a}_{\text{cur}})$$

where the constant is small enough so that one does not overshoot the minimum.

The Levenberg-Marquardt Method applies both the inverse-Hessian method and the steepest descent method to find the minimum of χ^2. Far from the minimum, where the quadratic form

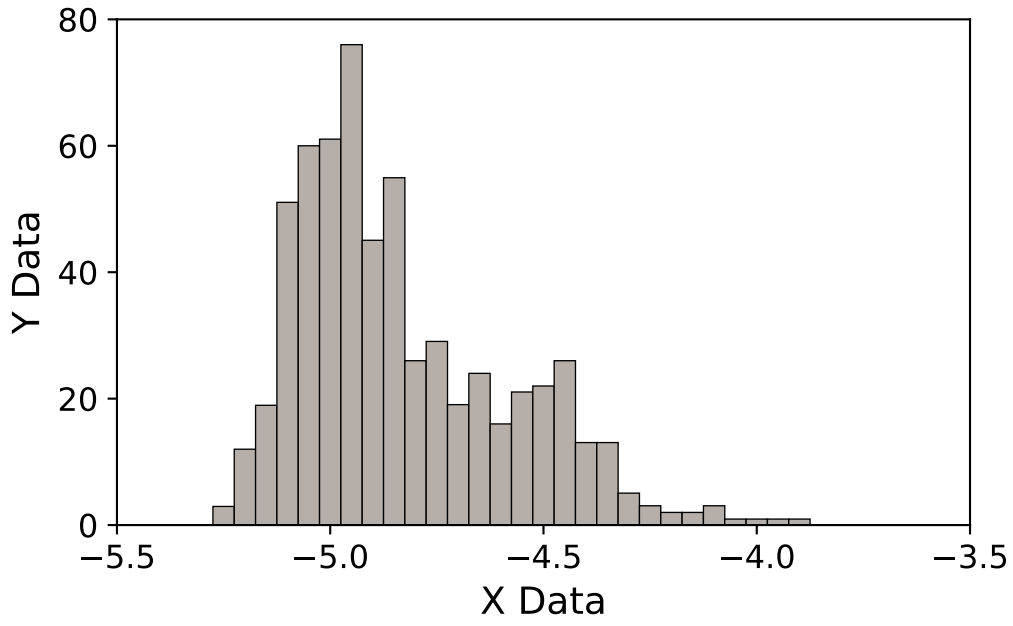
is not a good approximation, the Levenberg-Marquardt Method uses the method of steepest descent, but shifts smoothly over to the inverse-Hessian method as the minimum is getting near, and the quadratic form is a better approximation. The Levenberg-Marquardt Method also allows one to hold some of the a_i's fixed, and to vary the others. This method is implemented in the *Numerical Recipes* function `mrqmin` (found in `comphys.c`) which is declared in the following way:

```
void mrqmin(float x[], float y[], float sig[], int ndata, float a[],
int ia[], int ma, float **covar, float **alpha, float *chisq, void
(*funcs)(float, float [], float *, float [], int), float *alambda)
```

and comes with the directions: "Levenberg-Marquardt method, attempting to reduce the value χ^2 of a fit between a set of data points `x[1..ndata]`, `y[1..ndata]` with individual standard deviations `sig[1..ndata]` and a nonlinear function dependent on `ma` coefficients `a[1..ma]`. The input array `ia[1..ma]` indicates by nonzero entries those components of `a` that should be fitted for, and by zero entries those components that should be held fixed at their input values. The program returns current best-fit values for the parameters `a[1..ma]` and χ^2 = `chisq`. The arrays `covar[1..ma][1..ma]`, `alpha[1..ma][1..ma]` are used as working space during most iterations. Supply a routine `funcs(x,a,yfit,dyda,ma)` that evaluates the fitting function `yfit`, and its derivatives `dyda[1..ma]` with respect to the fitting parameters `a` at `x`. On the first call provide an initial guess for the parameters `a`, and set `alambda < 0` for initialization (which then sets `alambda = .001`). If a step succeeds `chisq` becomes smaller and `alambda` decreases by a factor of 10. If a step fails `alambda` grows by a factor of 10. You must call this routine repeatedly until convergence is achieved. Then, make one final call with `alambda = 0`, so that `covar[1..ma][1..ma]` returns the covariance matrix, and `alpha` the curvature matrix. (Parameters held fixed will return zero covariances)."

The constant `alambda` in the above discussion refers to the constant in the method of steepest descent.

The best way to explain how to use `mrqmin` is with an example. The following figure shows a histogram of data. The underlying theory is that the distribution of points in this diagram is a double Gaussian. We would like to fit these data to a function that looks like:



$$f(x; a_1, \ldots, a_6) = a_1 \exp\left[-\left(\frac{x - a_2}{a_3}\right)^2\right] + a_4 \exp\left[-\left(\frac{x - a_5}{a_6}\right)^2\right]$$

To apply the Levenberg-Marquardt Method, we must obtain the derivatives of f with respect to all of the a_i's. These derivatives are as follows:

$$\frac{\partial f}{\partial a_1} = \exp\left[-\left(\frac{x - a_2}{a_3}\right)^2\right]$$
$$\frac{\partial f}{\partial a_2} = 2a_1\left(\frac{x - a_2}{a_3}\right)\exp\left[-\left(\frac{x - a_2}{a_3}\right)^2\right]$$
$$\frac{\partial f}{\partial a_3} = 2a_1\left(\frac{x - a_2}{a_3}\right)^2\exp\left[-\left(\frac{x - a_2}{a_3}\right)^2\right]$$

and similarly for a_4, a_5 and a_6.

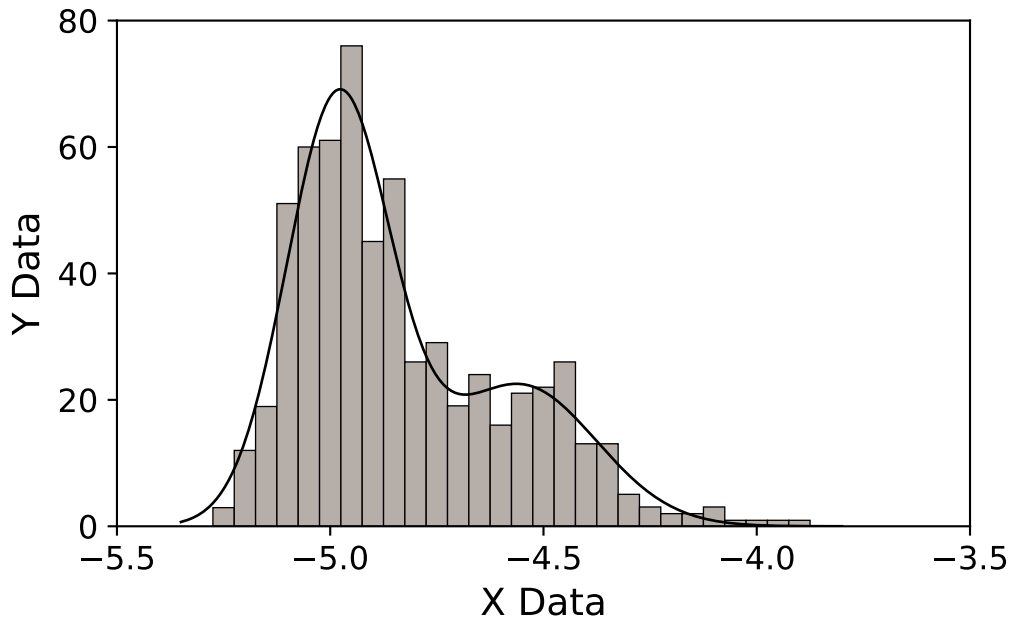
The C-code which performs this miracle is as follows:

```
#include <stdio.h>
#include <math.h>
#include "comphys.h"
#define NDATA 2000
```

```
#define MA 6
void fgauss(float x, float a[], float *y, float dyda[], int na);

int main()
{
  float *x,*y,*sig,**covar,**alpha,*a,chisq,ochisq,alambda;
  int *ia,ma;
  char name[80],initname[80],outname[80];
  FILE *in,*init,*out;
  int i,k,ndata;

  x = vector(1,NDATA);
  y = vector(1,NDATA);
  sig = vector(1,NDATA);
  a = vector(1,MA);
  covar = matrix(1,MA,1,MA);
  alpha = matrix(1,MA,1,MA);
  ia = ivector(1,MA);

  /* Datafile contains data which are to be fitted to a double
     Gaussian.  Initial parameters file contains rough estimates
     of Gaussian parameters. */

  ma = MA;
  printf("Enter name of datafile > " );
  scanf("%s",name);
  if((in = fopen(name,"r")) == NULL)
    nrerror("Cannot find input file");
  printf("Enter initial parameters datafile > ");
  scanf("%s",initname);
  if((init = fopen(initname,"r")) == NULL)
    nrerror("Cannot find initial parameters file");
  printf("Enter output file > ");
  scanf("%s",outname);
  out = fopen(outname,"w");

  /* Reads in data from datafile.  Defines sigma errors by taking
     square roots of y-values, as these are counts.  Adds a small
     number (0.1) so that zero values of y will not have 0 errors */

  k = 1;
  while(fscanf(in,"%f %f",&x[k],&y[k]) != EOF) {
```

```
    sig[k] = sqrt(y[k]) + 0.1;
    k++;
  }
  ndata = k-1;

  /* Reads in initial estimates for parameters */
  for(i=1;i<=4;i+=3) fscanf(init,"%f,%f,%f",&a[i],&a[i+1],&a[i+2]);

  /* We are fitting all parameters a[1] .. a[6], so we fill up all
     entries in ia[] with 1's */
  for(i=1;i<=MA;i++) ia[i] = 1;

  /* Initializing mrqmin using a negative value of alambda.
     Note we are passing the function fgauss to mrqmin.  See
     below. */
  alambda = -1.0;
  chisq = 0.0;
  mrqmin(x,y,sig,ndata,a,ia,ma,covar,alpha,&chisq,fgauss,&alambda);
  ochisq = chisq + 2.0;

  /* Iterating with mrqmin until relative change in chisq is less
     than 0.001 */
  while(fabs((ochisq - chisq)/ochisq) > 0.001 ) {
    ochisq = chisq;
    mrqmin(x,y,sig,ndata,a,ia,ma,covar,alpha,&chisq,fgauss,&alambda);
  }

  /* Here we run mrqmin one more time with alambda = 0.0 to get
     covariant matrix, in case we want the errors */
  alambda = 0.0;
  mrqmin(x,y,sig,ndata,a,ia,ma,covar,alpha,&chisq,fgauss,&alambda);

  /* Print out the results (the a's) along with the errors (the
     squareroots of the diagonal members of the covariant matrix
     divided by the number of degrees of freedom). */
  for(i=1;i<=MA/2+1;i+=MA/2) fprintf(out,"%f %f\n%f %f\n%f %f\n",
    a[i],sqrt(chisq*covar[i][i]/(float)(ndata-MA)),
    a[i+1],sqrt(chisq*covar[i+1][i+1]/(float)(ndata-MA)),
    a[i+2],sqrt(chisq*covar[i+2][i+2])/(float)(ndata-MA));
  return(0);
}
```

```
/* For a given value of x, and input parameters a[], this function
   returns the resulting value of y and the vector dyda[] which
   contains the derivatives of y with respect to all of the a[]
   parameters */

void fgauss(float x, float a[], float *y, float dyda[], int na)
{
  int i;
  float fac,ex,arg;

  *y = 0.0;
  for(i=1;i<=na-1;i+=3) {
    arg = (x-a[i+1])/a[i+2];
    ex=exp(-arg*arg);
    fac=a[i]*ex*2.0*arg;
    *y += a[i]*ex;
    dyda[i]=ex;
    dyda[i+1]=fac/a[i+2];
    dyda[i+2]=fac*arg/a[i+2];
  }
  return;
}
```



Exercise 9.6

Using the data in the file `decay.out` (see chapter 6), write a program which uses the Levenberg-Marquardt method to fit the four parameters A, γ, B and C. Compare the values you get with the Simplex and Powell methods.

References

Knuth, Donald E. 1984. "Literate Programming." *Comput. J.* (USA) 27 (2): 97–111. https://doi.org/10.1093/comjnl/27.2.97.

Press, William H., Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. 1992. *Numerical Recipes in c (2nd Ed.): The Art of Scientific Computing*. Cambridge University Press.